ARCHITECTURE DES RESEAUX INFORMATIQUES

Ecole Centrale de Lille
B.P. 48
59651 Villeneuve d'Ascq Cedex
FRANCE
Tel : (33)(0)3 20 33 54 04 (13)
Fax : (33) (0)3 20 33 54 18
samir.elkhattabi@ec-lille.fr

PLAN DU COURS

Fondements
Normes
Réseaux Locaux (support distinct)
Internet Protocol
Protocoles de Transport
Interface de Programmation

Système de multitraitement

On distingue 3 classes de systèmes de multitraitement :

- ➢ Architectures massivement parallèle (d < 1 cm) :
  - ❒ Composées de nombreux processeurs séquentiels et composants mémoire : application à grande capacité de calcul
  - ❒ Single Instruction, Multiple Data
  - ❒ Multiple Instruction, Multiple Data
- ➢ Systèmes multiprocesseurs (d < 1 m) :
  - ❒ Présence de plusieurs processeurs sur une même architecture avec des mémoires propres à chaque processeur et une mémoire partagée.
  - ❒ Ces processeurs sont reliés par un bus parallèle.
- ➢ Réseaux d'ordinateurs :
  - ❒ Ordinateurs individuels (architecture, système) reliés par un réseau
  - ❒ d < 200 m : Réseau local
  - ❒ d < 1 km : Réseau local
  - ❒ d < 10 km : Réseau de campus
  - ❒ d < 100 km : Réseau métropolitain
  - ❒ d < 1000 km : Réseau grande distance
  - ❒ d > 1000 km : Interconnexion de réseaux grande distance

Usage des Réseaux d'ordinateurs

Applications professionnelles

Partage de ressources autant matérielles que logicielles

Modèle Client/Serveur

Service offert

Requête Client / Réponse Serveur

Applications domestiques

Accès à des Informations distantes

Communication entre processus

Divertissements interactifs

Commerce électronique

- **B2C**
- **B2B**
- **G2C**
- **C2C**
- **P2P**

Utilisateurs Nomades

Portable / PDA / Smartphone

WIFI / Edge / UMTS

Classification des réseaux

Technologie de transmission

- **diffusion**
- **point à point**

Etendue géographique :

- **PAN**
- **LAN**
- **MAN**
- **WAN**

Débit

- **Haut débit**

Réseau à diffusion

Réseau point à point

Classification par étendue géographique

Une classification des réseaux selon la distance inter-sites :

- PAN ou Personal Area Network
  - Une étendue géographique très courte (d < 10 m)
  - Interconnexion des équipements personnels à un ordinateur
  - Exemples : BlueTooth
- LAN ou Local Area Network
  - Une étendue géographique courte (d < 1 km) et un débit élevé (10 à 100 Mb/s)
  - Gestion centralisée par une organisation unique
  - Exemples : Ethernet, Token Ring, Token Bus, Map
- MAN ou Metropolitan Area Network
  - Une étendue géographique moyenne et limitée (d < 10 km) et un débit élevé
  - Interconnexion de réseaux locaux
  - Exemple : protocole DBDQ
- WAN ou Wide Area Network
  - Une grande distance (en centaines de Km) et un débit élevé
  - Utilisation des supports de télécommunications existants.
  - Maillage d'interconnexion basé sur des nœuds de routage
  - Exemples : RNIS, Frame Relay, ATM, Arpanet, Transpac

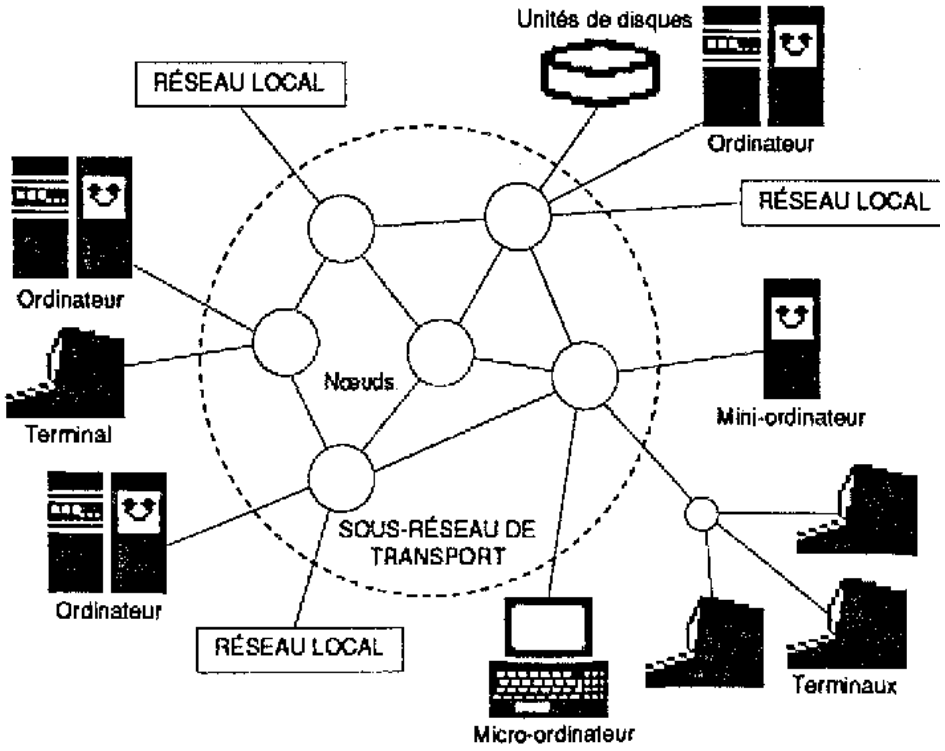
Classification par débit

Une tendance de classification des réseaux selon le débits est donnée :

- faibles débits (D < 20 kb/s)
- hauts débits (20 kb/s < D < 20 Mb/s)
- très hauts débits (D> 20 MB/s)

Les réseaux : Définition

On désigne par réseau informatique, un ensemble d'équipements informatique comprenant, entre autres des terminaux, des ordinateurs, des imprimantes, interconnectés par des liaisons de communication.



<u>Fonctions</u> : Traiter & Transporter l'information.

Historique

Fin 1969, début des travaux sur System Network Architecture par IBM.

Début des années 70 : le DoD lance ARPANET, réseau interconnectant les centres de recherche américains.

Le succès d'ARPANET a poussé au développement des réseaux locaux :

- ➢ En 75 ETHERNET est proposé par Xerox et Intel
- ➢ En 82, IBM commercialise Token Ring

En 85, la norme IEEE 802 sur les LAN fut publiée : 100 000 LAN ont été installés durant les années 80.

INTERNET (inspiré de ARPANET) a répondu présent aux besoins d'interconnexion des réseaux locaux hétérogènes.

Intégration et numérisation de RNIS (1990)

Réseaux hauts débits : ATM, RNIS large bande (1995)

Objectifs du cours

Présenter une vue générale des réseaux considérés comme instrument de partage de ressources, d'augmentation des performances, de tolerance aux pannes de communication dans l'entreprise.

Problématique liée à la communication :

- ➢ Un utilisateur (personne, terminal, programme, …) veut envoyer un message à un autre utilisateur
- ➢ Primitives de base : Envoyer(mess, dest) et Recevoir(mess,exp)

Contraintes :

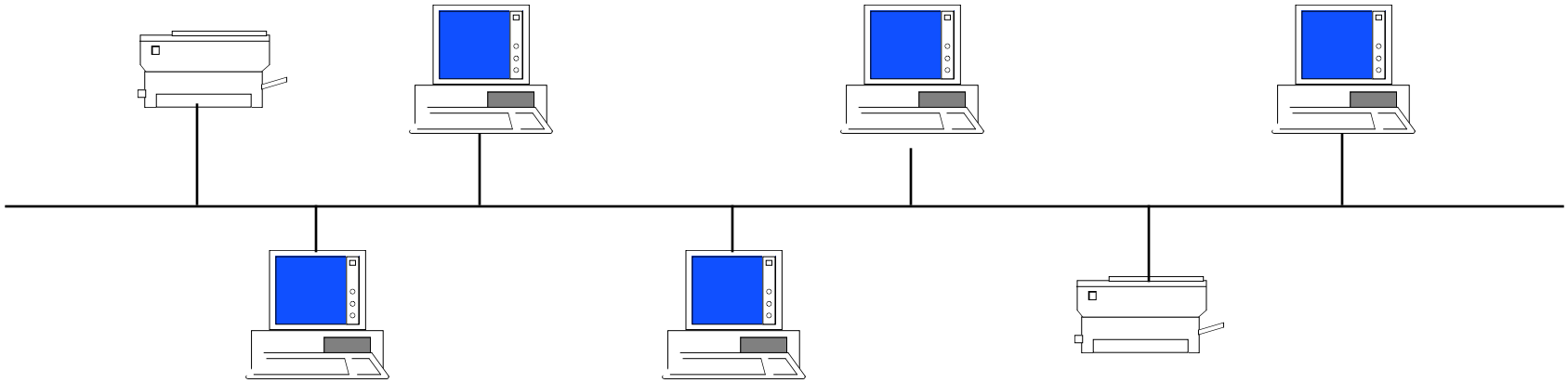
- ➢ Les informations doivent parvenir au bon destinataire
- ➢ Les informations doivent être entières et correctes
- ➢ Les informations doivent être livrées dans l'ordre d'émission

Caractériser fonctionnellement et techniquement les divers composants d'un réseau.

Type de liaison phyique



liaison point à point



liaison multipoint

Transformation de l'information

Les deux interlocuteurs d'un dialogue doivent être d'accord sur les procédures d'échange.

En général, un format est défini pour structurer l'échange permettant ainsi l'incorporation de contrôles afin de vérifier la validité de l'échange.

La forme du transfert est également définie : par bit, par paquet.

Une conversion parallèle-série est nécessaire à l'émission comme à la réception.

En effet, un ordinateur traite les bits sous forme de mots alors que la transmission se fait en série.

Le transfert d'informations numériques sur des supports analogiques nécessite également une conversion. On parle de modulation en émission et de démodulation en réception.

Liaison commutée ou permanente

Liaison permanente : permet une communication à tout instant. Elles sont utilisées dans le cadre de réseaux locaux

La retransmission pour reprise d'erreurs n'a pas de coût supplémentaire.

Commutation : les liaisons commutées permettent d'optimiser l'utilisation des moyens de communication et proposé en général dans le cadre d'une interconnexion d'un LAN.

La commutation nécessite une certaine signalisation (sélection, adressage, établissement, libération), activité qui peut prendre plus de temps que la communication effective. Dans ce contexte, pour des raisons économiques, on privilégiera les liaisons permanentes.

Deux familles de commutation : commutation de circuits et commutation de données

Mode d'acheminement

Mode connecté ou non connecté

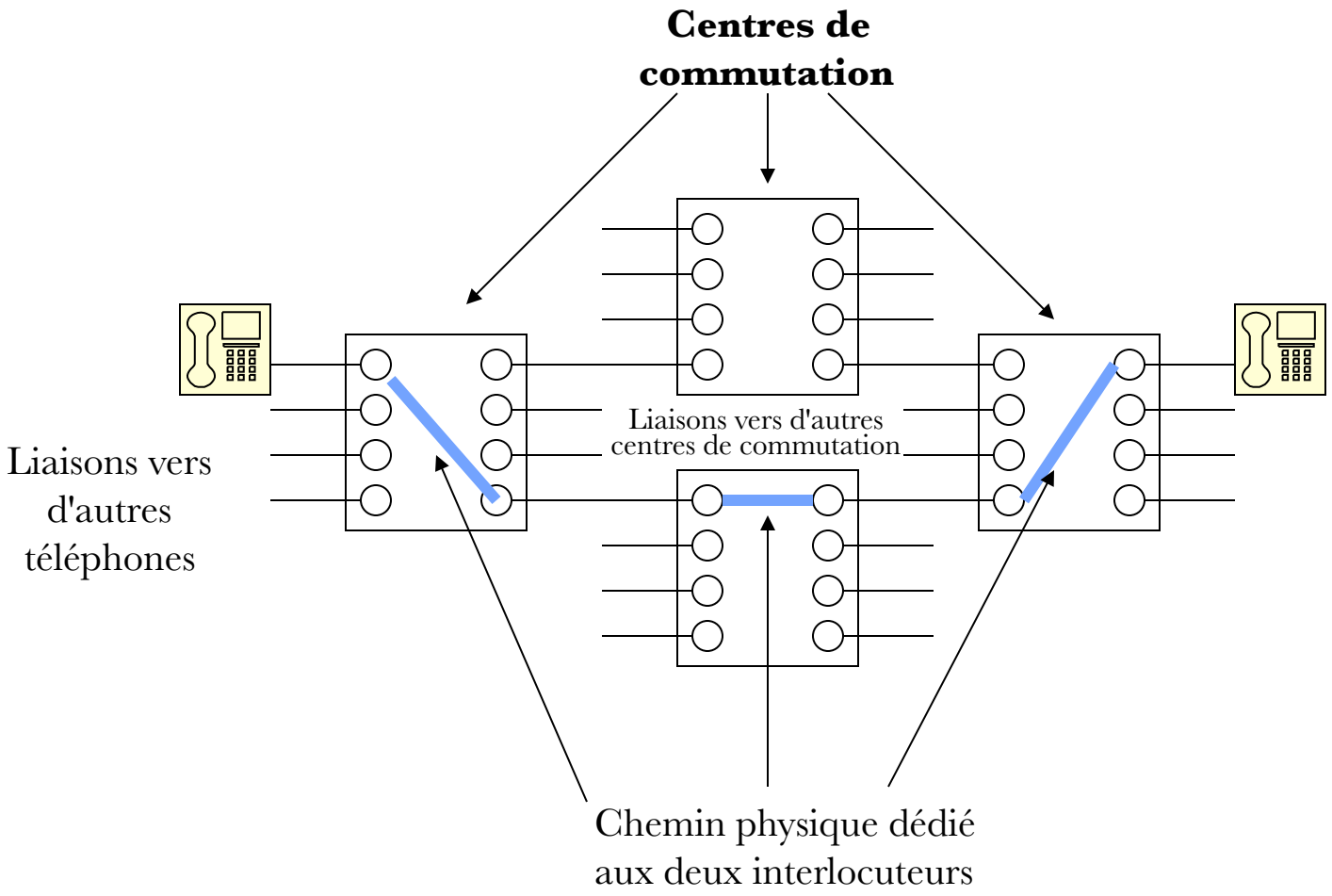
- ➢ Si la liaison est permanente, on peut choisir :
  - ❒ le mode connecté : nécessite un protocole (connexion, échange, libération),
  - ❒ le mode non connecté : on transmet directement sans protocole avec arrivée éventuelle dans le désordre
- ➢ Le mode non connecté est plus rapide, plus simple mais moins fiable.

Liaison	Mode
Commutée	Connecté
Permanente	Connecté
Permanente	Non connecté

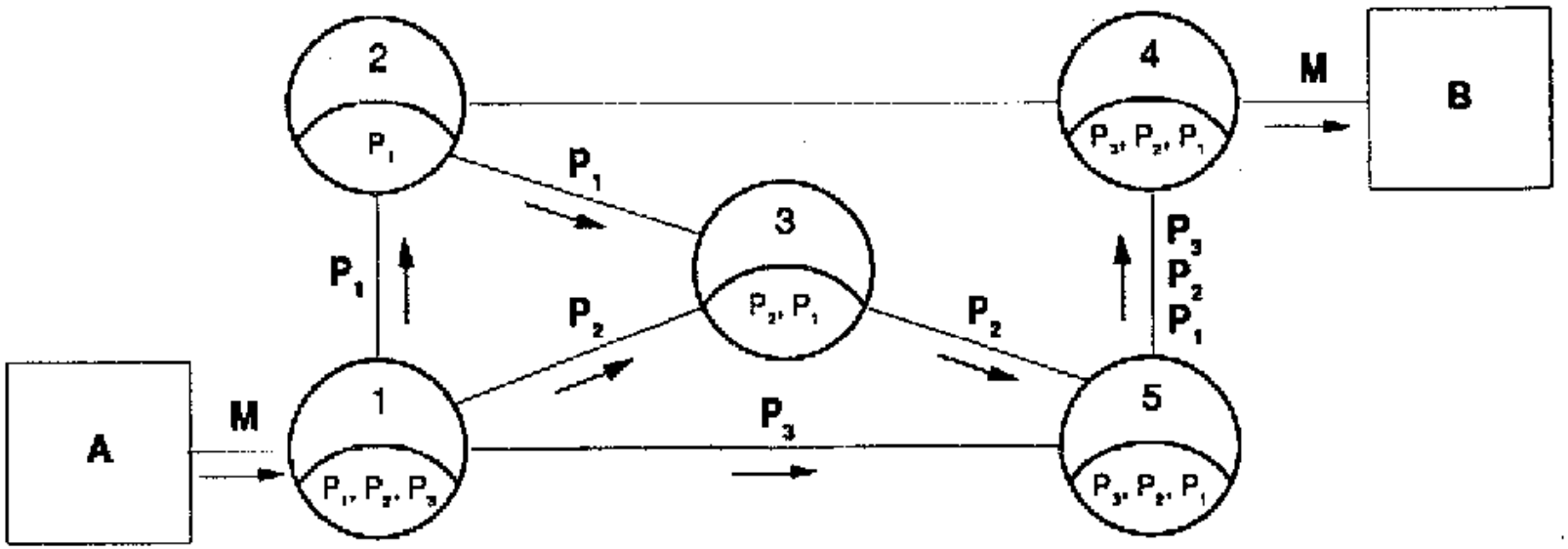
Type de transmission :

- ➢ full duplex : Transmission simultanée en émission-réception par les deux hôtes.
- ➢ half-duplex : Transmission à tour de rôle

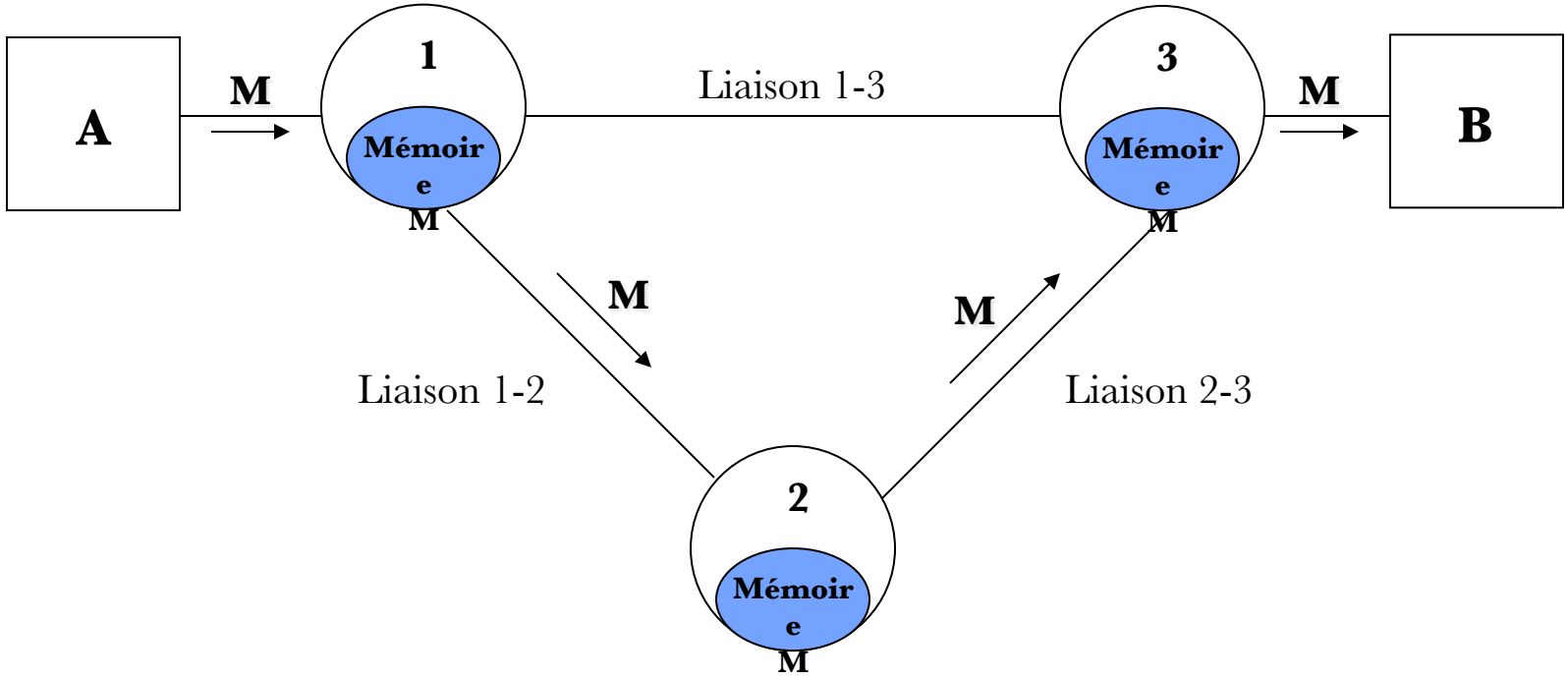
Commutation de circuits



Commutation de paquets



Commutation de messages

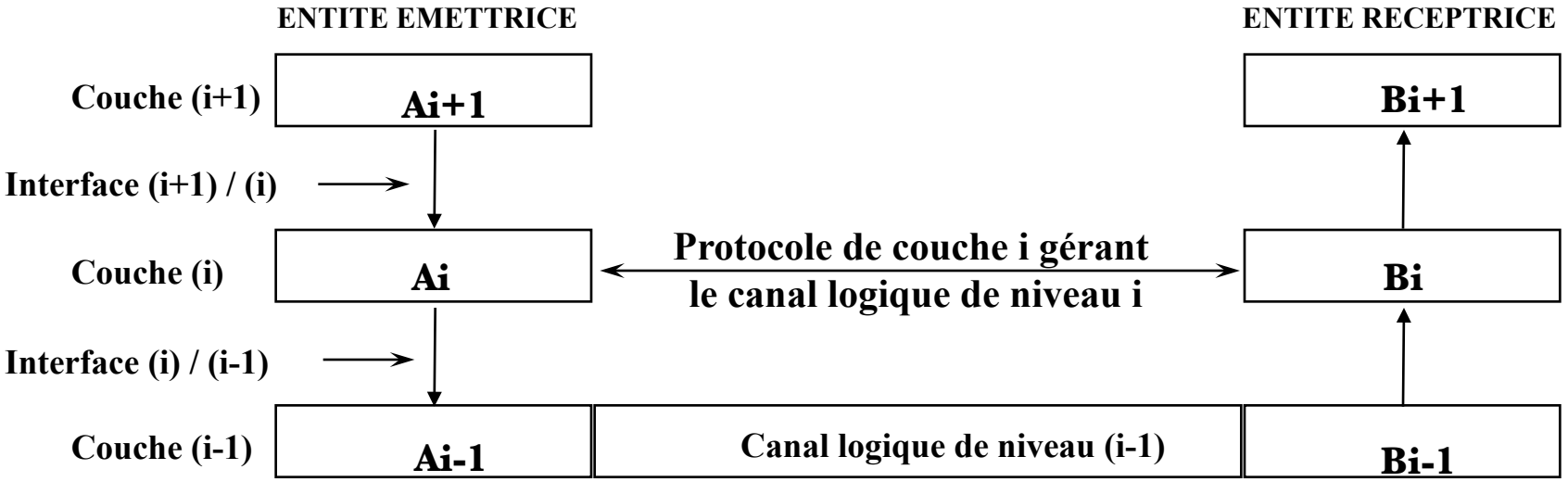


Logiciel de réseau : Hiérarchie de protocoles

Pour réduire la complexité de la conception, les réseaux sont organisés en couches.

Chaque couche étant construite sur la précédente. Une couche offre des services aux couches supérieures (par le biais de la couche adjacente) sans qu'elle donne le détail de l'implémentation.

Principe d'une architecture de réseau en couches :



Interfaces et protocoles

Une couche a pour rôle de fournir des services à la couche située directement au-dessus.

Une couche est constituée par des entités matérielles (ex : puce d'E/S) ou logicielles (ex : un processus). Les Entités paires sont des entités de même niveau sur deux machines différentes.

La couche n est fournisseur de service pour la couche n+1 qui est utilisateur de service.

Les services sont accessibles par des SAP (Service Access Point). Chaque SAP a une adresse unique (ex : téléphone).

Un *Protocole de communication* est un ensemble de règles ou de conventions qui gèrent le dialogue et l'échange d'information entre deux couches de même niveau, à partir d'interface de communication.

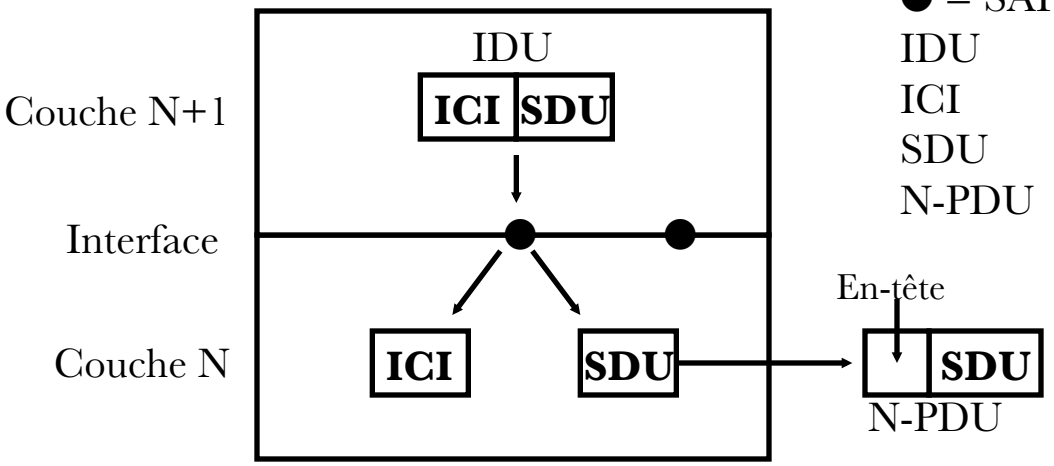
Remarque : Un protocole définit des interactions entre deux processus analogues ou remplissant des fonctions similaires tandis qu'une interface contrôle les interactions entre deux processus différents.

Le modèle OSI (1)

Le modèle OSI est un modèle en couches élaboré par l'ISO pour l'Interconnexion des Systèmes Ouverts (Open Systems Interconnection)

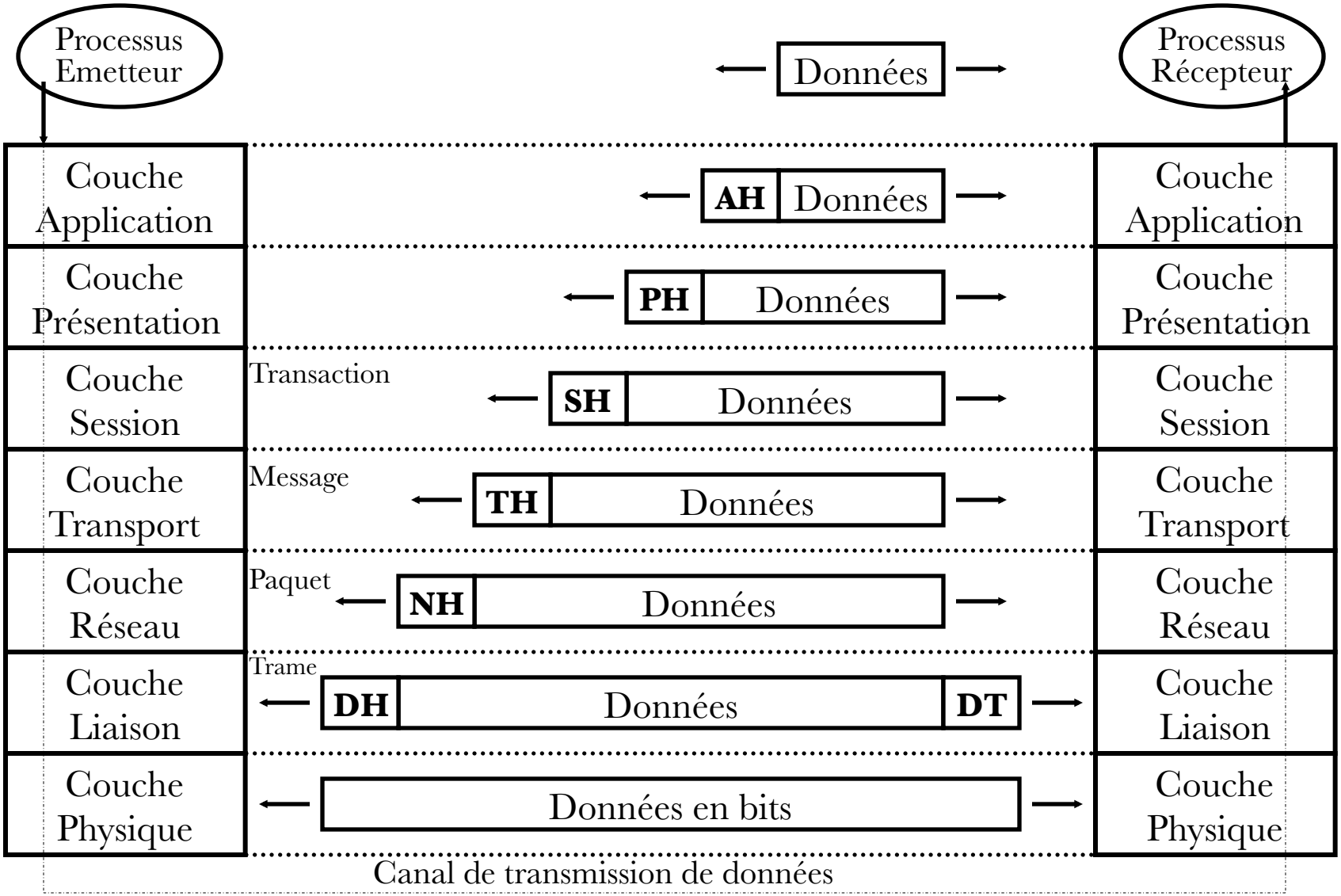
- ➢ **couches basses (1, 2, 3)** : elles sont responsables du **transport des informations**,
- ➢ **couches hautes (4, 5, 6, 7)** : elles gèrent les informations pour les **systèmes informatiques connectés**.

Principe de fonctionnement du modèle OSI : Norme X200

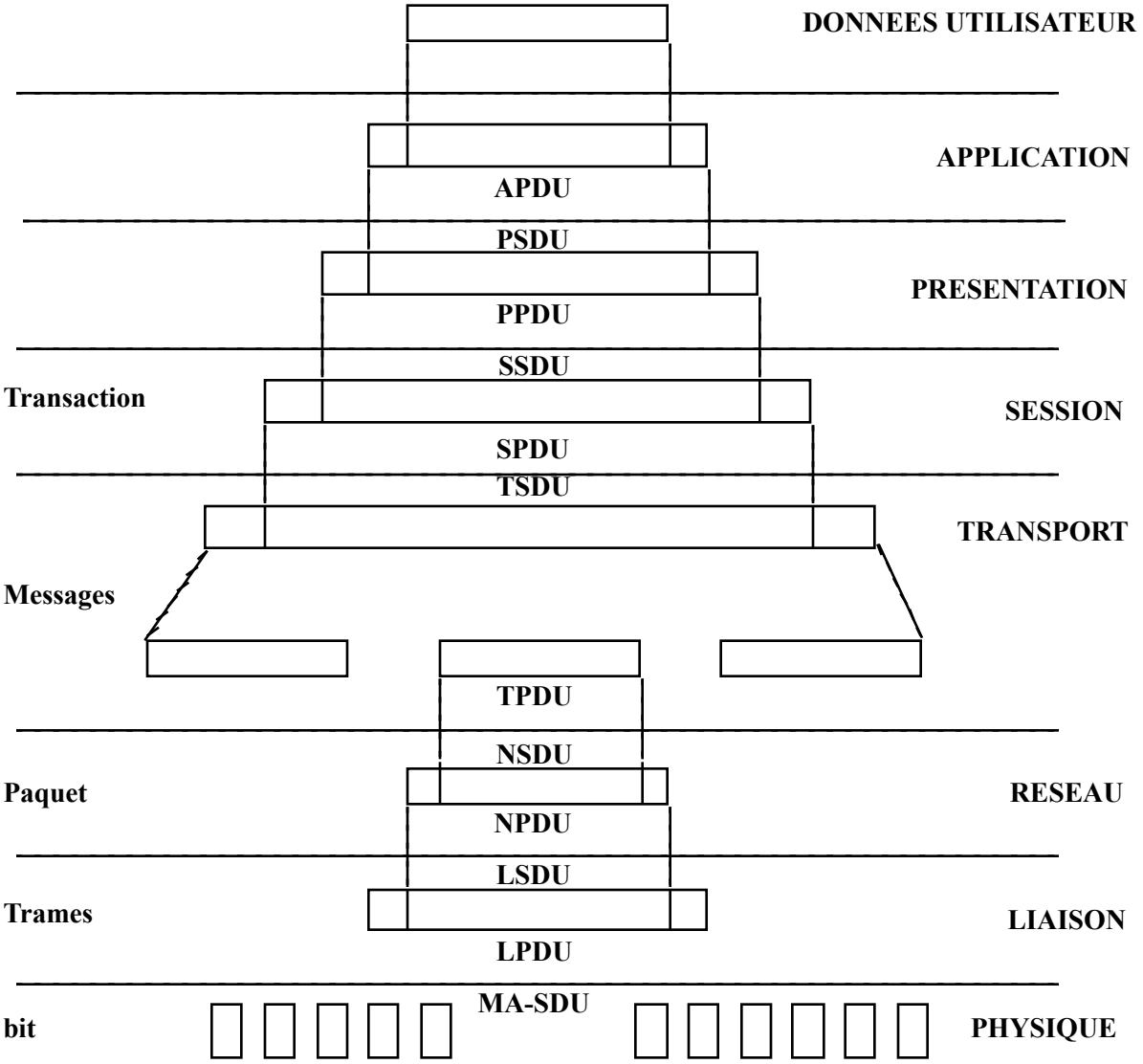


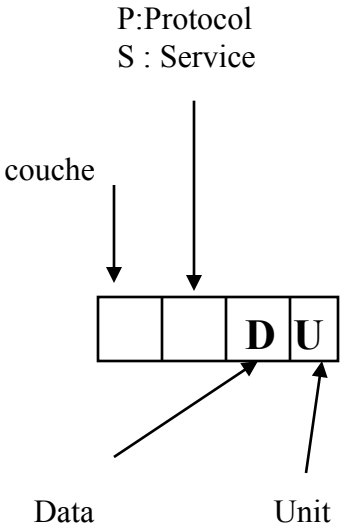
● = SAP : Service Acces Point
IDU : Interface Data Unit
ICI : Interface Control Information
SDU : Service Data Unit
N-PDU : N-Protocol Data Unit

Architecture des réseaux : Le modèle OSI (2)



Architecture des réseaux : Le modèle OSI (2b)



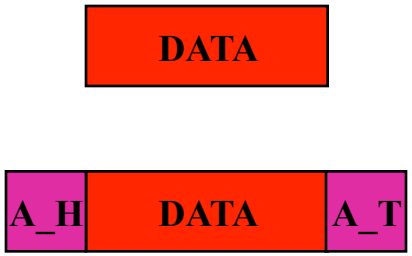


MA: Medium Access

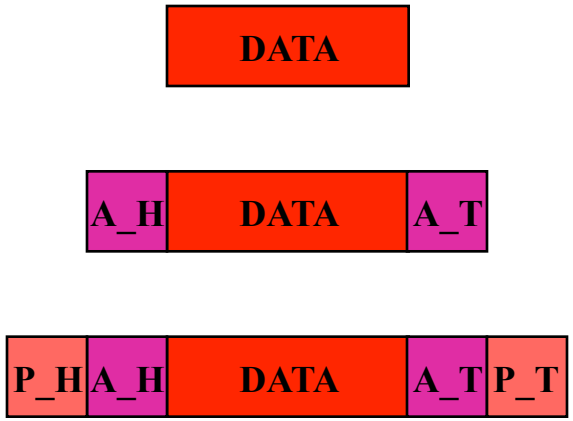
Architecture des réseaux : Le modèle OSI (2-1)



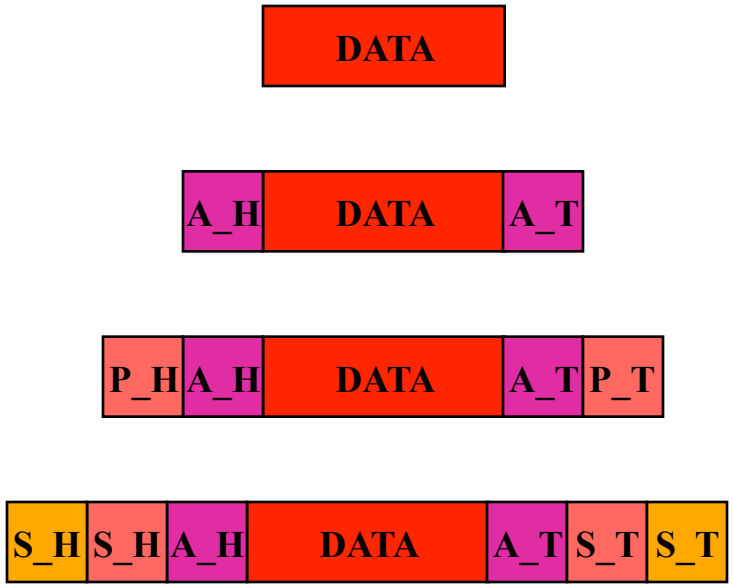
Architecture des réseaux : Le modèle OSI (2-2)



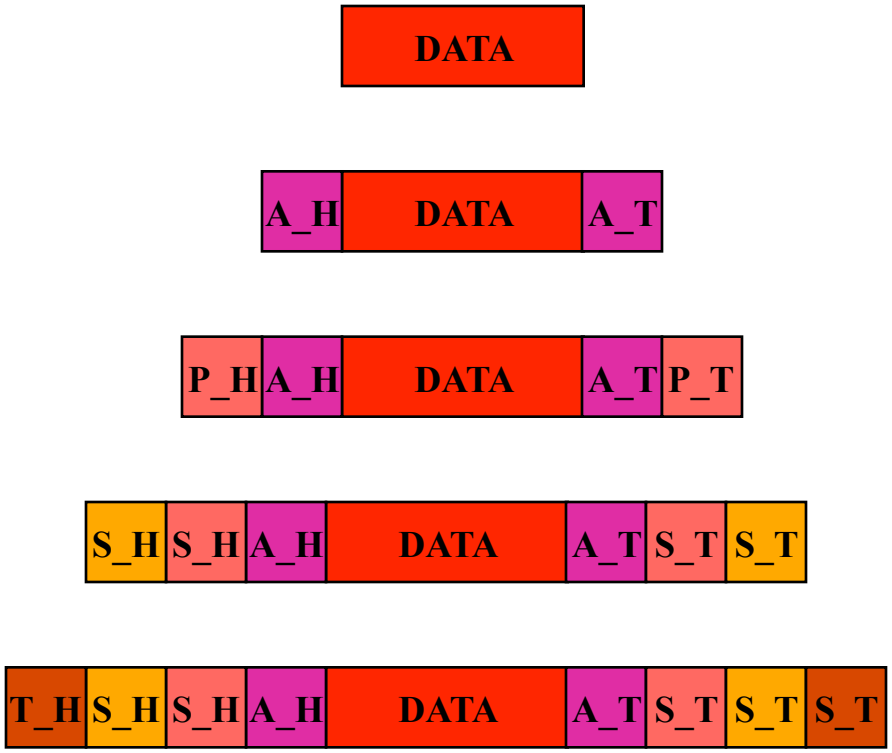
Architecture des réseaux : Le modèle OSI (2-3)



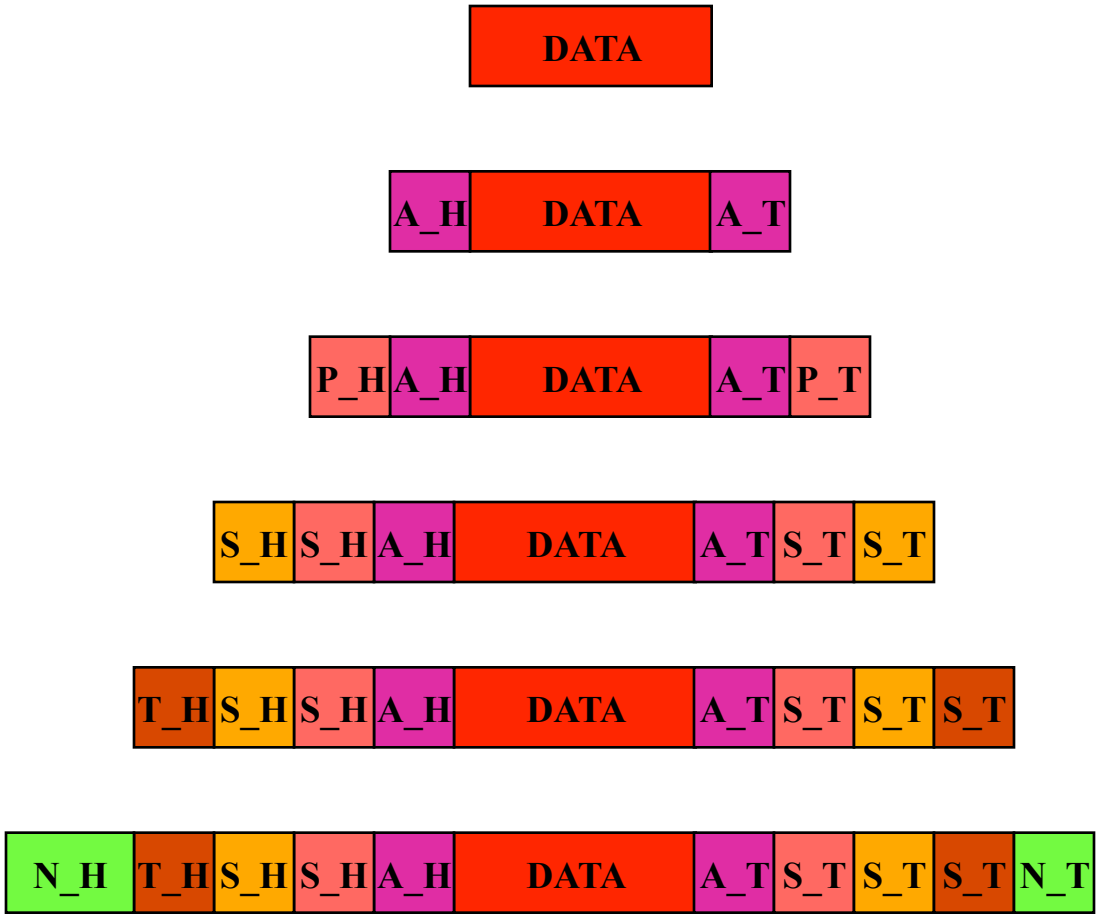
Architecture des réseaux : Le modèle OSI (2-4)



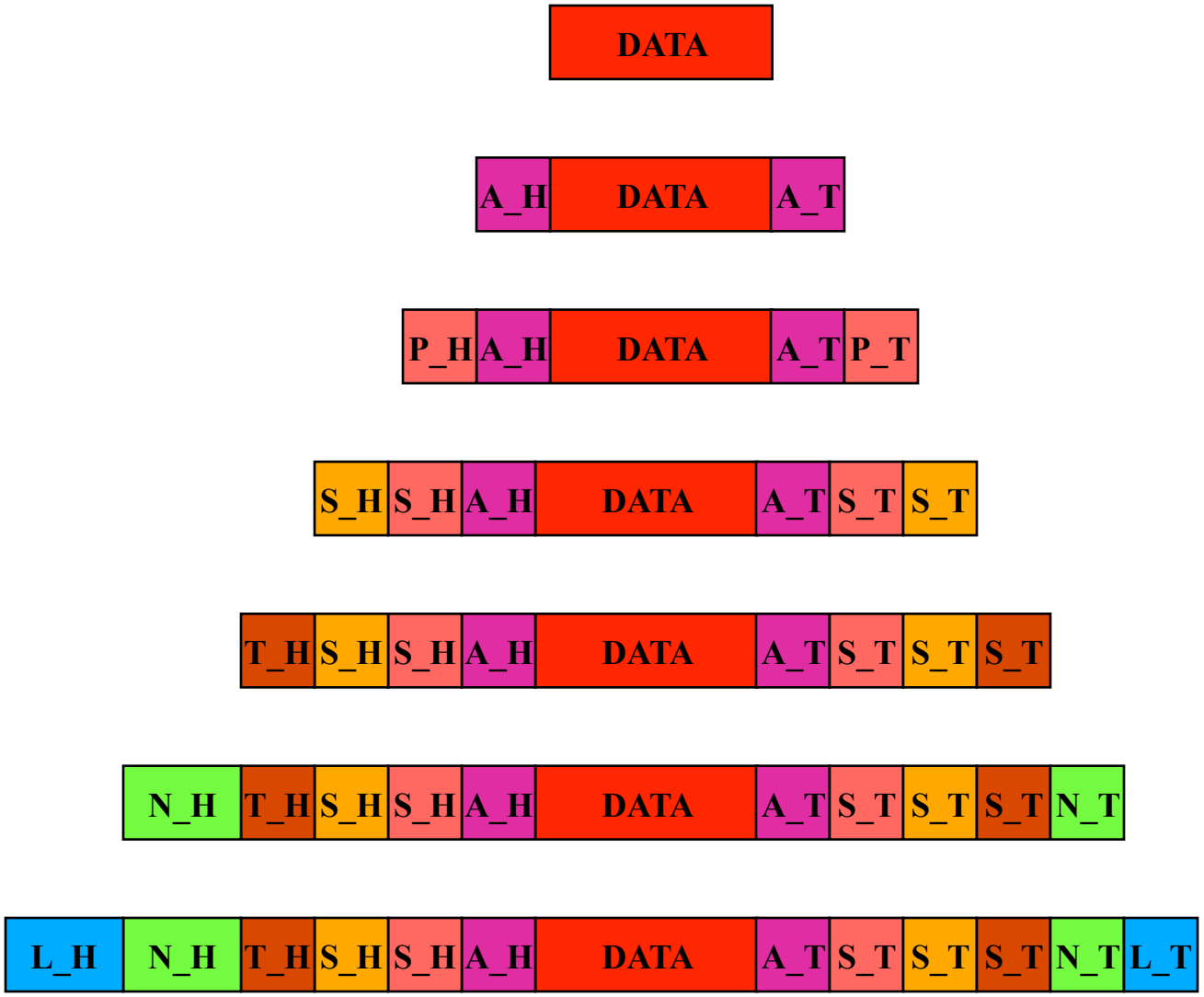
Architecture des réseaux : Le modèle OSI (2-5)



Architecture des réseaux : Le modèle OSI (2-6)



Architecture des réseaux : Le modèle OSI (2-7)



Architecture des réseaux : Le modèle OSI (3)

Couche physique (1) :

- ➢ Emission et réception des trains de bits,
- ➢ Spécification fonctionnelles, mécaniques (nombre de broches), électriques (Voltage, délai).
- ➢ Procédures d'établissement, de maintien et de libération du circuit de données

Couche liaison (2) :

- ➢ Emission et réception de trames, avec gestion de l'acquittement,
- ➢ Détection et reprise en cas d'erreurs (perte, collision, duplication),
- ➢ Contrôle d'accès au médium de communication

Couche réseau (3) :

- ➢ commutation et acheminement des paquets, à travers des passerelles d'interconnexion
- ➢ ordonnancement des paquets de données.

Architecture des réseaux : Le modèle OSI (4)

Couche transport (4) :

- ➢ Segmentation et ré-assemblage des messages en paquets
- ➢ fiabilité de transmission des messages d'un bout à l'autre du réseau,
- ➢ mise en séquence des messages.
- ➢ multiplexage et contrôle de flux,

Couche session (5) :

- ➢ gestion de l'interaction entre deux utilisateurs distants (translation d'une adresse logique en tâches réparties),
- ➢ connexion point à point, pas de multiplexage,
- ➢ synchronisation du transfert pour reprise

Couche présentation (6) :

- ➢ transformation de la syntaxe des données en fonction des langages, des gabarits de données, des codes - ASCII, ISO, EBDIC, BAUDOT - utilisés par les programmes,
- ➢ conversion de la représentation interne en représentation réseau normalisée
- ➢ Cryptage et compression de données

Couche application (7).

Internet : Historique

Premiers travaux sur les technologies d'interconnexion lancés par la DARPA (Defense Advanced Research Project Agency) mi-70 → réseau d'expérimentation ARPANET

➢ OBJECTIF : interconnecter des équipements et des réseaux hétérogènes.

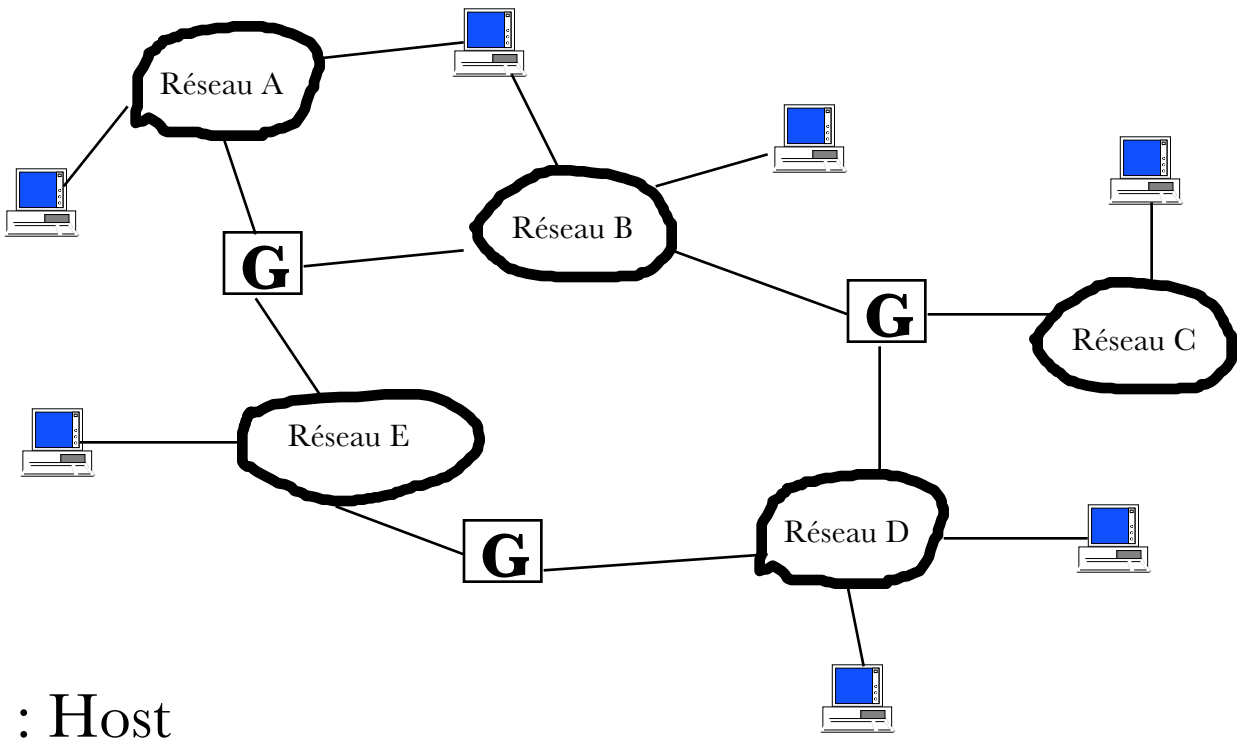
Début INTERNET vers 1980 avec adoption de TCP/IP en 1983

Intégration de TCP/IP à Unix 4 BSD → diffusion dans les universités

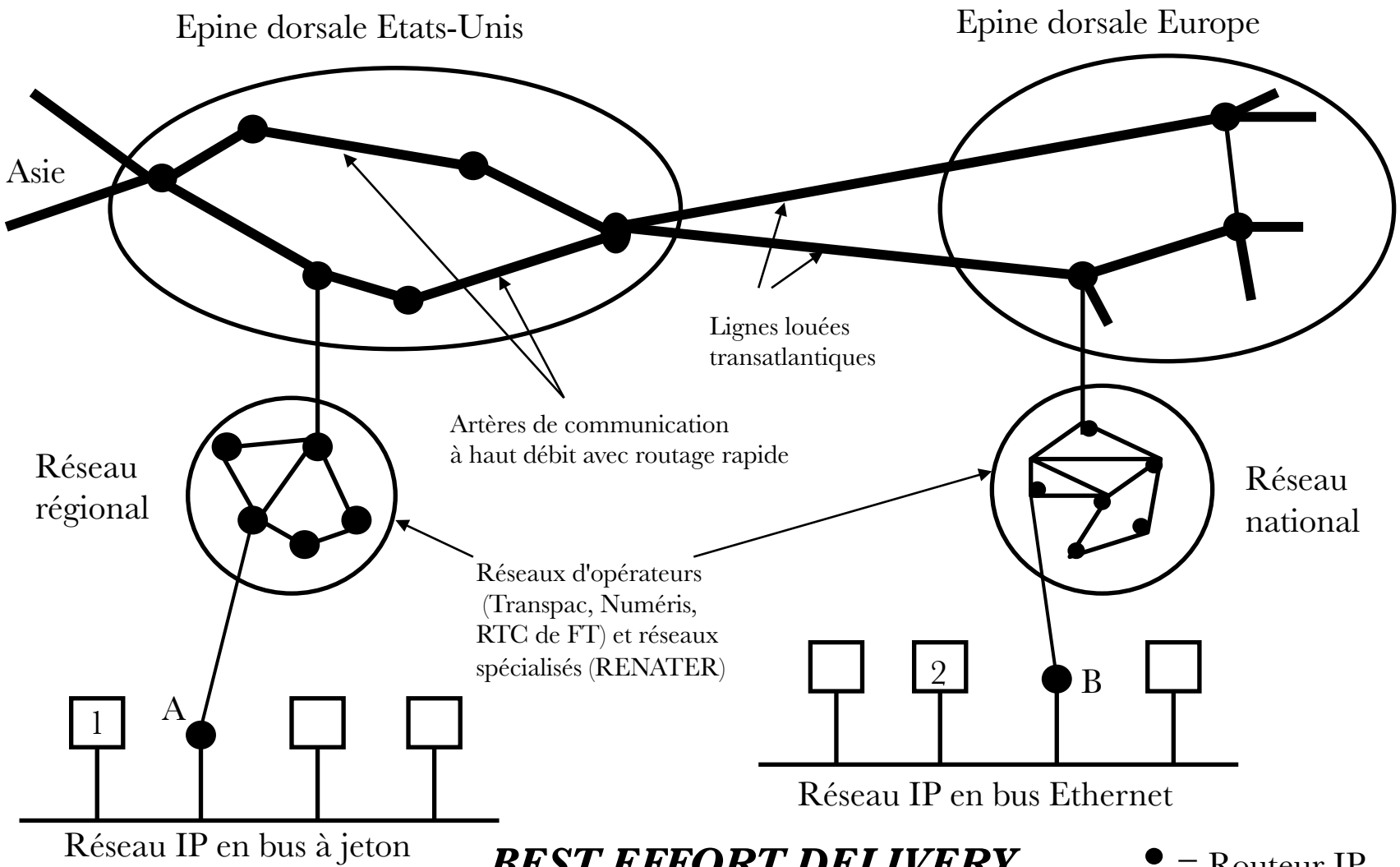
1993 : Explosion d'INTERNET aux USA puis en EUROPE.

Schéma général d'Internet

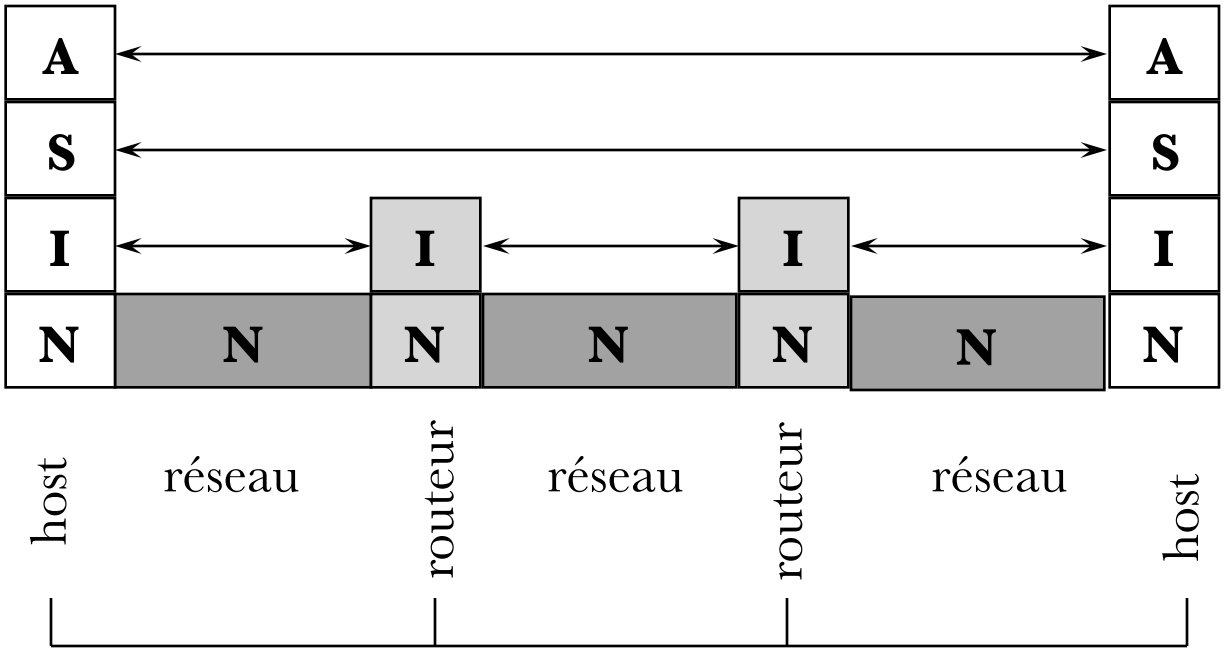
INTERconnection of NETworks = RESEAU DE RESEAUX (pas de limite géographique)

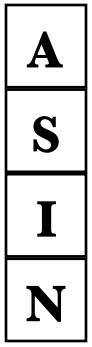


Organisation simplifiée de l'Internet



Couches de l'Internet





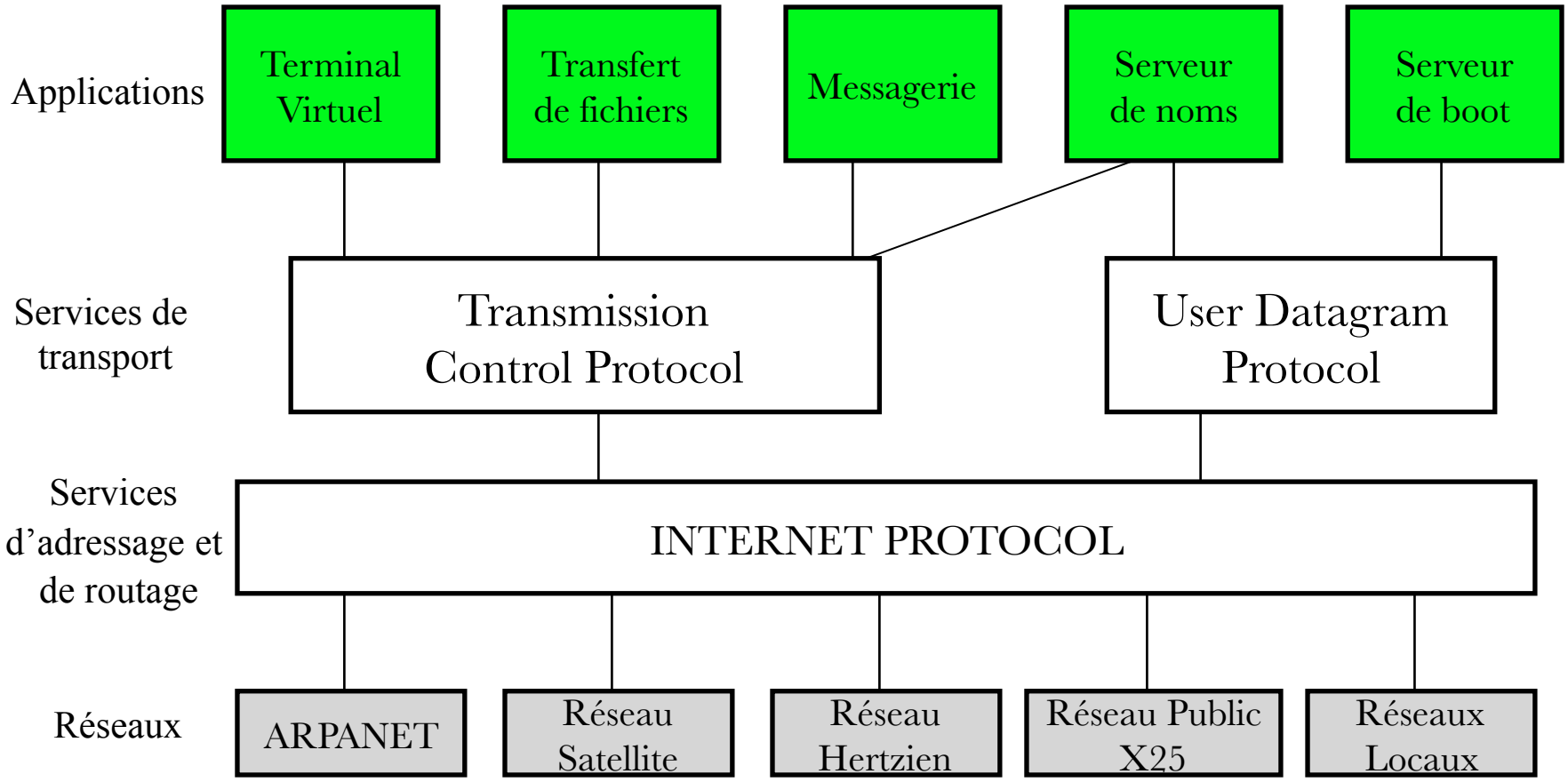
: APPLICATION PROTOCOLS

: SERVICE PROTOCOLS

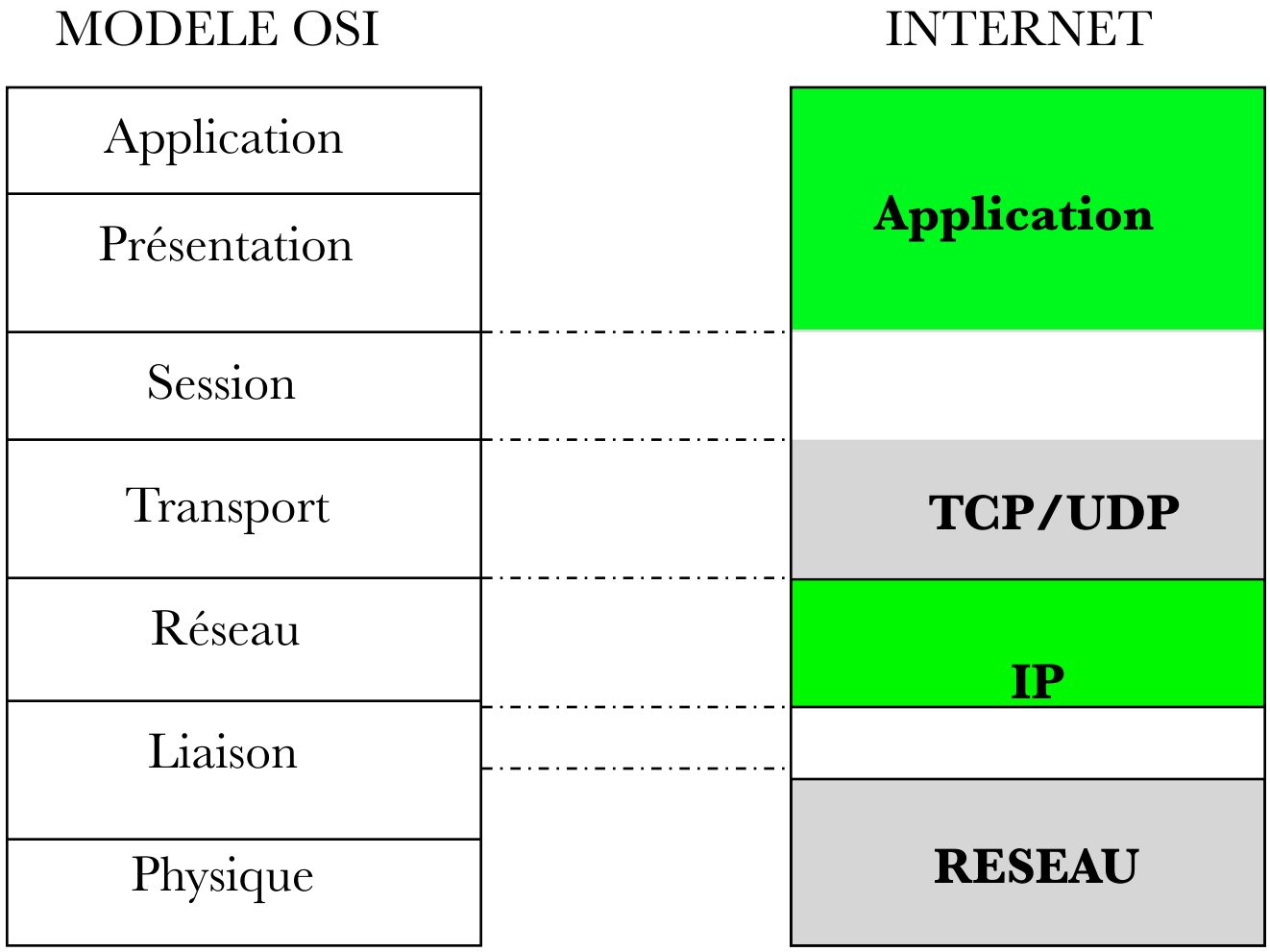
: INTERNET PROTOCOLS

: NETWORK PROTOCOLS

Composants des couches de l'Internet



Comparaison Internet / Modèle OSI



Architecture des Réseaux Informatiques : Le modèle Internet

Réseaux locaux & ETHERNET

Modèle INTERNET

Adressage IP

Format d'un datagramme IP

Routage

IP Layer : ARP / RARP /ICMP

Ils sont développés suivant les 4 couches suivantes : physique**,** liaison**,** réseau**,** application**.**

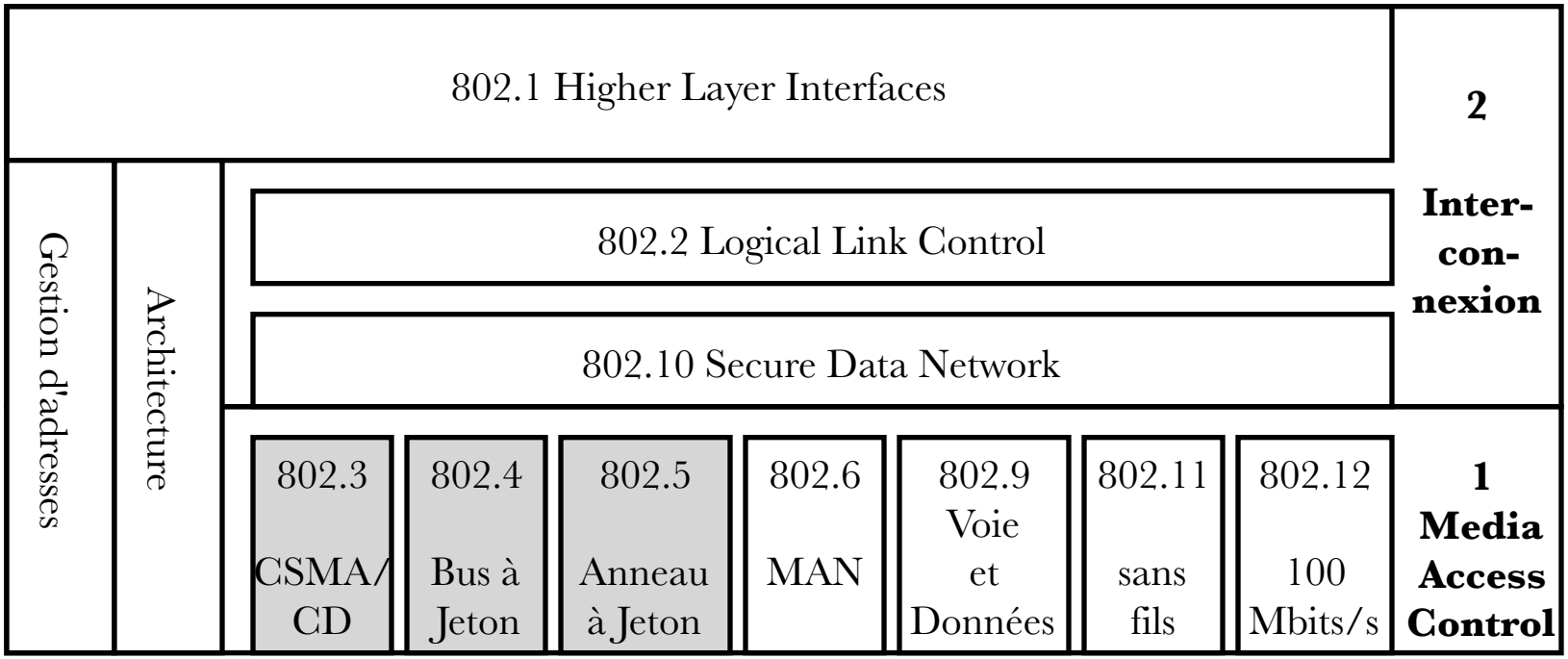
Quelques normes :

Caractéristiques / Normes	**Couche Physique**	**Mode d'accès**	**Débit Mbits/sec**
IEEE 802.3 ETHERNET	Câble coaxial épais ou fin de 2500 m max. (avec 4 répéteurs	CSMA/CD	10
IEEE 802.4 Bus à jeton	Câble coaxial à 75 ohms d'impédance	transmission de jeton suivant un anneau logique	1,5,10
IEEE 802.5 Anneau à jeton	Paire symétrique blindée	transmission de jeton suivant un anneau physique	1, 4

Les standards IEEE 802

Le type de trafic a une influence directe sur le débit

<u>Contraintes</u> : au moins 200 hôtes, au moins 2 km, débit entre 1 et 100 Mb/s, ajout et retrait de hôtes, taux d'erreur faible, adressage individuelle ou en groupe, conformité OSI, diffusion, équité d'accès, gestion possible des priorités, détection et récupération d'erreurs



Pré-câblage & matériel actif d'un réseau

Baie de brassage (pré-câblage) : flexibilité d'interconnexion.

HUB : émulation d'un bus avec connexion en étoile

- ➢ câblage téléphonique : 1 paire pour l'émission et une autre pour la réception;
- ➢ concentration et retransmission vers toutes les paires;
- ➢ en cas d'émission simultanée, un signal de collision est généré vers tous les équipements;
- ➢ gestion possible à distance;

Répéteur :

- ➢ regénérer le signal atténué (couche 1 de l'OSI)
- ➢ relier deux réseaux ayant des supports physiques différents.

Pont filtrant :

- ➢ filtrage selon l'adressage MAC (couche 2 de l'OSI).

Commutateur :

- ➢ création de circuits logiques entre machines
- ➢ mise en place d'une matrice de commutation permettant de traiter plusieurs trames simultanément : augmentation du débit global

Adressage MAC 802.1

La norme 802.1 propose deux types d'adressages :

- ➢ adresse courte sur 16 bits pour les réseaux non interconnectés
- ➢ adresse longue sur 48 bits pour les réseaux connectés

Une adresse universelle est divisée en deux parties :

- ➢ 3 octets de gauche désignent le constructeur (attribué par IEEE, RFC 1700)

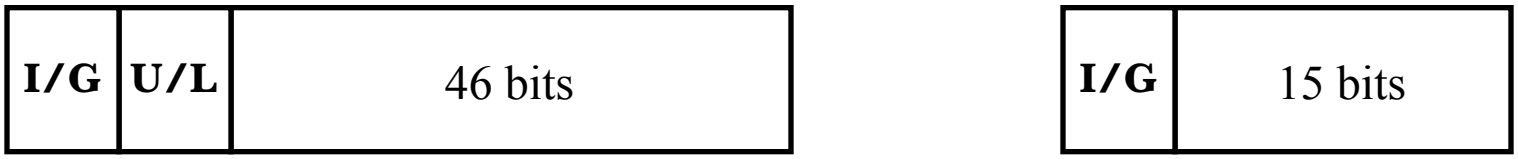
  08:00:20 (Sun), 08:00:2B (DEC), 08:00:5A (IBM), 00:00:0C (Cisco)
- ➢ 3 octets de droite désignent le numéro de série constructeur

Par construction, une adresse universelle est unique.

Adresses de groupe :

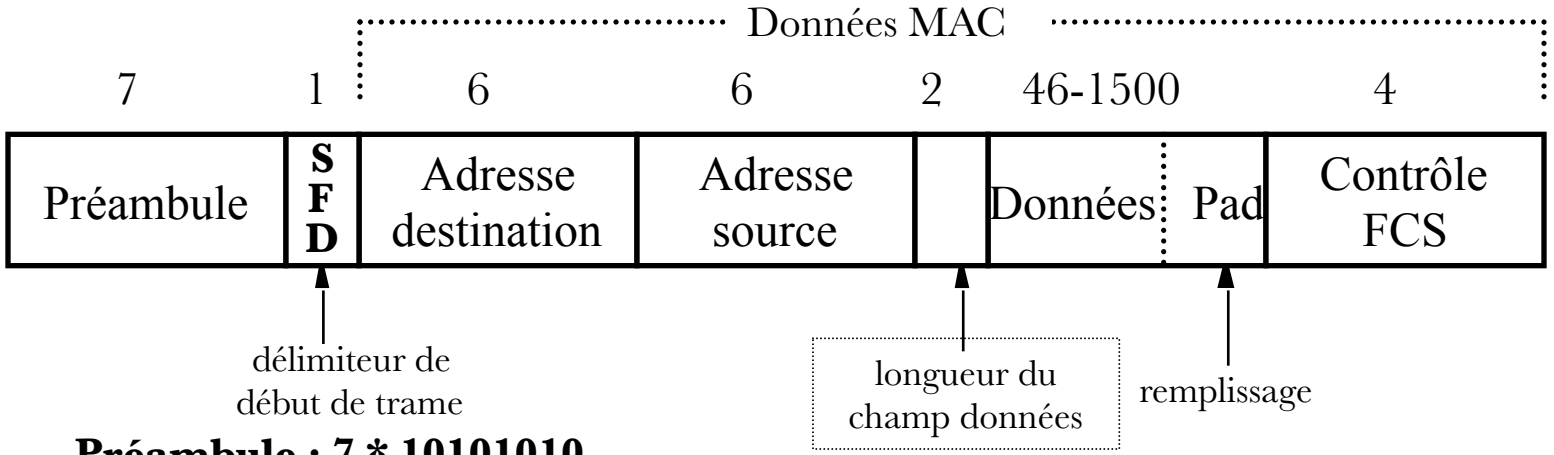
- ➢ Diffusion générale (Broadcast) : FF:FF:FF:FF:FF:FF
- ➢ Diffusion restreinte (Multicast) : abonnement explicite au service en affectant au composant MAC l'adresse de groupe

Emission des bits de droite à gauche (on commence par le poids faible (ex: 0x7A donnera 0x5E).



I Individuel U Universel
G Groupe L Local

Format d'une trame 802.3 :



Préambule : 7 * 10101010
SFD : 10101011

72 <= Taille Trame <= 1546

Format d'une trame ETHERNET :

➢ La seule différence se situe au niveau du champ longueur de données. En effet, ce champ désigne le protocole de service de la couche supérieure (0x0800 pou IP, 0X809B pour AppleTalk, …)

La tranche canal (TC) est au minimum = à deux fois la durée maximale de propagation d'un message (44,9µs).

TC = 51,2 µs ~ émission de 512 bits (64 octets) à 10 Mbits/s

Silence Inter Trame = 9,6 µs

Protocole CSMA/CD

Protocole d'accès à la MAC :

CSMA/CD (**Carrier Sense with Multiple Access and Collision Detection**)

Principes **:**

Toute station connectée au réseau, est en permanence à l'écoute de la porteuse. Une machine peut émettre à condition que le réseau soit libre (aucune communication durant un laps de temps d'environ 9,6 μs).

En cas d'émission simultanée, il y a collision. L'écoute du signal par la station émettrice lui permet de détecter les collisions occasionnées par sa trame.

Chacune des stations va re-émettre après un temps déterminé de manière aléatoire.

Internet protocols (IP layer)

Protocole IP (Internet Protocol) **:**

Adressage,

Routage de data-grammes.

Protocole ARP (Address Resolution Protocol)**:**

Résolution de la correspondance adresse IP → adresse physique.

Protocole RARP (Reverse Address Resolution Protocol) **:**

Résolution de la correspondance adresse physique → adresse IP.

Protocole ICMP (Internet Control Message Protocol) **:**

Gestion des messages d'erreur,

Contrôle entre systèmes INTERNET (échange de table de routage).

IP protocol

IP

- ➢ V4 (V6 ou NG déjà disponible)
- ➢ ISO 8473
- ➢ RFC 791

Fonction **:**

Fournir un service réseau **(niveau 3 du modèle OSI),** sans connexion préalable (**mode non connecté**) et sans garantie de fiabilité **(i.e. sans contrôle d'erreur, ni contrôle de flux).**

- ➢ Emission et réception de paquets de données (taille max. de 64 Ko) appelés **DATAGRAM**.

Adressage IP : Class Full (1)

Adresse IP = 32 bits répartis en 4 octets

Notation décimale pointée : <adresse réseau>.<adresse interface>

- ➢ chaque octet est exprimé en décimal,
- ➢ les différents octets sont séparés par un point,
- ➢ pas de distinction entre numéro de réseau et adresse locale

Un masque de réseau plein : une adresse réseau où tous les bits réseau sont à 1 et les autres (bits interfaces) à 0 :

<adresse réseau>.<adresse interface>

Exemples :

- ➢ **classe B** : 134.206.0.0 (université de Lille), 255.255.0.0 (masque de réseau)
  134.206.1.34 (alpha.univ-lille1.fr) → machine 1.34 du réseau 134.206.0.0
- ➢ **classe C** : 193.51.52 (EC Lille), 255.255.255.0 (masque de réseau)
  193.51.52.100 (lails1.ec-lille.fr) → machine 100 du réseau 193.51.52.0

Adressage IP : Class Full (2)

Poids fort du premier octet	Classe	Plage d 'adresses	Nombre de réseaux	Nombre d'interfaces par réseau (*)
0xxx	A	1 → 126 (**)	126	16 777 216
10xx	B	128.0 → 191.255	16 384	65 534
110x	C	192.0.0 → 223.255.255	2 097 152	254
1110	D	224.0.0 → 239.255.255	Multicast	
1111	E	240.0.0 → 255.255.255	NIC : Gestion du réseau	
Nb interfaces Total= 126*16 777 216 + 16 384*65 534 + 2 097 152*254= 532 692 992				

Adresses particulières (**): 0.0.0.0 → bootp, route par défaut, …
127.0 → Réseau de bouclage
127.0.0.1 → localhost

(*) Chaque plage réseau est privée de sa première adresse pour désigner le réseau et la dernière adresse pour désigner le broadcast

Adressage IP : Class Full (3)

L'adressage Class Full utilise un masque de réseau plein pour déduire l'adresse réseau d'une interface :

- ➢ Une adresse réseau est exprimé sur un nombre entier d'octets :
  - ❒ **UN** octet pour la classe **A** avec le masque **255.0.0.0**
  - ❒ **DEUX** octets pour la classe **B** avec le masque **255.255.0.0**
  - ❒ **TROIS** octets pour la classe **C** avec le masque **255.255.255.0**
- ➢ On réalise un **ET-LOGIQUE** entre l'adresse IP de l'interface et du masque de réseau pour obtenir le réseau d'appartenance de l'interface.

Chaque réseau dispose d'une adresse de broadcast obtenue par la mise à 1 de tous les bits désignant l'interface :

- ➢ classe **A : <adresse réseau>.255.255.255**
- ➢ classe **B : <adresse réseau>.255.255**
- ➢ classe **C : <adresse réseau>.255**
- ➢ Exemple :
  - ❒ 193.51.52.255 est l'adresse de broadcast pour le réseau 193.51.52.0

Adressage IP : Class Full (4)

Une adresse correspond à un accès réseau plutôt qu'à la caractérisation d'une machine. En effet, le déplacement géographique d'une machine (vers un autre réseau local), entraîne la modification de son adresse IP.

Une machine ayant plusieurs interfaces aura plusieurs adresses.

Pas de structuration de l'espace d'adressage :

- ➢ utilisation de grosses tables de routage pour l'interconnexion des réseaux.

Gaspillage de l'espace d'adressage :

- ➢ Au début de l'internet (années 80), il y a eu sur-dimensionnement : les adresses de classe A et de classe B étaient attribuées assez facilement.
- ➢ D'après une étude effectuée en 1985, plus de la moitié des réseaux de classe B ne dépassaient pas 50 ordinateurs.

Adressage IP : Subnetting (1)

Par défaut, l'adressage IP Class Full se fait sur deux niveaux

<adresse réseau>.<adresse interface>

<u>Problème</u> : Apparition et multiplication des réseaux locaux au sein d'une même organisation.

<u>Solution</u> (1985) : 3ème niveau d'adressage :

Notion de sous-réseaux (locale à une organisation)

<u>Notion de subnet-mask</u> :

- **masque définissant les <u>bits utiles</u> (ce n'est plus des octets) pour l'adressage de l'interface par leur mise à 0.**
- Adresse IP = <adresse réseau> ***<adresse sous-réseau>*** <adresse interface>
- Le masque de sous-réseau permet de déterminer l'adresse réseau et sous-réseau d'une interface : autrement dit, le sous-réseau d'appartenance de l'interface.

Adressage IP : Subnetting (2)

Exemple pour un réseau de classe B :

- ➢ Sans subneting : le masque = 255.255.0.0
- ➢ Utilisation de **6 bits** pour définir les sous réseaux : **255.255.252.0**
- ➢ Utilisation de **7 bits** pour définir les sous réseaux : **255.255.254.0**
- ➢ Utilisation de **8 bits** pour définir les sous réseaux : **255.255.255.0**
- ➢ Utilisation de **9 bits** pour définir les sous réseaux : **255.255.255.128**
- ➢ Utilisation de **10** bits pour définir les sous réseaux : **255.255.255.192**

Inconvénients :

- ➢ Le masque doit être le même pour toute les machines d'un réseau local. C'est l'administrateur réseau qui détermine le nombre de sous réseau qu'il veut créer, et donc le masque à utiliser.
- ➢ Nécessité d'utilisation de **routeurs** pour communiquer entre les différents sous-réseaux.
- ➢ la taille des sous réseaux est **fixe**.

Adressage IP

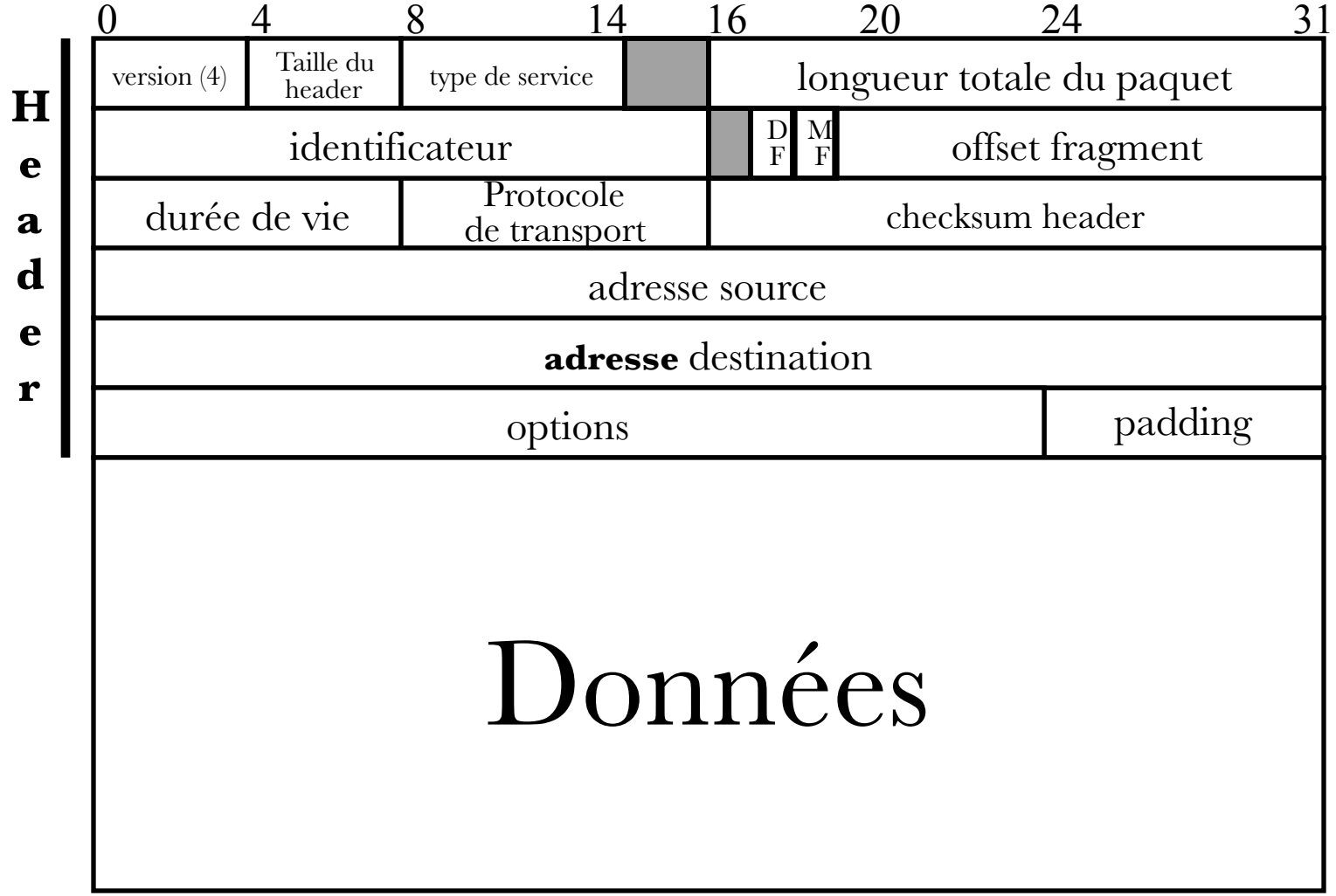
<u>Tendance actuelle</u> :

En raison du faible nombre d'adresses de classe A et B, le NIC encourage les grandes organisations, à utiliser plusieurs réseaux de classe C au détriment de la notion de subnetting.

<u>Avenir</u> : IPNG (IP New Generation) **: 128 bits**

nouveaux champs : capacité + service + sécurité

Format d'un data-gramme IP



Champs d'un data-gramme IP (1)

Champ	Signification du champ
Version 4 bits (=4/6)	Plusieurs versions peuvent cohabiter sur le réseau. Vérification de la version par les hôtes et les routeurs.
Lg_ent 4 bits (5..15)	Longueur entête : longueur en nombre de mots de 32 bits Sans option = 5 soit 20 octets Avec option = 15 soit 60 octets (40 octets en option)
Type de service 6 bits	Qualité de service et mode d'acheminement. Format : 3 bits de Priorité (0 : normal, 7 : Supervision), Delay, Throughput, Reliability (2 bits inutilisés)
Longueur Totale 16 bits	Taille max = 2^{16}-1 = 65635 octets = 64 Ko Les réseaux actuels sont conformes. Ce n'est pas une contrainte sévère.
Identifiant 16 bits	Tous les fragments d'un même datagramme portent le même identifiant.
Flags 2 bits	DF (don't fragment), MF (more fragment). DF = impossibilité de reconstruire (taille min = 576 octets) MF = tous les fragments sont à MF sauf le dernier
Offset Fragment 13 bits	Position du fragment dans le datagramme courant. Tous les fragments ont une taille multiple de 8 sauf le dernier

Champs d'un data-gramme IP (2)

Champ	Signification du champ
Durée De vie 8 bits	**Permet de compter le temps de séjour sur le réseau. On décompte de 1 à chaque saut (+sieurs si queue importante). Max = 255 , Min = 0 alors destruction et message ICMP**
Protocole 4 bits	**Numéro du protocole appelant : TCP, UDP, RAW, autres. RFC 1700 fournit la liste avec les numéros associés.**
Checksum header 16 bits	**Total de contrôle de l'entête vérifie la validité de l'entête. On calcule la somme des octets en complément à 1 et on prend le complément à 1 du résultat. *Re-calcul après chaque saut*.**
Longueur Totale 16 bits	**Taille max = 2^{16}-1 = 65635 octets = 64 Ko Les réseaux actuels sont conformes. Ce n'est pas une contrainte sévère.**
Adresse source 16 bits	**Adresse IP de la machine qui a émis le message.**
Adresse destination 16 bits	**Adresse IP de la machine à qui le message est adressé.**
Options 40 octets max	**Format libre : extension de IP, services non prévus, ...**

Options courantes de IP

Sécurité

- ➢ Degré de confidentialité : utilisé par les routeurs militaires
- ➢ Ce champ est généralement ignoré par les routeurs

Routage strict défini par la source sous forme d'une séquence d'adresses

- ➢ Le routage est strict et le chemin ne peut être changé
- ➢ Utilisé pour des tâches d'administration (alarmes , reconfiguration de routes, mesure de délai)

Routage lâche défini par la source

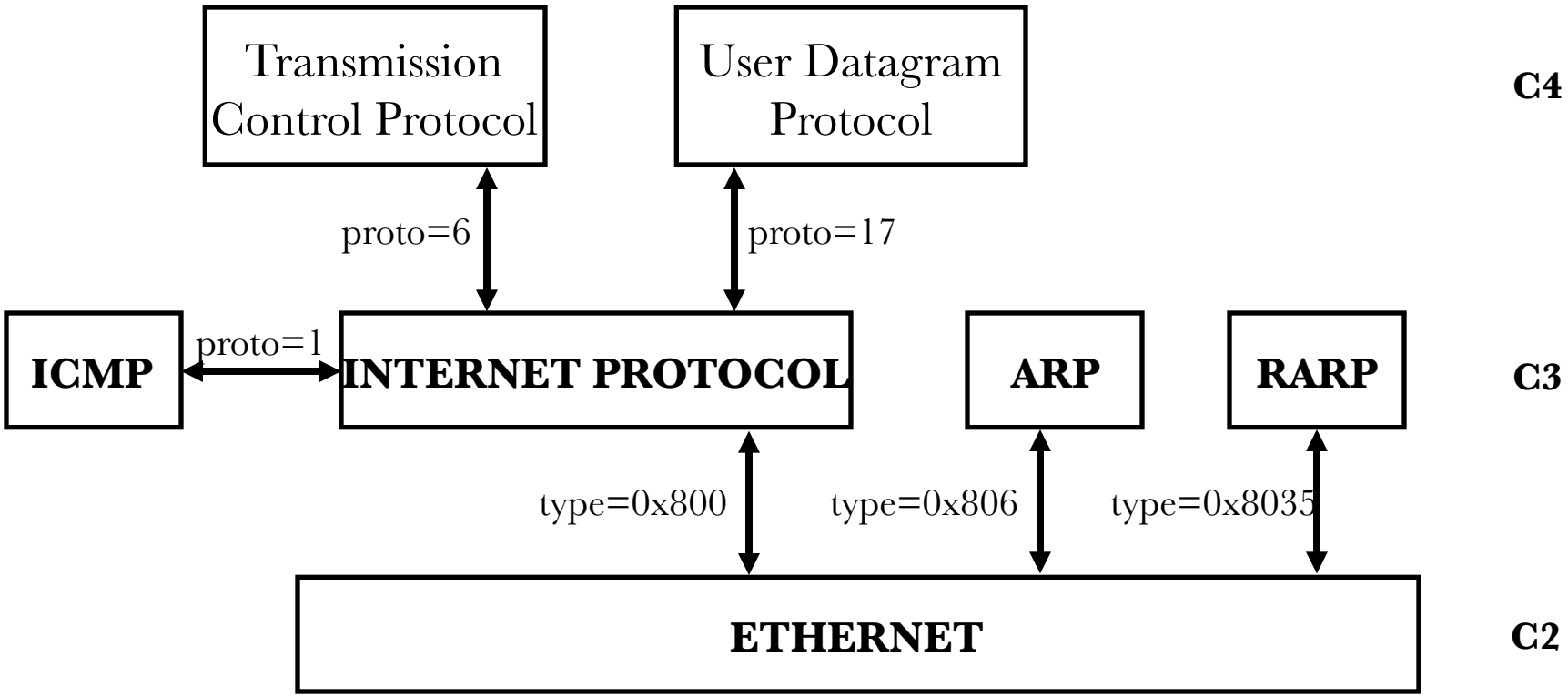
- ➢ Le routage peut être modifié en cas d'indisponibilité d'une route
- ➢ Choix de routes spécifiques pour des raisons de coût, de fiabilité ou de confidentialité

Enregistrement de routes

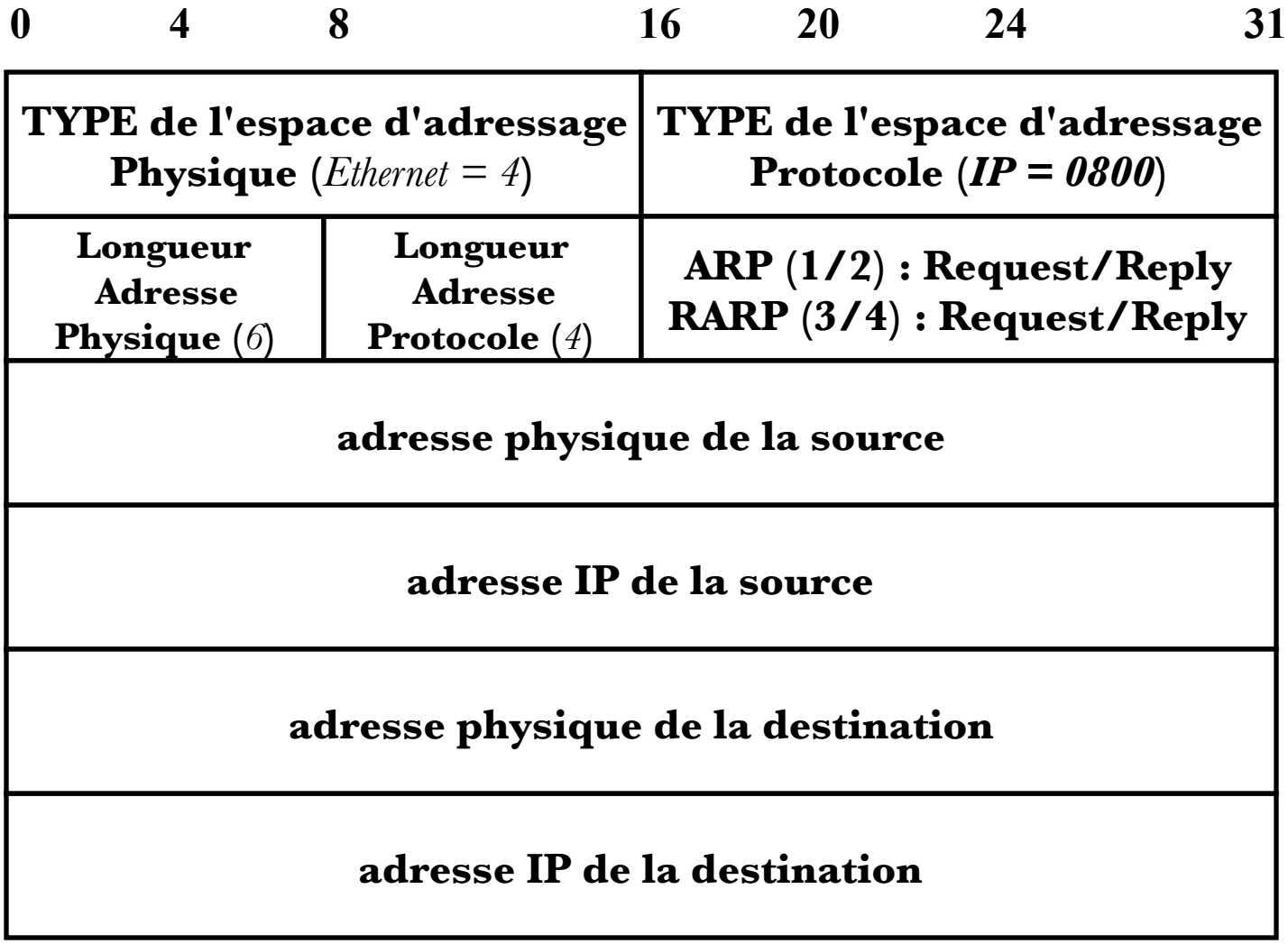
- ➢ Utilisé pour mémoriser les adresses des routeurs traversés
- ➢ Rechercher les raisons d'un mauvais routage

Horodatage

Architecture de la couche IP



Format d'un data-gramme ARP / RARP



Format d'un message ICMP

Type	Code	Checksum

ICMP (RFC 792) est utilisé entre routeurs et hôtes pour rapporter les anomalies, fournir des indications sur le routage.

Un message ICMP est encapsulé dans un data-gramme IP avec le champ protocole égal à 1.

En cas d'erreur, l'entête IP et les 8 premiers octets sont retransmis.

Types de message :

- ➢ Destination inconnue ou inaccessible
- ➢ Durée de vie expirée
- ➢ Entête invalide
- ➢ Limitation de production pour cause d'engorgement
- ➢ Re-direction : changement de route
- ➢ Demande/Renvoi d'écho
- ➢ Demande/Renvoi d'écho avec horodatage
- ➢ Demande/Renvoi de masque de réseau

Routage : principes

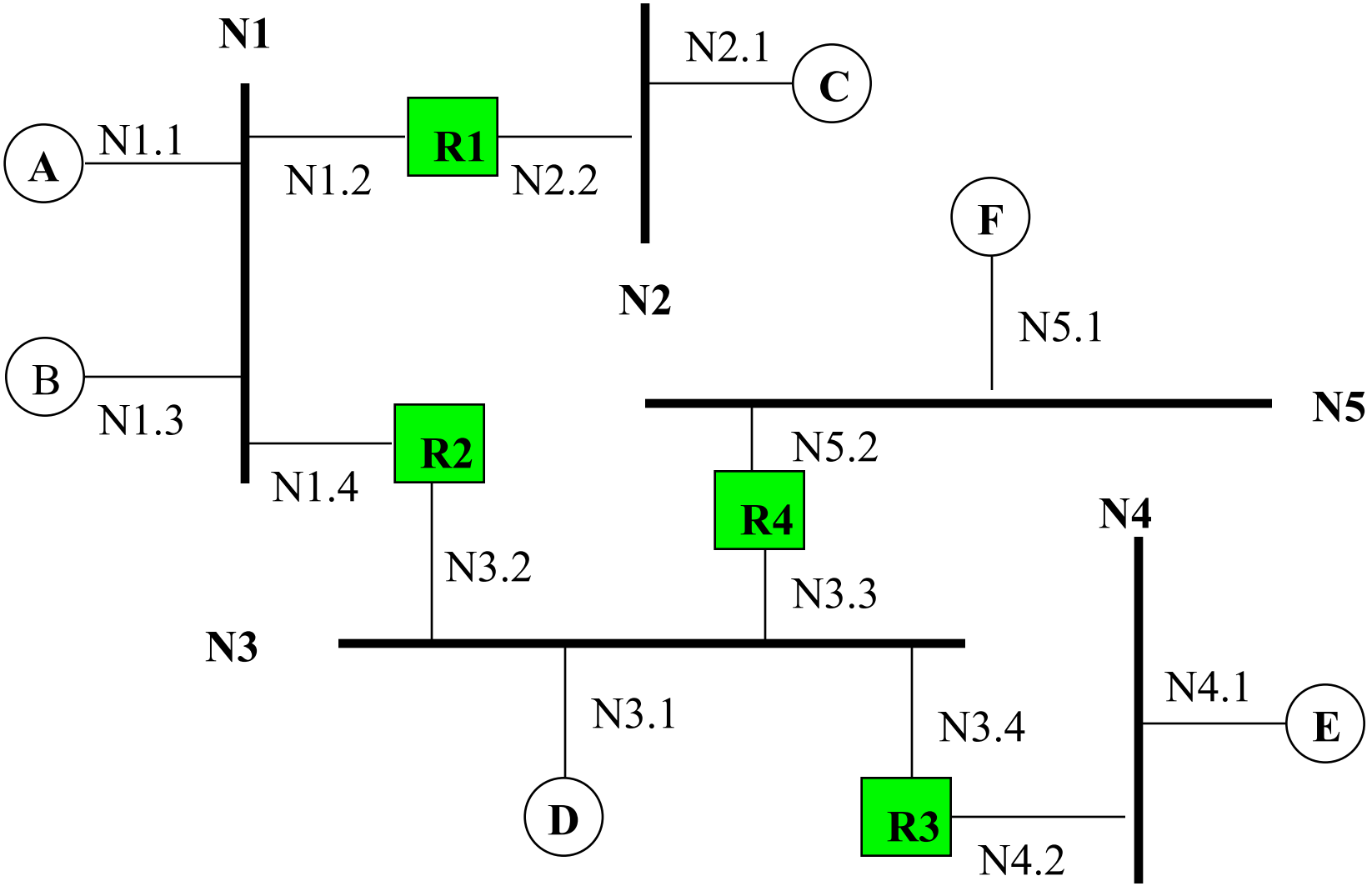
Acheminement des data-grammes (ou des fragments) de l'émetteur vers le récepteur par l'intermédiaire de routeurs reliant les différents réseaux intermédiaires.

Les algorithmes de routage sont basés sur l'utilisation de tables de routages gérées par chaque machine du réseau.

Mise à jour des tables de routage :

- Manuelle: utilisation des commandes *ifconfig* et *route* sur systèmes UNIX ou à vos souris dans un environnement sous Windows.
- Dynamique : tables de routages diffusées par les routeurs.

Routage : exemple



Routage : table de routage

Table de routage pour la station A

Réseau ou Host	Chemin local	Distance Maximale (en HOP)
N2	N1.2	1
default	N1.4	2

Table de routage pour la station D

Réseau ou Host	Chemin local	Distance Maximale (en HOP)
N1	N3.2	1
N2	N3.2	2
N4	N3.4	1
N5	N3.3	1

Routage : routages dynamiques

<u>Principe</u> :

les routeurs s'échangent des informations de routage et diffusent des informations aux stations.

Concept de base : Autonomous System (AS)

- ➢ Ensemble de réseaux ayant une administration commune (partage de ressources, politique de routage)
- ➢ Un AS a un numéro délivré par le NIC (où l'INRIA pour la France) → RENATER = AS 1720

Protocole de routage :

- ➢ **Interne** à un AS : RIP (RFC 1058), HELLO, OSPF, IGRP
- ➢ **Externe** (entre AS) : EGP, BGP

Light IP

Serial IP (SLIP)

- ➢ Encapsulation des data-grammes IP sur ligne série
- ➢ Chaque extrémité doit connaître l'adresse de l'autre
- ➢ Peu efficace : lignes à faible débit et overhead de IP important

PPP

Limites de l'adressage IPv4

1996 : 100 000 Réseaux Locaux

L'adressage IPv4 devient insuffisant :

- Capacité : 2 milliards d'adresses
- Organisation de l'espace d'adressage :
  - Un calasse A (16 millions d'adresses) est trop (trop) grande.
  - Une classe C (256 adresses) est trop petite.
  - Une classe B (65 536 adresses) semble être la mieux dimensionnée. Toutefois, plus de la moitié des classes B ont moins de 50 hôtes
- Imaginons que les classes C disposent de 10 bits pour coder les hôtes. Dans ce cas, les réseaux de classe C auraient satisfaits la plus part des entreprises : un demi million de réseaux classe C contre 16 384 classe B
- Pb de routage : chaque routeur Internet devrait disposer d'une table de routage d'un demi million d'entrées (en 2002 c'est réalisable avec un coût supportable). Mais qu'en est il pour les performances des algorithmes de routage (sélection d'une route, transmission de table à un autre routeur ? Les routeurs étaient prévus avec 10000 entrées !!

CIDR

CIDR : Classless InterDomain Routing (RFC 1519)

Partage de l'espace classe C non alloué par zone géographique :

192.0.0.0 à 193.255.255.255	**Multi-régional**
194.0.0.0 à 195.255.255.255	**Europe**
196.0.0.0 à 197.255.255.255	**Autres**
198.0.0.0 à 199.255.255.255	**Amérique du Nord** :
200.0.0.0 à 201.255.255.255	**Amérique Centrale & Amérique du Sud** :
202.0.0.0 à 203.255.255.255	**Pacifique** :
204.0.0.0 à 207.255.255.255	**Autres**

Cette technique permet l'utilisation de routeurs continentaux : n'importe quel routeur hors Europe recevant un data-gramme à destination d'une adresse 194 ou 195, il doit l'expédier au routeur européen.

Une fois le paquet est reçu, il est nécessaire d'avoir des tables de routages plus détaillées (une table exhaustive nécessiterait 131 072 octets) :

Extension de la table d'entrée en ajoutant un champ pour le masque sur 32 bits.

CIDR : exemple

Mise en place de 3 réseaux :

- ➢ 2048 adresses : 194.24.0.0 à 194.24.7.255 / **255.255.248.0**
- ➢ 4096 adresses : 194.24.16.0 à 194.24.31.255 / **255.255.240.0**
- ➢ 1024 adresses : 194.24.8.0 à 194.24.11.255 / **255.255.252.0**

Table de routage :

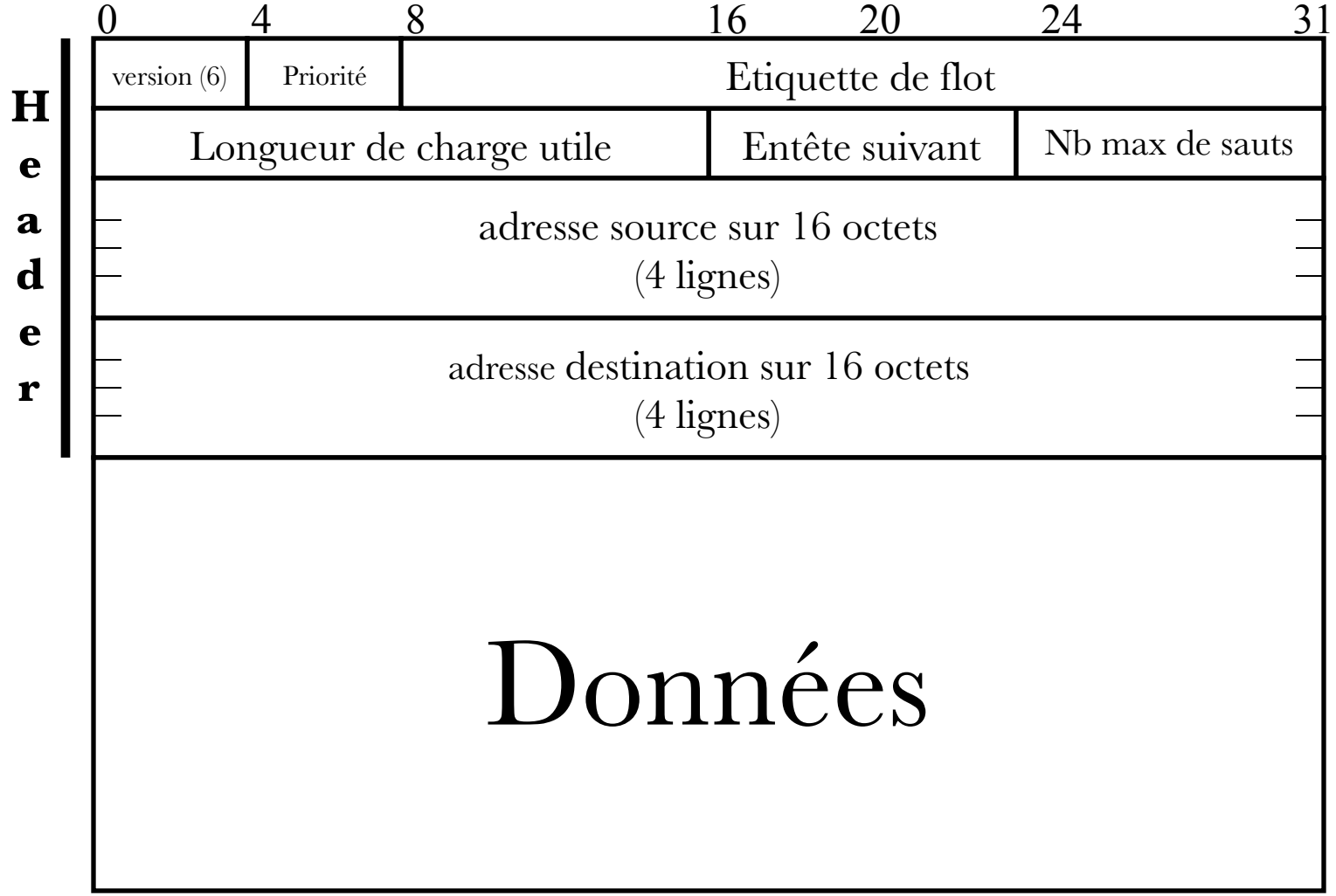
➢ **Réseau1**	**194.24.0.0**	**255.255.248.0**	**11 bits**
➢ **Réseau2**	**194.24.16.0**	**255.255.240.0**	**12 bits**
➢ **Réseau3**	**194.24.8.0**	**255.255.248.0**	**10 bits**

On réalise un ET logique entre l'adresse de destinataire et le masque de l'entrée courante que l'on compare avec l'adresse réseau. Si égalité alors le routeur de destination est sélectionné sinon, on passe à l'entrée suivante dans la table (indexation par famille de routeurs de même masque, ...).

Un résultat double est possible. Dans ce cas, on choisit le réseau ayant le masque le plus plein.

Cette technique rend le routage basé sur les classes obselète.

Format d'un datagram IPv6



Architecture des Réseaux Informatiques : Protocoles de transport

User Datagram Protocol

Transmission Control Protocol

Protocoles de transport

UDP (User Datagram Protocol) : transport de données en mode non connecté

TCP : (Transmission Control Protocol) : transport de données en mode connecté

UDP est utilisé principalement pour la communication locale alors que TCP est utilisé pour la communication locale et distante.

UDP : principes

UDP = User Datagram Protocol (RFC 768)

Fournit le même type de service que IP avec en plus la fonction de multiplexage**.**

- ➢ Multiplexage : Permet de distinguer plusieurs destinations sur une même machine (cf. notion de ports)

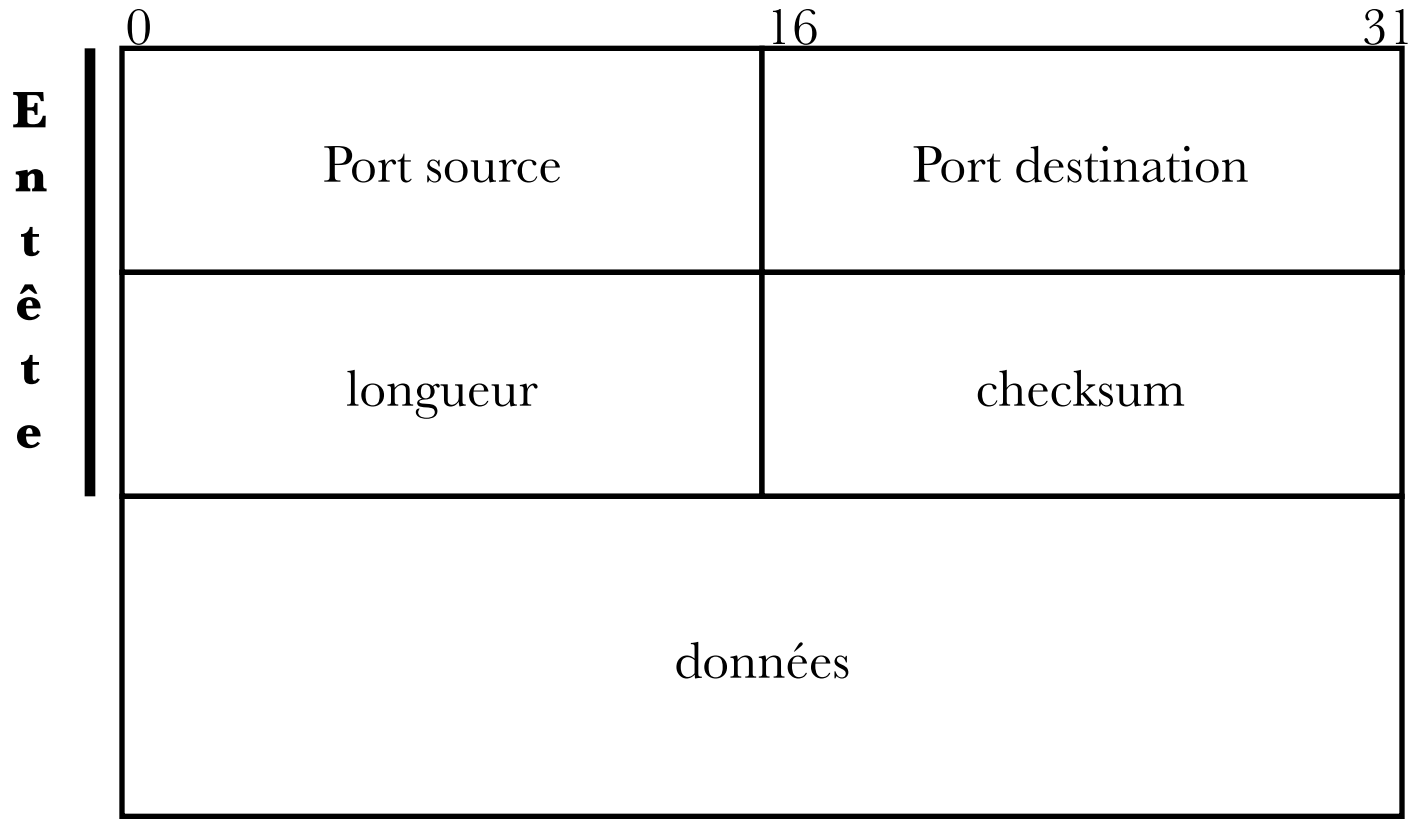
Caractéristiques :

- ➢ Mise en oeuvre simple (cf. format datagram) → rapidité
- ➢ Pas de sécurité (mode non connecté) → A utiliser lorsque le réseau physique sur lequel s'appuie INTERNET est très fiable (fibre optique ...)

Format :

- ➢ Encapsulation dans le champ données d'un datagram IP
- ➢ Valeur du champ protocole dans le paquet IP = 17

UDP : format d'un datagram



longueur = entête + données (octets)
checksum = CRC sur pseudo entête

TCP : principes

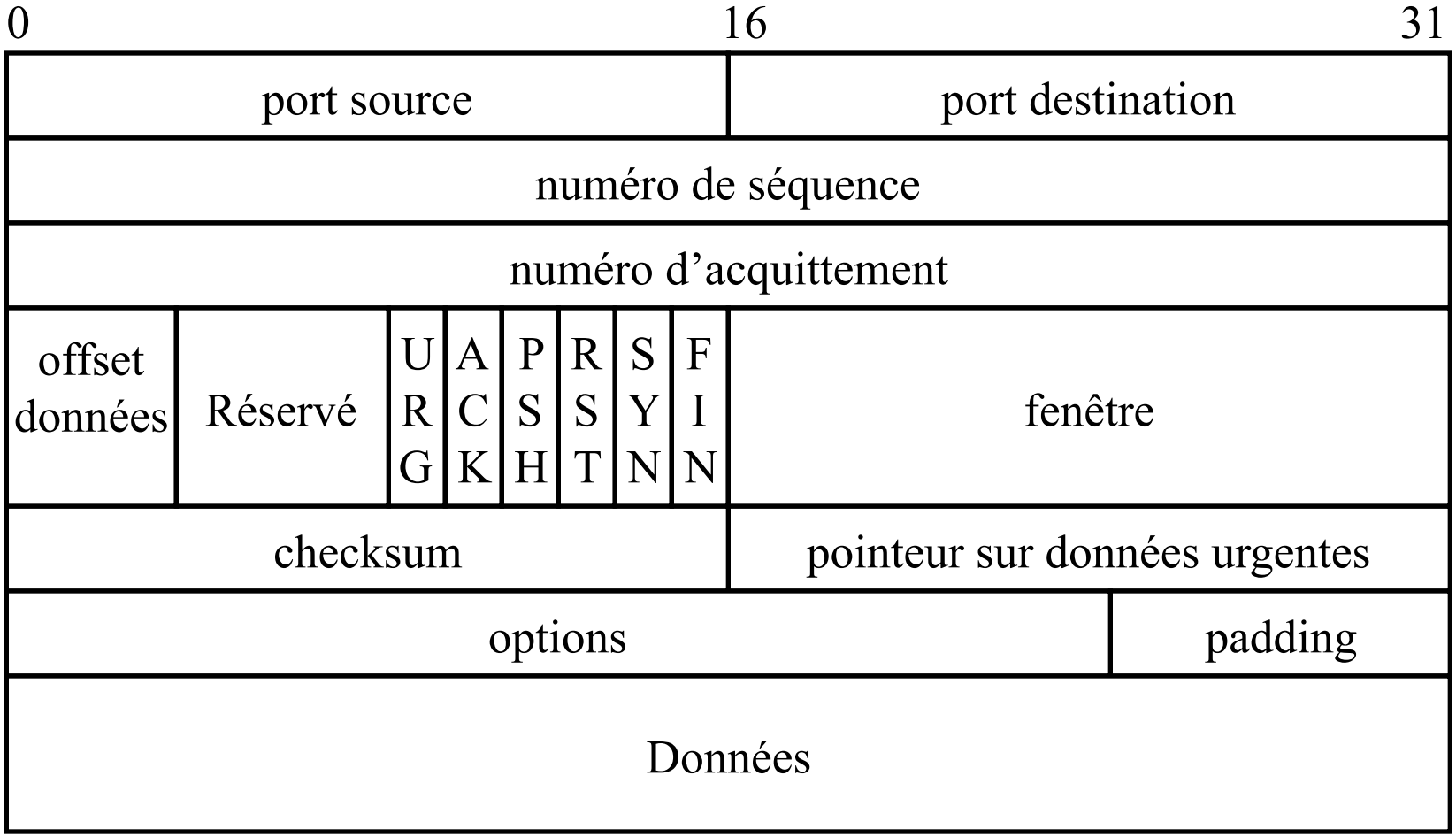
TCP = Transmission Control Protocol (RFC 793)

Principe **:**

Protocole de transport avec connexion qui assure une communication fiable **et** ordonnée **des messages. Les messages appelés** segments **sont d'une taille quelconque.**

- **fiabilité** : Ne suppose aucune fiabilité des protocoles des couches inférieures. Met en oeuvre un système d'acquittement positif, numérotation des segments et retransmission de ceux-ci en cas de non acquittement au bout d'un délai fini.
- **contrôle de flux** : utilisation de fenêtre à anticipation.
- **Multiplexage** → notion de ports

TCP : format d'un segment



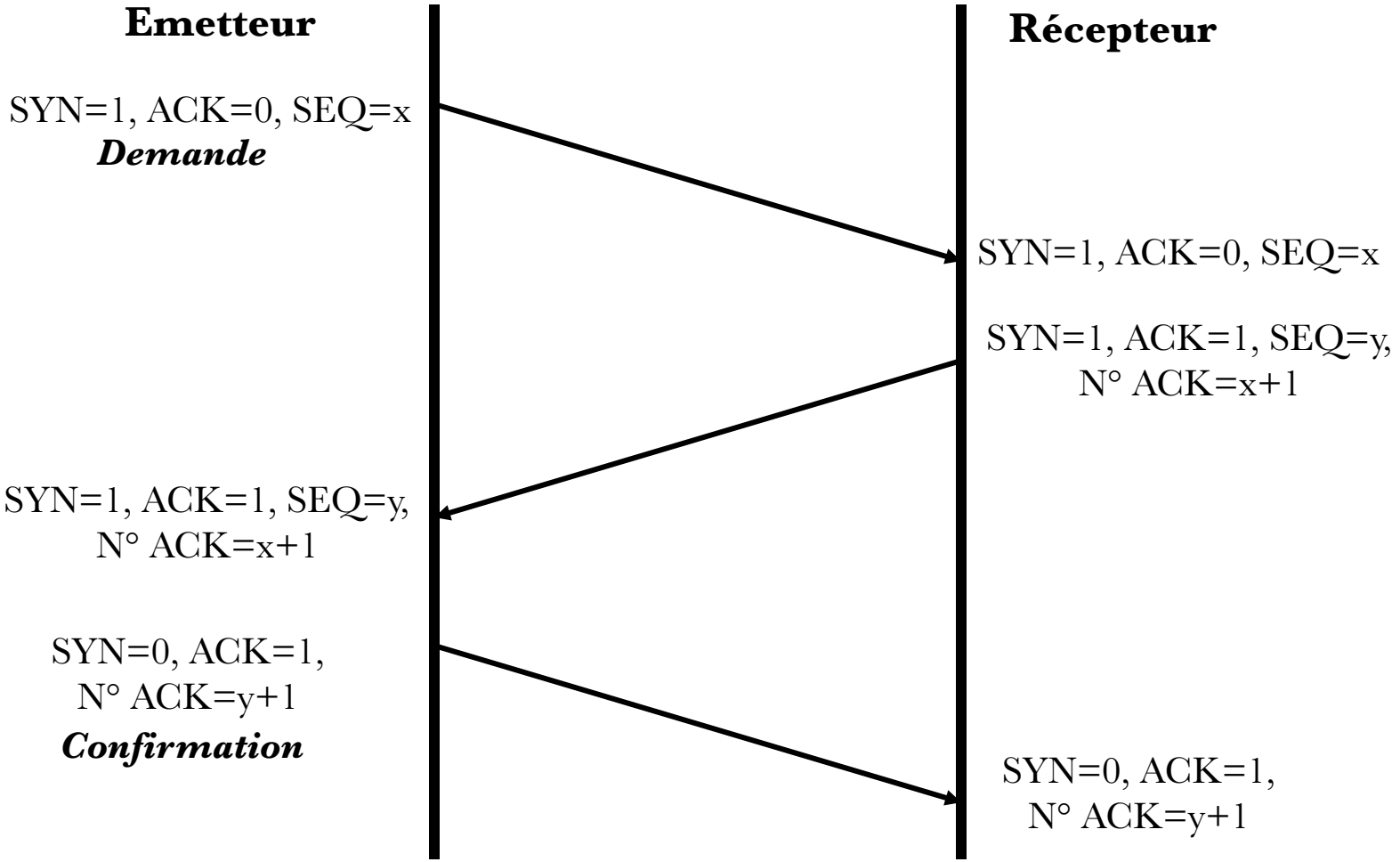
Champs d'un segment TCP (1)

Champ	Signification du champ
Port Source (16 bits)	**Indentifie l'application utilisant le service TCP pour l'échange du segment**
Port Destination (16 bits)	**Indentifie l'application destinatrice du service TCP pour l'échange du segment**
Numéro de Séquence (32 bits)	**Numéro d'ordre du premier octet de données de ce segment**
Numéro d'acquitement (32 bits)	**Si le bit ACK est positionné, ce champ indique le prochain numéro de séquence attendu par TCP**
Offset données 4 bits	**Longueur de l'entête TCP en mots de 32 bits**
Flags 6 bits	**SYN : synchro n° séq, ACK : Acquittement valide** **RST : reset connexion, FIN : fermeture connexion** **PSH : empêche la bufferisation,** **URG : pointeur URG positionné**

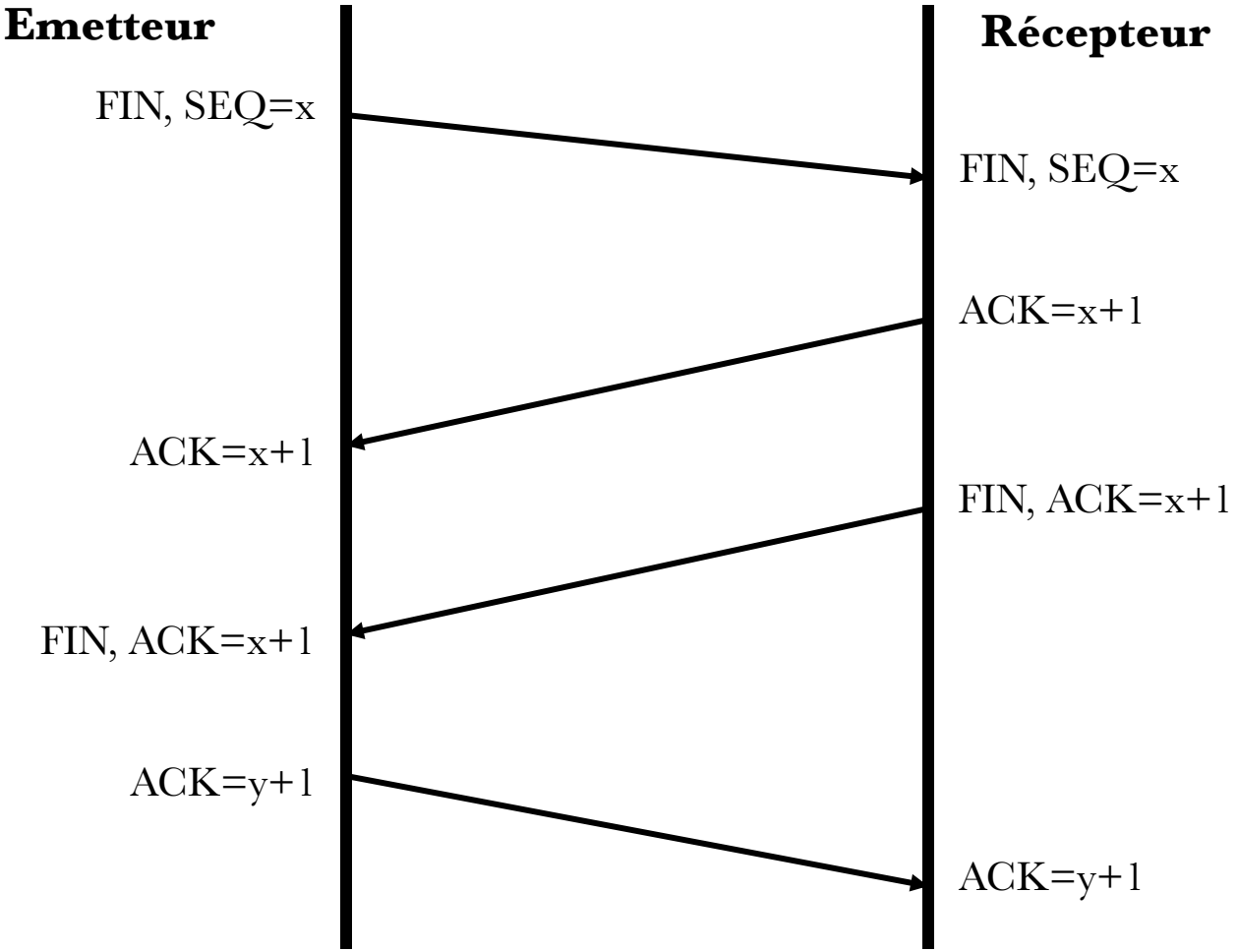
Champs d'un segment TCP (1)

Champ	Signification du champ
Fenêtre 16 bits	Nombre d'octets que TCP est prêt à recevoir à partir du numéro d'acquittement
Checksum 16 bits	CRC calculé sur un pseudo-header et les données
Pointeur de données urgentes 16 bits	Si URG est positionné, on l'additionne au numéro de séquence et on obtient le numéro de séquence des données urgentes
Options et Padding	Options : 0 ou plusieurs octets Padding : 0 à 3 octets de bourrage

TCP : Etablissemnet d'une connexion



TCP : Libération d'une connexion



TCP : transfert de données

Après établissement de la connexion, les données sont envoyées sous forme d'un flot numéroté.

Chaque segment indique le nombre d'octets que TCP (émetteur) est prêt à recevoir sans acquittement à partir du numéro d'acquittement.

Le numéro d'acquittement indique le numéro du prochain octet attendu et acquitte tous les précédents.

A chaque transmission TCP arme un timer de retransmission :

- ➢ Perte et timer élevé engendrent une baisse de débit
- ➢ Congestion et retransmission rapide : on aggrave la congestion
- ➢ On utilise un algorithme adaptatif à chaque connexion
  - ❒ Basé sur RTT : temps moyen d'un aller/retour
  - ❒ À chaque retransmission, on double le timer
  - ❒ Fenêtre d'anticipation divisé par deux en cas de congestion

Ports : services de l'Internet

0 → 255 : réservés applications INTERNET

- ➢ 21 → ftp
- ➢ 23 → telnet
- ➢ 25 → smtp
- ➢ 53 → Name Server

256 → 1024 : services UNIX

> 1024 : développements

/etc/services (exemple d'une ligne sous UNIX)

```
ftp   21/tcp          21/udp          #protocole ftp
```

Architecture des Réseaux Informatiques : Communication inter-processus par sockets

Notion de Socket

domaine, protocole, adressage

Modèles de communication

DATAGRAM & STREAM

Primitives de gestion des sockets

socket, bind, accept, listen, connect, …

Primitives de manipulation de l'adressage IP

résolution de noms, port de service, …

Communication inter-processus ou IPC

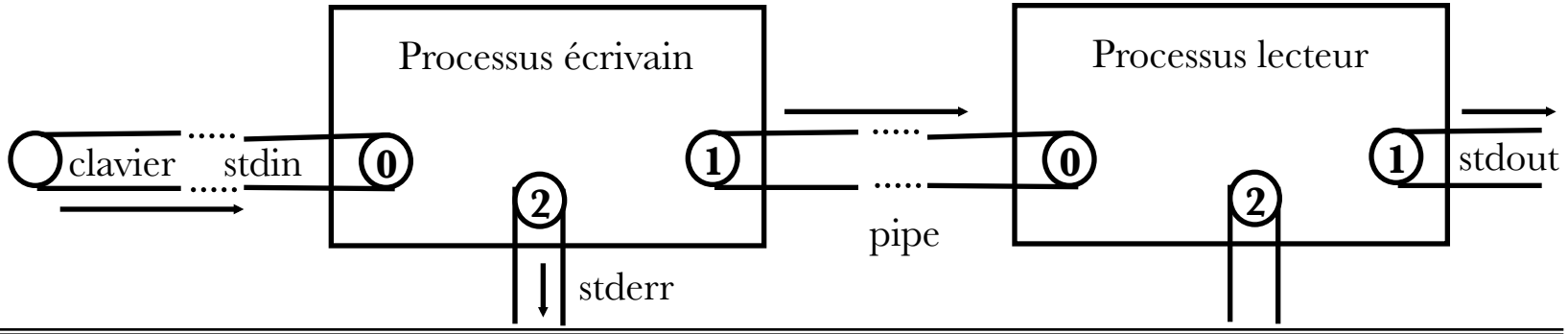
La communication entre deux processus se fait par l'utilisation d'un canal logique (rappel : tout processus possède par défaut trois canaux logiques : *STDIN*, *STDOUT*, *STDERR*).

Deux modes de communication sont possibles :

- ➢ connecté (téléphone) : un canal de bout en bout (attaché)
- ➢ non-connecté (courrier) : un canal non attaché

On distingue trois types d'IPC :

- ➢ **fichier** : entre deux processus distincts en mode non-connecté
- ➢ **pipe, pairsocket** : entre un processus **père** et un processus **fils**
- ➢ **socket** : entre deux processus **distincts.** Ce mécanisme prend tout son sens dans une application réseau (Client/Serveur).



Notion de socket

Les pipes et les pairsockets offrent une communication entre un processus père et un processus fils : nécessité d'avoir un ancêtre commun.

Deux processus créés séparément (éventuellement sur deux machines différentes : souvent le cas) qui veulent communiquer ensemble doivent utiliser le mécanisme de socket.

La notion de socket est introduite par UNIX BSD pour banaliser la communication dans le monde Internet.

- ➢Fournir un modèle **standard** de communication entre deux processus quelconques.
- ➢La communication se fait en **full-duplex**.
- ➢Une socket peut être assimilée à un fichier réseau → nécessité d'un mécanisme d'adressage universel.

Adressage & Domaine des sockets

L'initiative de communication est initiée par un processus. Par conséquent, le destinataire est connu à l'avance.

Une socket utilise un nom qui doit être translaté en adresse correspondante avant l'établissement de la communication.

La nature des sockets utilisées dépend du domaine d'application :

- AF_UNIX,
- AF_INET,
- AF_ROUTE,

Chaque domaine nécessite un système d'adressage pour désigner le destinataire :

- un chemin de fichier de type socket pour AF_UNIX,
- une adresse IP (hôte) et un numéro de port (service applicatif)

Protocole & mode de communication d'une socket

Le protocole définit les formats de données, les règles et les conventions régulant la communication.

Le protocole permet d'établir une connexion vers le destinataire et d'acheminer les données entre sockets.

Deux modes de communication sont possibles :

- **SOCK_DGRAM : DATAGRAM ou mode non connecté (courrier).**
- **SOCK_STREAM : STREAM ou mode connecté (Téléphone).**
- **SOCK_RAW : administrateur (couche 3 : IP, ICMP, …)**

Pour chaque mode de communication (STREAM, DATAGRAM) dans un domaine donné (AF_UNIX, AF_INET, …), il n'existe généralement qu'un seul protocole de communication.

Rappel : La création d'une socket nécessite la spécification du *domaine*, du *mode* et du *protocole*.

Création d'une socket

La création d'une socket se fait par l' appel système :

```
#include <sys/socket.h>
sock = socket(int domain, int mode, int protocol);
/*sock=numéro du canal logique attribué*/
```

La donnée *domain* indique la famille qui sera utilisée : *AF_UNIX*, *AF_INET*, ...

La donnée *mode* indique le mode d'échange qui sera utilisé : *SOCK_DGRAM*, *SOCK_STREAM*, *SOCK_RAW*.

Le *protocole* est implicite en fonction du domaine et mode qui seront choisis :

- la valeur 0 permet d'utiliser les protocoles par défaut
- protocole *UDP* pour AF_UNIX et AF_INET en mode SOCK_DGRAM
- protocole *TCP* pour AF_INET en SOCK_STREAM

En cas d'échec l'appel renvoie -1 et positionne errno à :

- *INVALID_SOCKET*.
- *ENOBUFS* : pas assez de mémoire
- *EPROTOTYPE* :protocole non supporté

Fermeture d'une socket

A la fin de l'utilisation, une socket doit être fermée par l'appel :

```
close(sock);
int sock;
```

En cas d'erreur errno vaut *SOCKET_ERR*.

Dans le cas d'utilisation d'une communication en mode connecté, le système va continuer à écrire des données sur la socket. Il est donc préférable d'utiliser l'appel :

```
shutdown(sock, how);
int sock, how;
```

L'argument how vaut :

- 0 : plus de lecture
- 1 : plus d'écriture
- 2 : plus de lecture/écriture

Le domaine AF_UNIX

Les sockets de type AF_UNIX sont utilisées dans le cadre d'une communication entre deux processus tournant sur la même machine.

La communication se fait via un fichier de type socket : adressage par un chemin d'accès (pathname) qui est soit *relatif* ou *absolu* : les chemins relatifs sont à éviter.

Il y a donc création d'un noeud dans le système de fichiers désignant le socket (vnode(), unlink()).

Les processus utilisateurs s'y réfèrent en utilisant son pathname.

Structure d'une socket en AF_UNIX : #include <*sys/un.h*>

```
struct sockaddr_un {
    short    sun_family;
    char     sun_path[108];}
```

Le domaine AF_INET

Les sockets de type AF_INET sont une implémentation des protocoles DARPA Internet :

- ➢ adressage IP
- ➢ protocole TCP ou UDP.

Les sockets AF_INET sont utilisées pour une communication entre deux processus tournant sur des machines différentes mais également sur la même machine (adresse de bouclage).

La communication se fait via un canal réseau de type socket.

Structure d'une socket en AF_INET : #include <*netinet/in.h*>

```
struct sockaddr_in {
    short           sin_family;
    u_short         sin_port;
    struct in_addr  sin_addr;
    char            sin_zero[8];}
```

sockaddr_in et sockaddr_un sont une spécialisation de sockaddr

Association d'un adressage à une socket

Une socket créée doit être adressée avant toute utilisation par l'appel système :

```
bind(sock, addr, addrlen);
int sock, addrlen;
struct sockaddr *addr;
```

Cet appel en cas d'échec renvoie -1 et errno vaut *SOCKET_ERROR*

En AF_UNIX, *addr* indique le chemin d'accès au fichier socket.

En AF_INET, la donnée *addr* n'est pas obligatoire (utilisation de l'interface réseau par défaut). Pour une machine ayant plusieurs interfaces, *addr* vaut :

- soit l'adresse de l'interface utilisée,
- soit *INADDR_ANY* pour toutes les interfaces.

Si dans l'adressage, on utilise 0 comme valeur de port, alors le système attribuera un numéro de port libre (cf. *getsockname()*).

DATAGRAM : caractéristiques

Ce mode ne nécessite pas de connexion.

L'échange se fait par l'envoi d'un message entier, appelé datagram.

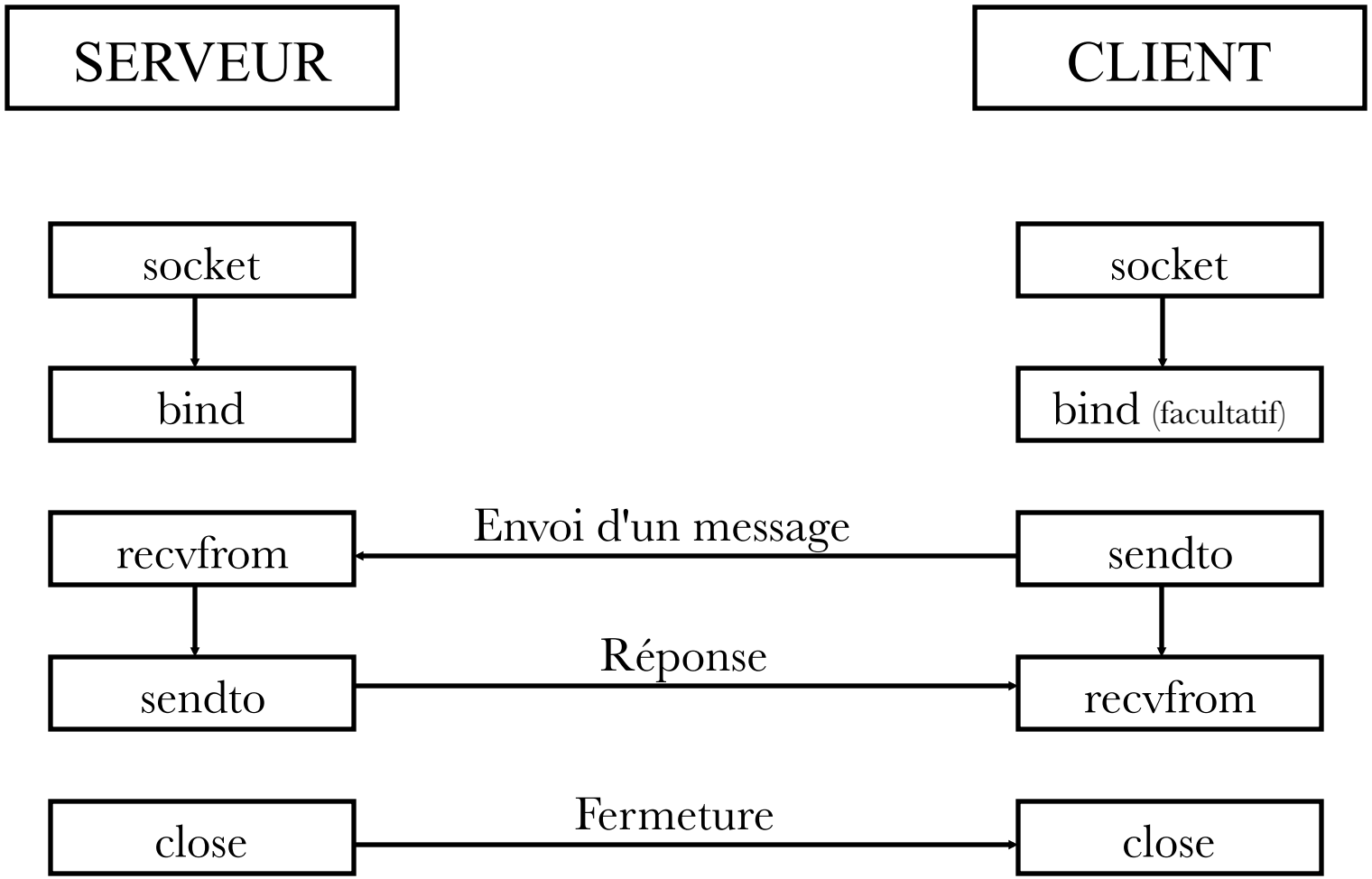
Aucune gestion des erreurs de transmission n'est effectuée.

Le seul traitement d'erreurs effectué est basé sur un checksum de l'entête UDP et d'un TimeOut. L'émetteur doit recevoir un ACK du destinataire durant une fenêtre temporelle définie par le chien de garde.

Ce mode est rapide.

Ce mode de communication nécessite le protocole UDP.

DATAGRAM : modèle de communication



STREAM : caractéristiques

Ce mode nécessite l'établissement d'une connexion.

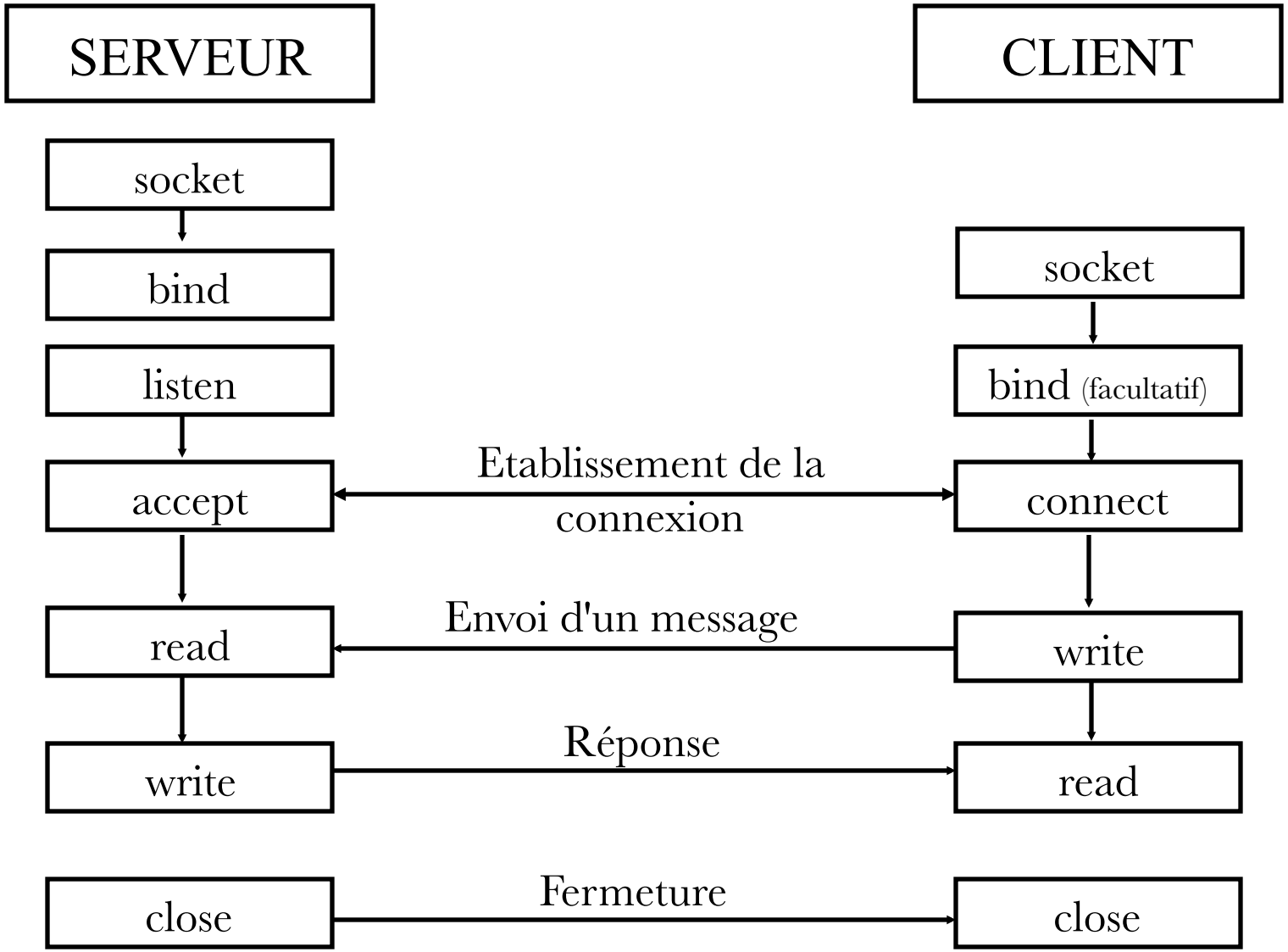
Ce mode offre un transfert de données fiable :

- ➢ **garantie d'acheminement des données,**
- ➢ **respect de l'ordre des paquets : reconstitution du message entier avant délivrance,**
- ➢ **envoi des données urgentes.**

Une gestion automatique des erreurs de transmission (error-free) est assurée par le mode STREAM.

Ce mode nécessite un protocole se caractérisant par une transmission de messages segmentés et d'une gestion des erreurs : TCP.

STREAM : modèle de communication



STREAM : établissement d'une connexion

En mode connecté, le serveur doit établir une connexion avant tout échange.

Le serveur doit être en attente de réception de requêtes de connexion de la part des clients.

Demande de connexion par le client :

```
etat = connect(sock, addr, addrlen);
int etat, sock, addrlen;
struct sockaddr *addr;
```

Un serveur multiple doit indiquer le nombre de clients mis en attente de connexion par l'appel :

```
listen(sock, 5);
```

Acceptation d'une requête de connexion par le serveur :

```
newsock = accept(sock, from, fromlen);
int newsock, sock, *fromlen;
struct sockaddr *from;
/*sock est la socket d'écoute et newsock celle de
  la connexion*/
```

Réception de message en mode connecté

La réception des données se fait par l'appel :

```
nbcar = read(sock, buff, bufflen); /*nbcar = nb de carac. reçus */
int sock, bufflen;
char  *buff;
```

L'appel *read* ne peut être utilisé qu'en mode STREAM (connecté) et renvoie le nombre de caractères reçus, ou 0 si le processus distant a fermé la connexion.

L'appel *read* est construit à partir de l'appel :

```
nbcar = recv(sock, buff, bufflen, flag);
int sock, bufflen, flag;          /* flag = 0 dans le cas du read */
char  *buff;
```

L'argument *flag* permet de configurer la connexion :

- *MSG_OOB* : out of band → lecture de messages urgents.
- *MSG_PEEK* : lecture sans retrait du tampon.

Erreurs de lecture :

- *ENOTCONN* : socket non connectée

Envoi de message en mode connecté

L'envoi des données se fait par l'appel :

```
write(socket, buff, bufflen);
int socket, bufflen;
char  *buff;
```

L'appel *write* ne peut être utilisé qu'en mode STREAM (connecté) et renvoie -1 en cas d'erreur.

L'appel *write* est construit à partir de l'appel :

```
send(sock, buff, bufflen, flag);
int sock, bufflen, flag;          /* flag = 0 dans le cas du write */
char  *buff;
```

L'argument *flag* permet de configurer la connexion :

- *MSG_OOB* : out of band → écriture de messages urgents.
- *MSG_DONTROUTE* : message à destination d'un réseau local.

Erreurs d'écriture :

- *ENOTCONN* : socket non connectée
- *EPIPE* : socket cassée

DATAGRAM : Réception/envoi de message

La réception de données sur une socket non connectée se fait par l'appel :

```
nbcar = recvfrom(sock, buff, bufflen, flag, from, fromlen));
int sock, bufflen, flag, fromlen;  char  *buff;
struct sockaddr *from;
```

L'argument *from* sert à mémoriser l'adresse de l'émetteur. Cette information est utile pour renvoyer une réponse à l'émetteur.

L'envoi de données sur une socket non connectée se fait par l'appel :

```
nbcar = sendto(sock, buff, bufflen, flag, to, tolen));
int sock, bufflen, flag, tolen;      char  *buff;
struct sockaddr *to;
```

L'argument *to* sert à mémoriser l'adresse du destinataire. Cette information est utile pour que le datagramme trouve sa destination.

En cas d'échec, ces fonctions renvoient la valeur -1.

Primitives annexes à l'utilisation des sockets

La programmation des applications réseau nécessite le respect de standards.

Les données doivent être converties au format réseau avant leur transmission.

La résolution de noms doit passer par l'utilisation de mécanismes communs : DNS, NIS, base d'adresses locale.

Passage d'une représentation humaine d'une adresse IP (en décimal) à une représentation machine (4 octets).

Les services offerts par une machine sont nommés (cf /etc/services) → utilisation des noms au lieu des numéros de port.

Primitives de conversion de données

Les données avant transmission sur le réseau doivent être converties dans un format standard appelé "network order".

On dispose des primitives suivantes :

- htons(val) : host order to network order short
- htonl(val) : host order to network order long
- ntohs(val) : network order to host order short
- ntohs(val) : network order to host order long

Primitives de résolution de noms de machines

Structure d'un adressage IP :

```
struct hostent {
    char *h_name;
    char        **h_aliases;
    int         h_addrtype;         /* ~ AF_INET */
    int         h_length; / *  4   */
    char        **h_addr_list; }
#define h_addr h_addr_list[0];
```

Nom d'une machine → Adresse IP :

```
struct hostent gethostbyname (char *name);
```

Adresse IP → Nom d'une machine :

```
struct hostent gethostbyaddr (struct in_addr addr);
```

Adresse IP décimale → Adresse IP machine :

```
char *inet_ntoa (struct in_addr addr);
```

Adresse IP machine → Adresse IP décimale :

```
struct in_addr inet_aton (char *addr_mach);
```

Primitives de résolution de noms de services

Un service réside sur un port et possède un protocole.

Structure d'un service IP :

```
struct servent {
    char *s_name;
    char        **s_aliases;
    int         s_port;
    char        *s_proto; }
```

Nom d'un service → Numéro de port :

```
struct servent getservbyname (char *name, *proto);
```

Numéro de port → Nom d'un service :

```
struct servent getservbyport (port; proto);
int port; char *proto;
```

Création d'un processus fils

Un serveur multiple doit détacher un processus fils pour répondre à une requête de connexion cliente.

Le serveur (processus père) exécute alors l'appel système fork().

Cet appel crée un deuxième processus (fils ou clone) à l'identique du père. La seule différence résulte dans le résultat renvoyé par le fork().

Le fork() renvoit le numéro du processus fils dans le cas du père et la valeur 0 dans le cas du fils.

En cas d'erreur, le résultat vaut -1.

Les deux processus continuent leur évolution en parallèle et ce à partir de l'appel du fork().

Attention à la création en boucle : un premier fils créé par une boucle, créera à partir de cet instant autant de fils que son père.

Bibliographie

[1] Andrew Tannebaum, Réseaux, Ed; InterEditions, 1997

[2] Laurent Toutain, Réseaux lovaux & Internet, Ed. Hermes, 1998

[3] Samuel Pierre, Réseaux Locaux : Fondements , implantation et études de cas , Ed. Eyrolles, 1991

[4] C. Machi, J.F. Guilbert, et 13 Co-auteurs, Téléinformatique : transport et traitement de l 'information dans les réseaux et systèmes téléinformatiques et télématiques, Ed. Dunod, 1987.

[5] G. Pujolle, D. Seret, D. Dromard, E. Horlait, Réseaux et Télématique, Tome1, Edition Eyrolles, 1987.

[6] P. Rolin, Réseaux Locaux : normes et protocoles, Ed. Hermes, 1993

Architecture des Réseaux Informatiques : LES FONDEMENTS

Généralités : concepts, classification, commutation

Transmission : circuits, modes, codage, matériel

Anomalies : détection, traitement

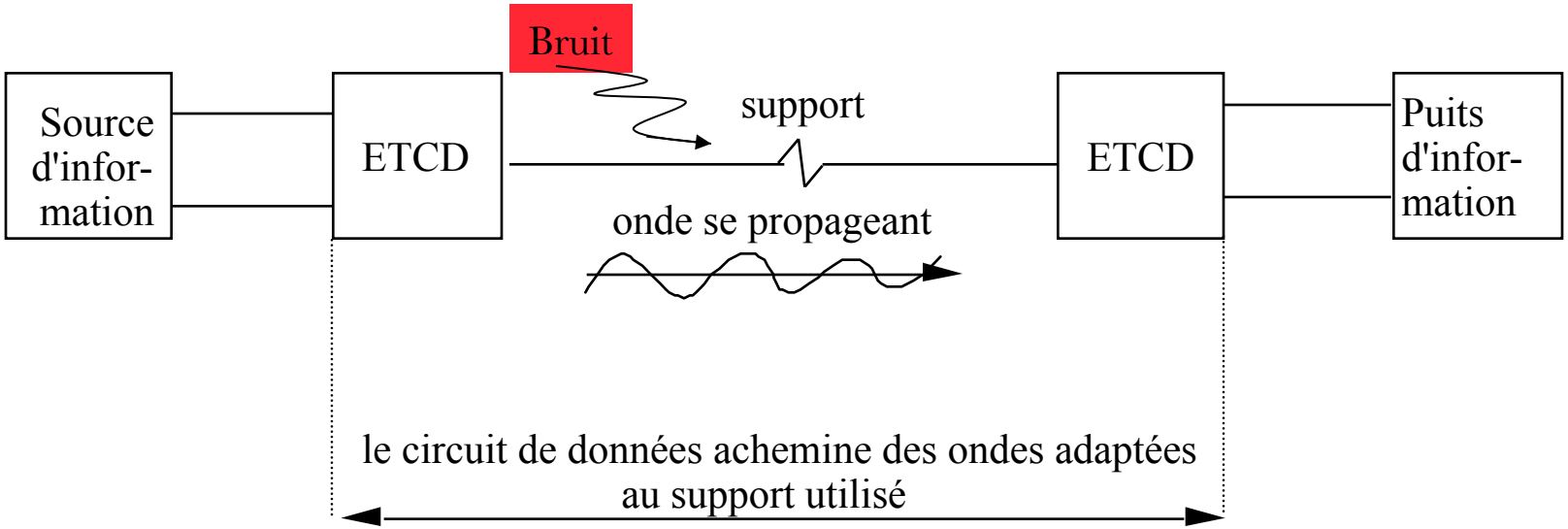
Notion de circuit de données

Notion d'ETCD (Equipement Terminal de Circuit de Données)

Notions de Bande de Base et de Large Bande

Problèmes liés au codage numérique

Circuit de données



Notion de Signal et Nature

On désigne par onde toute vibration donnant lieu à des lignes ou surfaces qui se propagent dans un milieu donné (ondes matérielles, ondes électriques ou électromagnétiques).

Un signal est caractérisé par trois attributs fondamentaux :

- ➢ la fréquence,
- ➢ l'amplitude,
- ➢ la phase.

Signal analogique : C'est un signal dont l'amplitude varie dans le temps comme une fonction continue (audio ; video) . La transmission est modulée.

- ➢ Exemple d'onde sinusoïdale : $a(t) = A \sin(2\pi ft + \phi)$

Signal numérique : C'est un signal dont l'amplitude ne varie qu'en prenant une valeur unique par intervalle de temps (signal en provenance d'un calculateur), on parle de créneau.

Nature du Canal

Les réseaux de télécommunications développés pour la transmission de la voix sont analogiques : on utilise un canal analogique

Les réseaux informatiques ont été développés pour la transmission de données numériques (voix numérisée, vidéo numérisée, …) utilisent des canaux numériques.

On peut distinguer 4 cas de figure :

- ➢ Réseau et Information analogiques : diffusion du son et de l'image (TV, Téléphone).
- ➢ Réseau et Information numériques : LAN ou RNIS.
- ➢ Réseau numérique et Information analogique : diffusion du son et de la vidéo numérisés sur un LAN (multimédia).
- ➢ Réseau analogique et Information numérique : transmission de données informatiques sur les WAN (X25).

Spectre d'un signal périodique

Support analogique

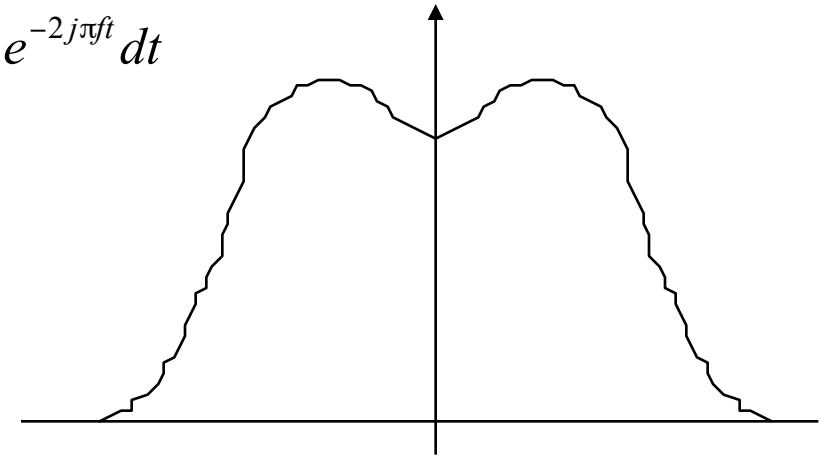
spectre = énergie transportée par le signal.

Tout signal d'énergie finie x(t) peut s'écrire sous la forme

$$x(t) = \int_{-\infty}^{+\infty} X(f) e^{2j\pi ft} df$$

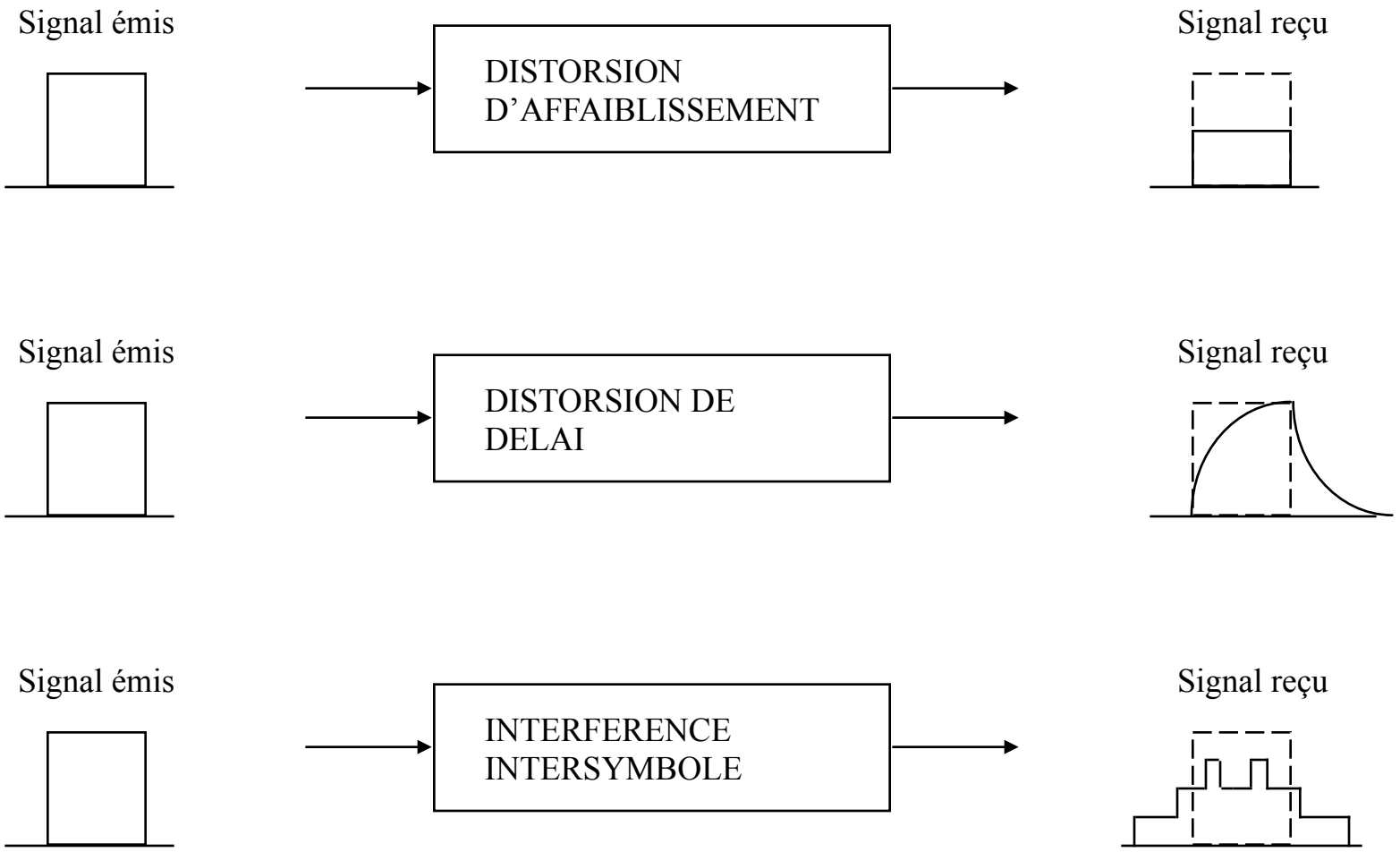
avec

$$X(f) = \int_{-\infty}^{+\infty} x(t) e^{-2j\pi ft} dt$$



La largeur de bande traduit la richesse de l'information transmise

Anomalies de transmission



Bande passante d'un support

<u>Définition</u> :

C'est la bande de fréquences dans laquelle des signaux transmis par un *support donné*, sont "convenablement" reçus. Elle est définie par rapport à un niveau *d'atténuation du signal*, donné en décibels.

$$W = f2 - f1$$

La largeur de bande passante est une caractéristique essentielle :

- ➢ Restriction de l'espace de fréquence : filtres passe-bande
- ➢ Exemple du téléphone :
  - ❒ Bande passante de 300 Hz à 3400 Hz : W = 3100 Hz
  - ❒ Si le canal contient des fréquences en dehors de cette bande, le canal agit comme un filtre et les élimine.

Débit : capacité d'un canal

<u>Définition</u> :

On appelle débit binaire ou vitesse d'un canal la quantité d'informations maximale que le canal peut transporter par unité de temps (bit/s)

<u>Formule de Shannon</u> : Le débit d'un canal est limitée par la bande passante et par le rapport signal/bruit

- W -> largeur de bande
- S/B est le rapport signal/bruit
- Valeur courante : S/B=1000
- Atténuation : $10\log_{10}(S/B)$

<u>Capacité maximale d'un canal</u>

$$W \log_2 (1 + \frac{S}{B})$$

Exemple : Soit un canal téléphonique perturbé (S/B=20db), on obtient un débit max de l'ordre de : $C=3100\log_2(1+100) = 20000$bits/s

Les modes de transmission

La transmission en bande de base (numérique)

- ➢ C'est une transmission directe de signaux numériques, sans décalage de fréquence,
- ➢ La fréquence de changement d'état est exprimée en bauds,
- ➢ Faibles distances : dégradation, nécessité de régénération.

La transmission large bande (analogique)

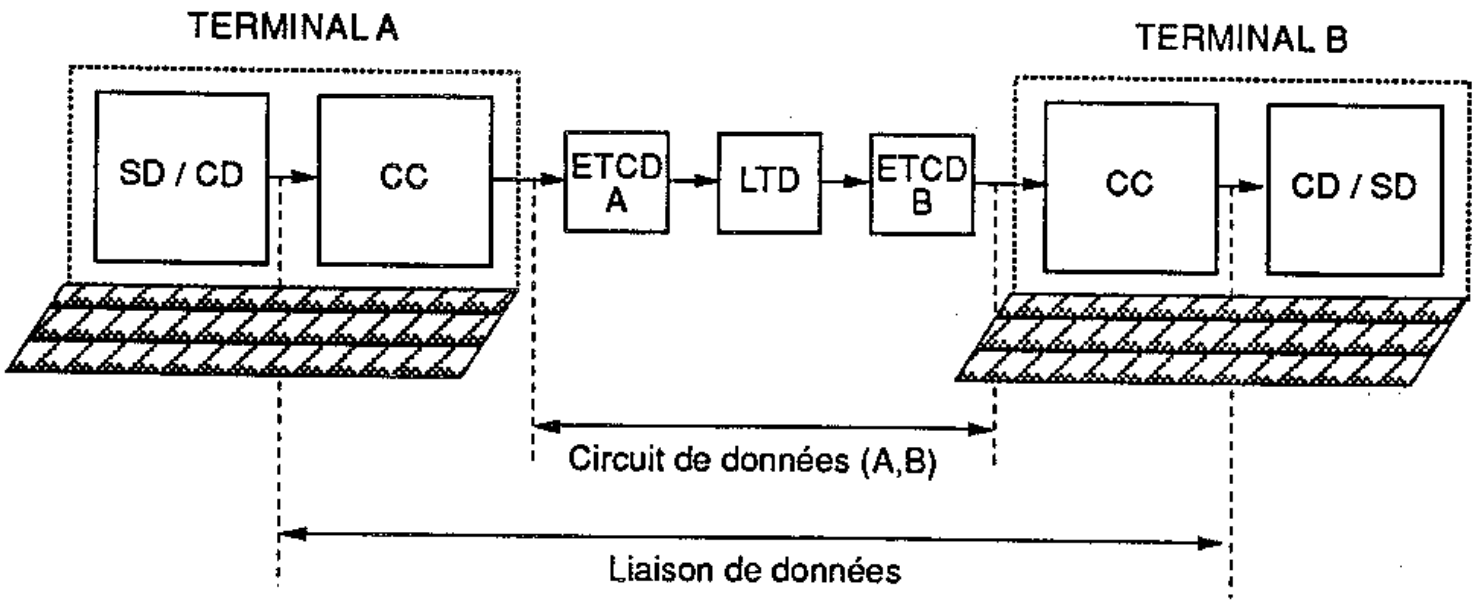
- ➢ C'est une transmission analogique utilisant le principe de la modulation d'une onde porteuse.
- ➢ La modulation permet la restitution du signal d'origine même si le signal reçu est affibli
- ➢ La transformation d'un signal analogique en signal numérique nécessite une opération d'échantillonage.

Système de transmission de données (1)

Fonctions nécessaires à l'établissement d'une communication entre un émetteur et un récepteur

- le contrôle de la communication entre l'émetteur et le récepteur,
- l'adaptation des signaux reçus de l'émetteur aux caractéristiques du support de transmission,
- l'adaptation des signaux transmis par le support de transmission,
- l'ajout de mécanismes de détection et de correction d'erreurs engendrées par la transmission.

Système de transmission de données (2)



SD : Source de données
CC : Contrôleur de communications
LTD : Ligne de transmission de données
CD : Collecteur de données

Système de transmission de données (3)

Contrôleur de communication :

➢ protection contre les erreurs,

➢ introduit des caractères de services.

Equipement Terminal de Circuit de Données :

➢ Adapter le signal électrique,

➢ Etablir et libérer les circuits

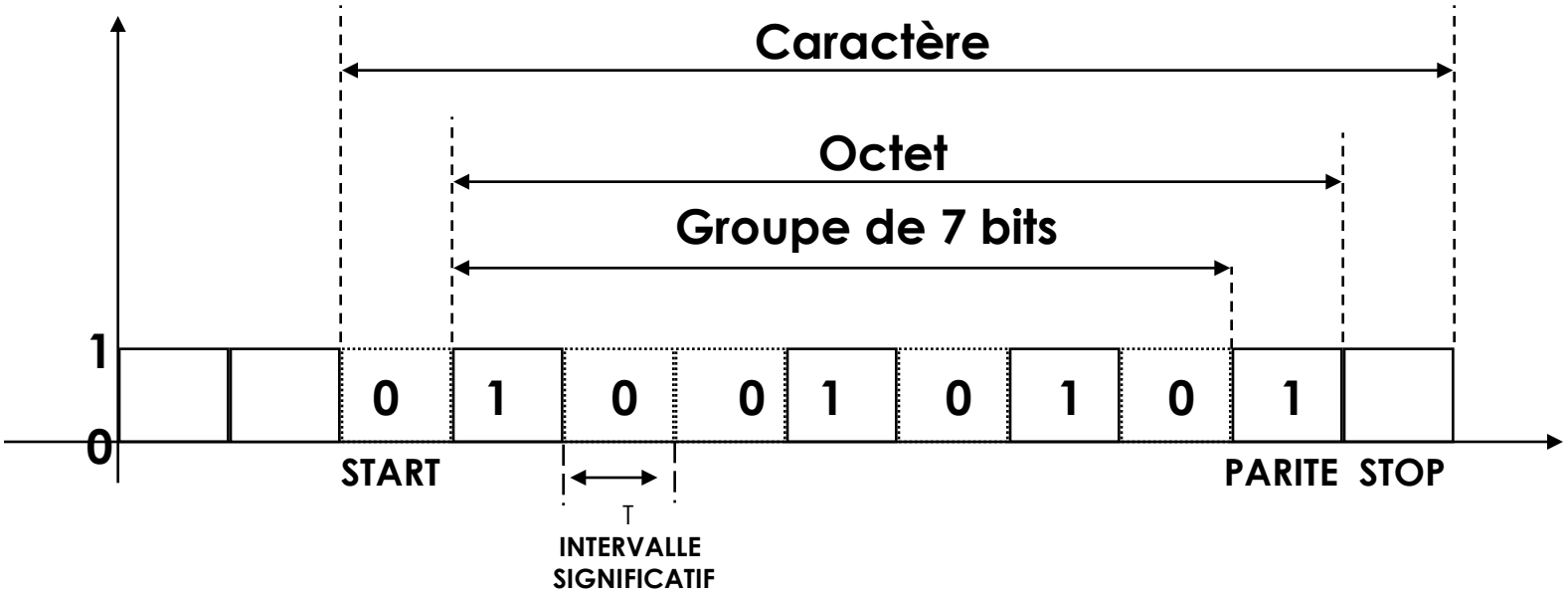
Jonction standardisée entre ETTD et Modem (ETCD) : Avis V24 du CCITT

Synchronisation d'un signal numérique

Transmission synchrone : transmission de l'horloge de synchronisation

- ➢ Grande vitesse
- ➢ Longueur de trame quelconque

Transmission asynchrone : C'est une succession de trains de bits séparés par des intervalles quelconques. Cela nécessite l'ajout d'éléments de repérage à chaque suite binaire.



Transmission large bande

<u>Modulation</u> :

Elle transforme le signal à transmettre en un signal adapté à la transmission sur un support passe-bande.

<u>Avantages</u> :

- ➢ transposition du signal dans un domaine de fréquence correspondant au support,
- ➢ meilleur protection du signal contre le bruit,
- ➢ possibilités de transmission simultanée de plusieurs messages sur un même support.

signal modulé :

$$x(t)=a(t)\cos(\Phi(t))$$

Transmission en bande de base

Transmission numérique => problème de codage du signal numérique

ETCD est un codeur/décodeur

Plusieurs techniques de codage du signal pour maintenir la synchronisation acquise. En effet, dans une longue suite de 0 ou de 1, le récepteur avec l'imprécision de l'horloge pourra lire plus ou moins de bits.

Il est donc nécessaire d'introduire des transitions entre les bits :

- ➢ à 2 niveaux : (NRZ, Manchester, Manchester Différentiel, Miller)
- ➢ à 3 niveaux : (Codage bipolaire, bipolaire entrelacé d'ordre 2, codage BHDn).
- ➢ Codes par blocs : 4B/5B

Le codage de l'information numérique

Code Baudot : Codage à 5 bits qui a été conçu pour le télex; il n'est pas adapté au traitement direct par ordinateur.

Code BCD : C'est un code à 6 bits auquel est ajouté un bit de parité.

Code ASCII (American Standard Code for Information Interchange) ou Code CCITT N°5 : C'est un code à 7 bits plus un bit de parité. Il permet donc de coder 128 caractères.

Code EBCDIC (Extended Binary Coded Decimal Interchange Code) : C'est un code à 8 bits d'information ; il n'y a pas de contrôle de parité.

Bruit de transmission

bruit thermique (bruit permanent ou bruit Gaussien)

bruit d'intermodulation -> propre à la communication large bande -> création de nouveaux signaux dont les fréquences sont des combinaisons linaires des fréquences normalement transmises.

bruit de fond (cf. lignes téléphoniques)

bruit d'impulsion -> troubles électromagnétiques externes

Correction d'erreurs

Correction directe (CED) : C'est une correction effectuée par le décodeur du récepteur.

Correction par retransmission :

- ➢ avec arrêt et attente (ARQ-ACK)
- ➢ continue (ARQ-NACK)
- ➢ à répétition sélective.

Types des erreurs

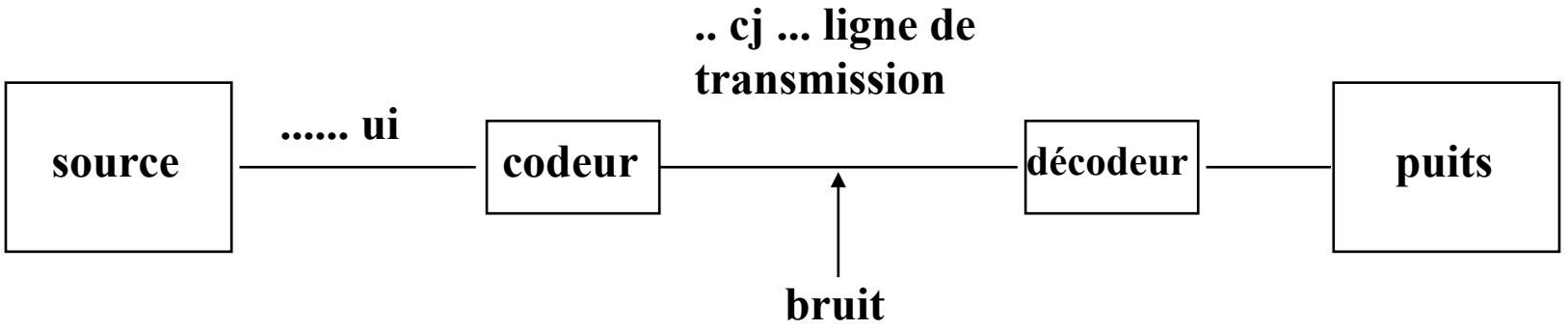
Deux types d'erreurs :

- ➢ erreur isolée : l'occurrence de l'erreur est indépendante de l'occurrence des erreurs précédentes.
- ➢ paquet d'erreurs de longueur i : l'erreur commence au premier bit et et se termine sur le ième bit du paquet.

A l'émission : 0 1 0 1 1 0 1 0 0 1 0 1

A la réception : 0 1 **1** 1 **0** 0 1 0 **1** 1 0 1

Détection d'erreurs (1)



Principe : A un message de m bits, l'émetteur rajoute r bits de contrôle permettant au récepteur de vérifier l'intégrité du message.

L'ensemble des bits du message et de contrôle forment un mot de code de n bits / n=m+r -> mot de code (n,m)

- n -> longueur de code
- m-> dimension du code

Exemple : mot de code (3, 2)

- parité paire : le nombre de bit à 1 dans le mot de code est paire
- parité impaire : le nombre de bit à 1 dans le mot de code est impaire

Détection d'erreurs (2)

SUITE BINAIRE			INFORMATION		BIT DE CONTROLE	NOMBRE DE 1	PARITE
0	**0**	**0**	**0**	**0**	**0**	**0**	**paire**
0	0	1	0	0	1	1	impaire
0	1	0	0	1	0	1	impaire
0	**1**	**1**	**0**	**1**	**1**	**2**	**paire**
1	0	0	1	0	0	1	impaire
1	**0**	**1**	**1**	**0**	**1**	**2**	**paire**
1	**1**	**0**	**1**	**1**	**0**	**2**	**paire**
1	1	1	1	1	1	3	impaire

Définitions générales

Suite de données : $U = u_1 u_2 \ldots u_m$ **Mot de code :** $X = x_1 x_2 \ldots x_n$

Le code est dit *systématique* si pour tout i / 0 < i <m+1 xi =u i

Le code est appelé *code en bloc* si le mot de code X ne dépend que du bloc U et non des blocs précédents. Sinon le code est un code *convolutionnel* ou *récurrent*.

<u>Rendement</u> : $R = \frac{m}{n}$

Poids de Hamming : C'est la somme des bits non nuls d'un mot de code

Distance de Hamming : C'est le nombre de bits de même rang de valeur différente

Les codes linéaires

Définition : C'est un code systématique dans lequel les (n-m) bits de contrôle dépendent linéairement des m bits d'information.

Matrice de contrôle de parité H :

- H=(A, In-m) avec A matrice (n-m)*m
- Propriété : $H.X^t = 0$

Matrice génératrice G : $G = (I_m, -A^t)$

- X=U.G

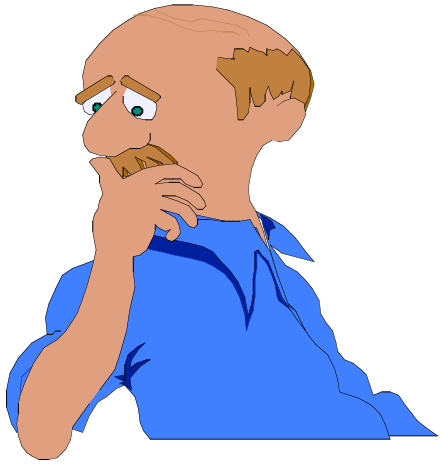
<u>Propriété</u> :$GH^t = 0$

Correction directe d'erreurs (1)

Mot reçu : Y

Vecteur d'erreur : E=Y-X

Le décodeur doit évaluer à partir du mot Y, quel mot de code X a été transmis. Mais il ne connaît pas le vecteur E. Il doit donc le **choisir avec la plus grande probabilité**



Correction directe d'erreurs (2)

Distance minimum d'un code d :

- ➢ d=min dist (U,V) $\forall U, \forall V \in C$
- ➢ d=min poids (U-V) $\forall U, \forall V \in C, U \neq V$

<u>Théorème 1</u> :

La distance minimum d'un code linéaire est égale au poids minimum des mots de code non nul.

<u>Théorème 2</u> :

Un code linéaire de distance minimum d peut corriger E((d-1)/2) erreurs. Si d est paire, le code peut corriger (d-2)/2 erreurs et détecter d/2 erreurs.

Correction directe d'erreurs (3)

<u>Définition</u> : Soit C(n,m) un code linéaire, on appelle classe de U, l'ensemble

$$C(U) = \{U + V, \forall V \in C)$$

<u>Définition</u> : Le vecteur de poids minimum d'une classe est appelé le représentant de cette classe.

<u>Stratégie</u> : Le décodeur doit choisir le vecteur E de poids minimum dans la classe contenant Y.

Code de Hamming, Code BCH

Codes polynomiaux

Il généralise les codes de Hamming et de BCH.

Utilisation de code polynomial pour la détection de paquet d'erreurs.

<u>Principe</u> : A chaque suite binaire bn-1, ….,b1,b0 on associe un polynôme p(x) de degré (n-1) à coefficients binaires :

$$p(x) = b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \ldots. + b_1 x + b_0$$

Les CRC (Cyclic Redundancy Check) sont des codes polynomiaux dont le polynôme générateur g(x) divise

$$x^n + 1$$

Codes cycliques ou CRC (1)

Définition :

Un code est cyclique s'il est binaire et si toute permutation linéaire d'un mot de code est encore un mot de code.

Exemple :

C={000, 101, 011,110}

A toute suite binaire U on fait correspondre un polynôme U(X) dont les coefficients sont les bits de U.

$$U = u_1 u_2 .. u_m \rightarrow U(x) = u_1 + u_2 x + ... + u_m x^{m-1}$$

Codes cycliques ou CRC(2)

Théorème :

Soit C un idéal non vide de Rn :

- ➢ il existe un polynôme irréductible g(x) de degré minimal dans C
- ➢ g(x) est polynôme générateur de C
- ➢ g(x) divise $x^n + 1$
- ➢ la dimension de C est m=n-r où r est le degré de g(x). Le bloc U=f(x) est codé en X=f(x).g(x)
- ➢ La matrice génératrice du code est :

$$\begin{pmatrix} g(x) \\ xg(x) \\ . \\ . \\ . \\ x^{n-r-1}g(x) \end{pmatrix}$$

p*CRC-CCITT* : $g(x) = x^{16} + x^{12} + x^5 + 1$ pour n=260, 500, 980 ou 3560 bits

Codes cycliques ou CRC(3)

Définition du polynôme h(x) de contrôle de parité

➢ $h(x) = (xn + 1)/g(x)$

Définition de la matrice de contrôle de parité H :

➢ $H = (h(x)\ xh(x)\ \ldots.\ x\ n\text{-}r\text{-}1\ h(x))$

Remarque :

G est de dimension n*m et H est de dimension m*n pour un code cyclique.

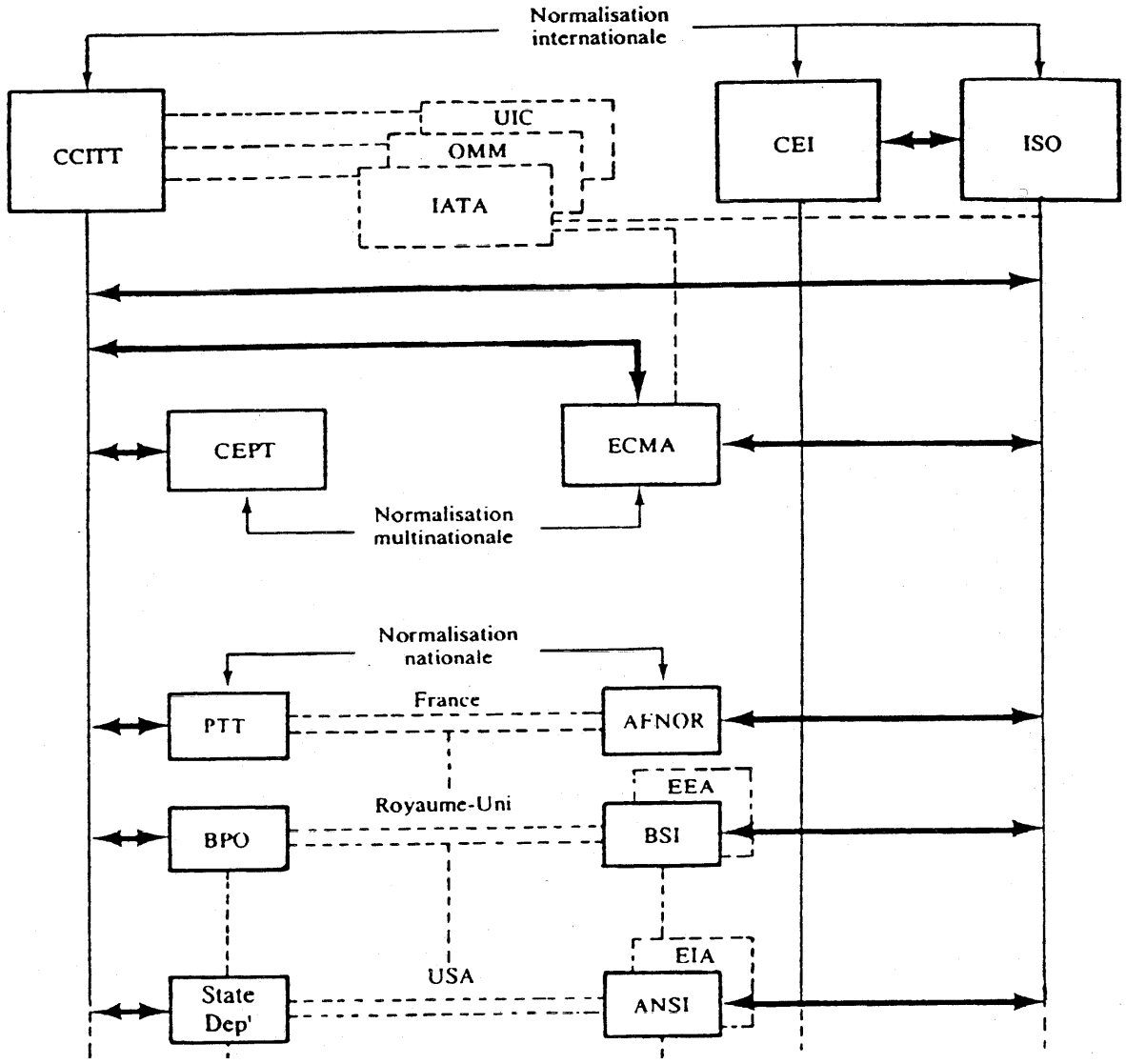
Architecture des Réseaux Informatiques : LES NORMES

Concepts du modèle en couches : Interface, Protocole

Modèle de référence : OSI

Méthodes d'accès aux réseaux

Organismes de normalisation



La couche de liaison : Niveau 2

Elle fournit les procédures et moyens fonctionnels nécessaires pour établir, maintenir et libérer des connexions de liaison de données entre ETTD, ETCD et CCD.

Elle est responsable de l'acheminement sans erreurs de blocs d'information sur la liaison physique.

Dans le cadre de liaison multipoints, elle doit permettre l'identification des différents points de liaison de données utilisant le support physique.

Principaux protocoles :

- ➢ BSC (IBM),
- ➢ SDLC (architecture SNA d'IBM).
- ➢ HDLC (ISO).

Le protocole BISYNC ou BSC

BInary SYNchronous Communication Protocol : protocole de communication synchrone binaire développé par IBM pour connecter ses terminaux à ses ordinateurs.

Procédure du type "envoyer et attendre".

Le protocole opère avec les codes :

- EBDIC ou à six bits (contrôle d'erreurs par CRC),
- ASCII (contrôle d'erreurs par VRC et LRC).

SYN	**SYN**	**SOH**	**En-tête**	**STX**	**Texte**	**ETX ou ETB**	**BCC**

Le protocole SDLC

Le protocole Synchronous Data Link Control est la procédure de liaison introduite par IBM dans le cadre de son architecture SNA.

Elle permet d'émettre jusqu'à sept trames sans attente d'acquittement.

Lorsqu'une erreur se produit le récepteur demande la retransmission de la trame erronée ainsi que des suivantes. Il n' y a donc pas d'acquittement positif.

SDLC a inspiré la procédure dissymétrique en mode normal d'HDLC (Norme ISO).

SDLC a été modifié par l'ANSI pour donner ADCCP (Advanced Data Communication Control Procedure) et par le CCITT pour donner LAP (Link Access Procedure) utilisé dans les réseaux X25.



Le protocole HDLC

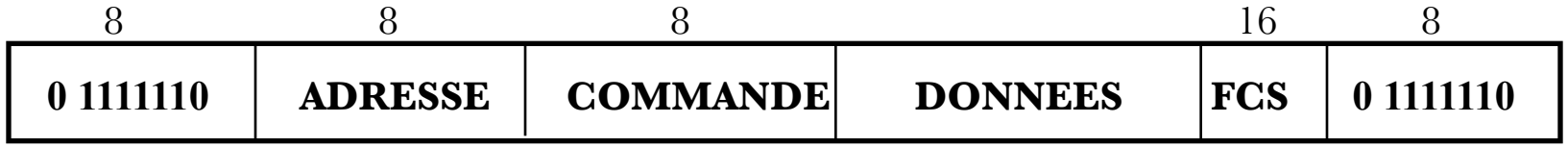
HDLC (High level Data Link Control) est un ensemble de classes de procédures normalisées par l'ISO en 1976.

ADRESSE : 11000000 (trame de E vers S) et 10000000 (trame de S vers E)

COMMANDE : spécifie le type de la trame, le numéro de la trame, les acquittements, …

DONNEES : champ contenant les données de service

FCS -> détection d'erreurs de transmission (adresse, contrôle et informations)
CRC sur 16 bits



Le protocole HDLC : mode de communication (1)

Trois modes de communication sont en vigueur :

- ➢ Asynchronous Response Mode (ARM): Mode de communication dans lequel une station secondaire peut émettre sans recevoir de permission de la station primaire.

- ➢ Normal Response Mode (NRM) : Mode de communication dans lequel une station secondaire ne peut émettre qu'avec l'autorisation de la station primaire.

- ➢ Asynchronous Balanced Mode (ABM): Mode de communication dans lequel les deux stations en communication ont un statut équivalent de station primaire (Mode adopté par X25).

Le protocole HDLC : mode de communication (2)

Répertoire de base

Mode non équilibré Primaire/Secondaire				Mode équilibré Primaire/Primaire	
Réponse asynchrone (**ARM**)		Réponse normale (**NRM**)		Asynchrone équilibrée (**ABM**)	
Station Primaire	Station Secondaire	Station Primaire	Station Secondaire	Station Primaire	Station Primaire
Commande I RR RNR **SARM** DISC	Réponse I RR RNR UA DM FRMR	Commande I RR RNR **SNRM** DISC	Réponse I RR RNR UA DM FRMR	Commande I RR RNR **SABM** DISC	Réponse I RR RNR UA DM FRMR
(Numérotation modulo 8)					

Le protocole HDLC : Type de trame (1)

Il y a 3 types de trames HDLC : Information, Supervision, Unnumbered.

Le champ commande permet de les distinguer.

Trame d'Information	0	N(S) : Numéro de Séquence		P/F	N(R) : Numéro de Séquence attendue
Trame de Supervision	1	0	S : Bits de Supervision	P/F	N(R) : Numéro de Séquence attendue
Trame non-numerotée	1	1	M : Bits de Modification	P/F	Modificateur

Le protocole HDLC : Type de trame (2)

P/F est le bit indiquant :

- ➢ P : une demande de réponse immédiate pour les transmissions primaires,
- ➢ F : une réponse au bit P, mais aussi en mode normal, une trame finale pour les transmissions secondaires

Trame I : Utilisée par les stations primaires et secondaires pour effectuer un transfert d'information.

Trame S : servent à effectuer des fonctions usuelles de supervision comme l'envoi d'un accusé de réception, le rejet d'une trame, la suspension de la transmission, la demande de retransmission.

Trame U : Format non séquentiel permettant de définir 32 fonctions supplémentaires de supervision.

Le protocole HDLC : Trames de Supervision (1)

Fonctions de supervision de base :
- ACK,
- demande de retransmission

COMMANDE	3	4	REPONSE
RR (Receive Ready)	0	0	RR
REJ (Reject)	0	1	REJ
RNR (Receive Not Ready)	1	0	RNR
SREJ (Selective REJect)	1	1	SREJ

Le protocole HDLC : Trames de Supervision (2)

RR :

- ➢ utilisé par une station primaire ou secondaire,
- ➢ indique qu'elle est prête à recevoir,
- ➢ ou ACK des N(R)-1 trames.

REJ :

- ➢ Demande de retransmission de trames à partir de N(R)

RNR :

- ➢ La station n'est pas prête à recevoir,
- ➢ ACK des N(R)-1 trames précédentes.

SREJ :

- ➢ Demande de retransmission de la trame N(R).

Le protocole HDLC : Trames non-numérotées (1)

Commandes et requêtes supplémentaires en mode non séquentiel

1	2	3	4	5	6	7	8	Bits de Commande
1	1	1	1	P	0	0	0	SARM (mode réponse autonome)
1	1	0	0	P	0	0	1	SNRM (mode réponse normale)
1	1	0	0	P	0	1	0	DISC (libération)
1	1	0	0	F	1	1	0	UA (Acceptation non numérotée)
1	1	1	0	F	0	0	1	CMDR (commande rejetée)

Le protocole HDLC : Trames non-numérotées (2)

SARM (Set Asynchronous Response Mode) :

- ➢ Dans ce mode, une station secondaire peut émettre à son gré. Mais elle doit auparavant accepter ce mode (Rep. UA) et mettre ses compteurs à 0,
- ➢ Retransmission s'il n'y a pas d'ACK par la station primaire au bout d'un certain temps (à définir pour chaque système).

SNRM (Set Normal Response Mode) :

- ➢ Invite une station secondaire à fonctionner en mode normal (transmission sur invitation de la station primaire)
- ➢ Acceptation du mode UA, compteur à 0,
- ➢ Après la permission d'émettre (RR, P=1), la station secondaire peut émettre plusieurs trames (dernière avec F=1),
- ➢ Puis arrêt jusqu'à nouvelle autorisation.

Le protocole HDLC : Trames non-numérotées (3)

DISC : est utilisée sur les réseaux commutés pour libérer la liaison.

➢ Déconnexion d'une station secondaire après émission de rep. UA.

UA :

➢ Réponse utilisée par une station secondaire pour ACK d'une commande.

CMDR :

➢ Réponse utilisée par une station secondaire pour rejeter une commande bien reçue.

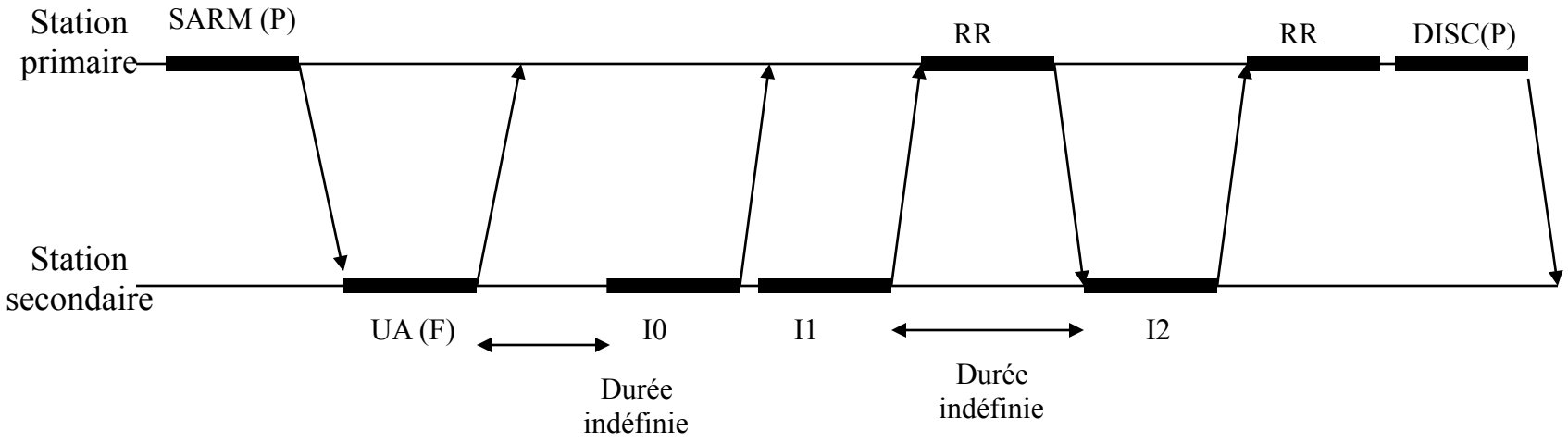
Champs information

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Commande rejetée								0	N(S)			0	N(R)			W	X	Y	Z

W=1 -> commande reçue non valable ou non utilisée
X=1 -> commande reçue non valable car le champ info correspondant n'est pas compatible
Y=1 -> le champ info reçu déborde de la capacité mémoire de la station
Z=1 -> la commande reçue comporte un N(R) incorrect.

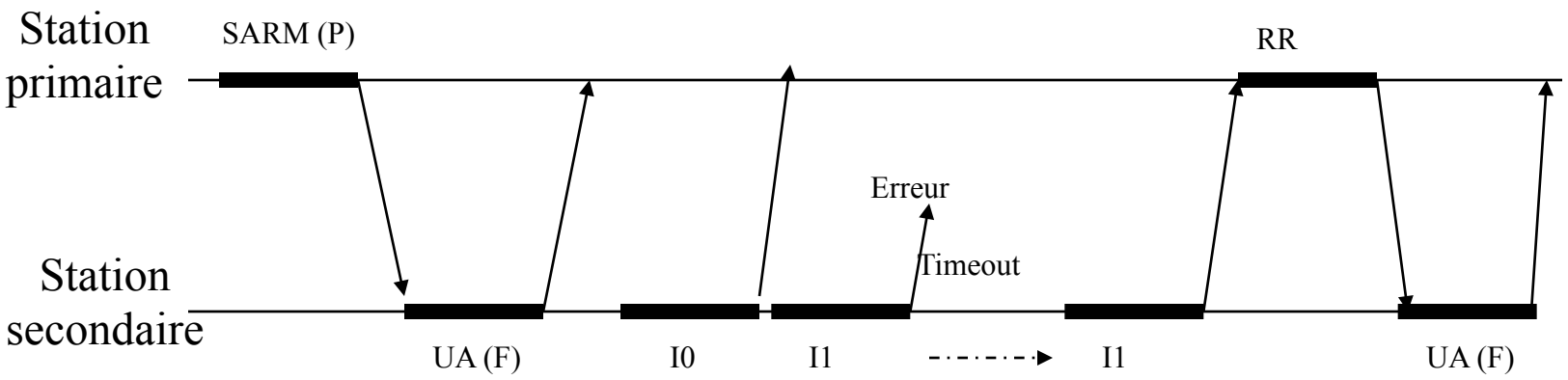
Exemple de communication en mode SARM (1)

Etablissement du mode SARM à l'alternat avec transfert d'information depuis le secondaire seulement



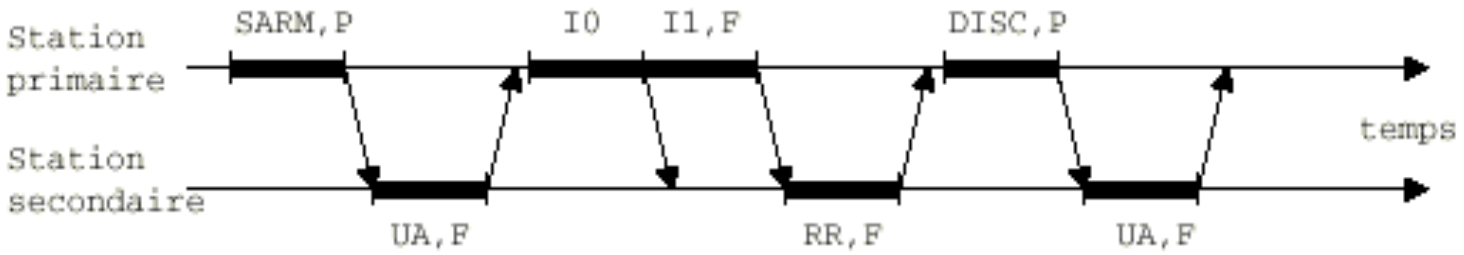
Exemple de communication en mode SARM (2)

Cas d'une erreur de transmission

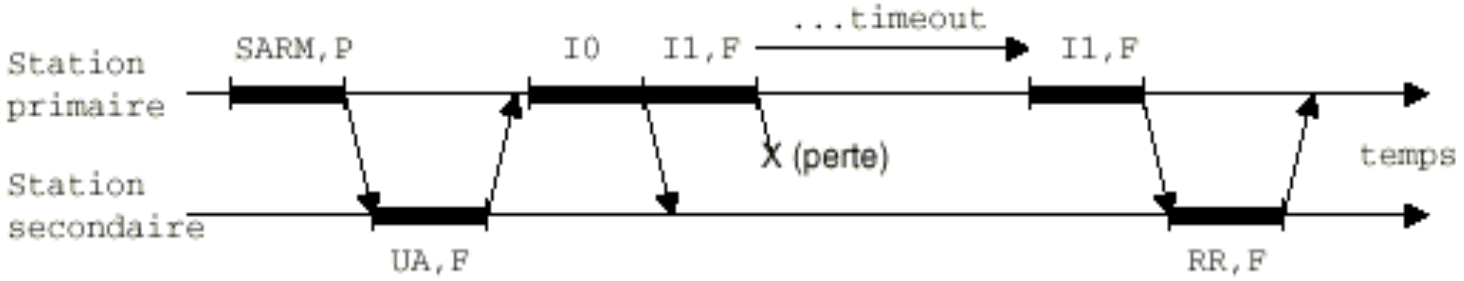


Exemple de communication en mode SARM (3)

- Transmission sans erreur en mode ARM

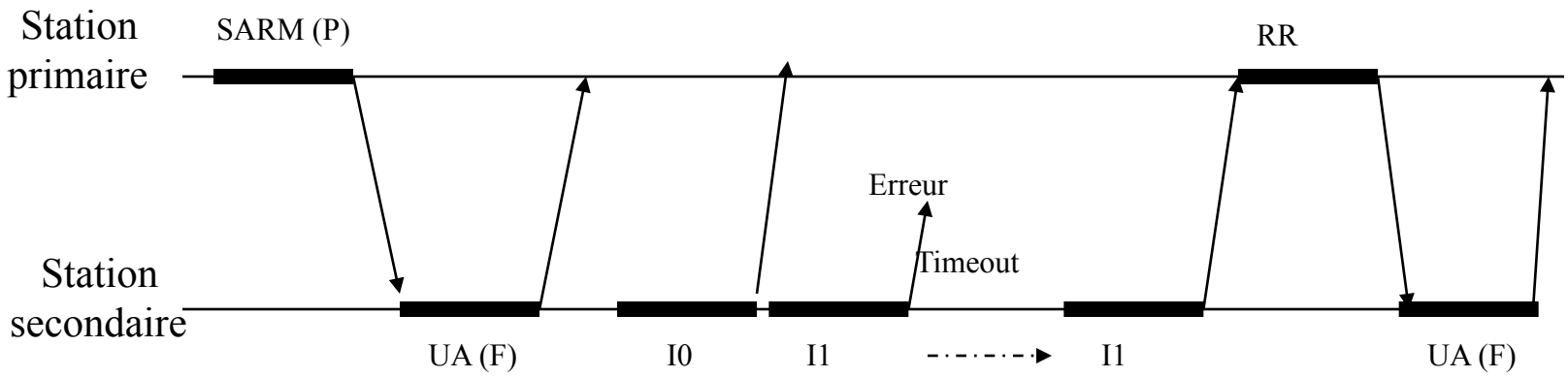


- Présence d 'une erreur de transmission en mode ARM



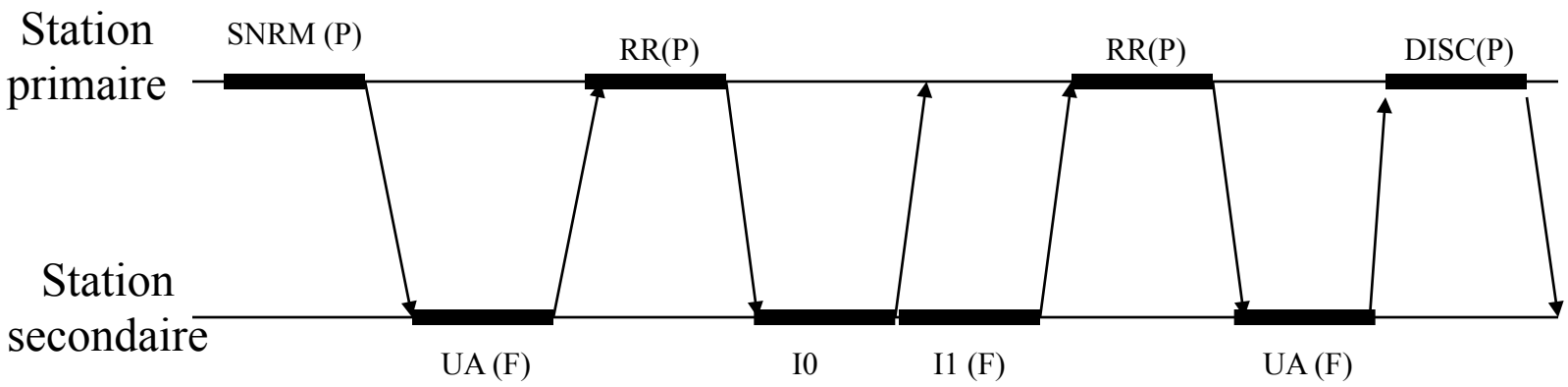
Exemple de communication en mode SARM (4)

Tentative de transmission en mode ARM à l'alternat avec transfert d'information par le primaire et le secondaire



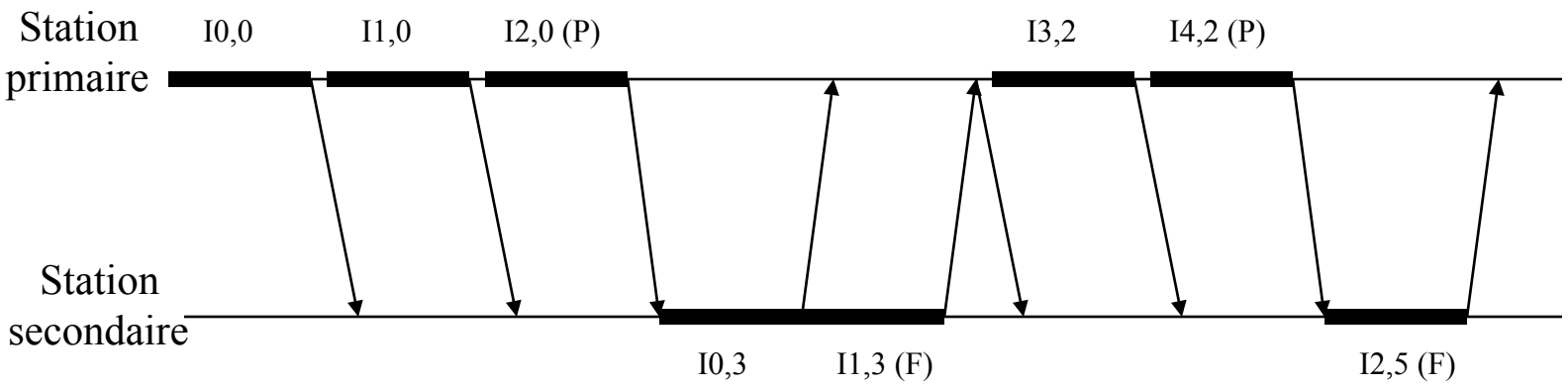
Exemple de communication en mode SNRM (1)

Etablissement du mode NRM à l'alternat avec transfert d'information depuis le secondaire seulement



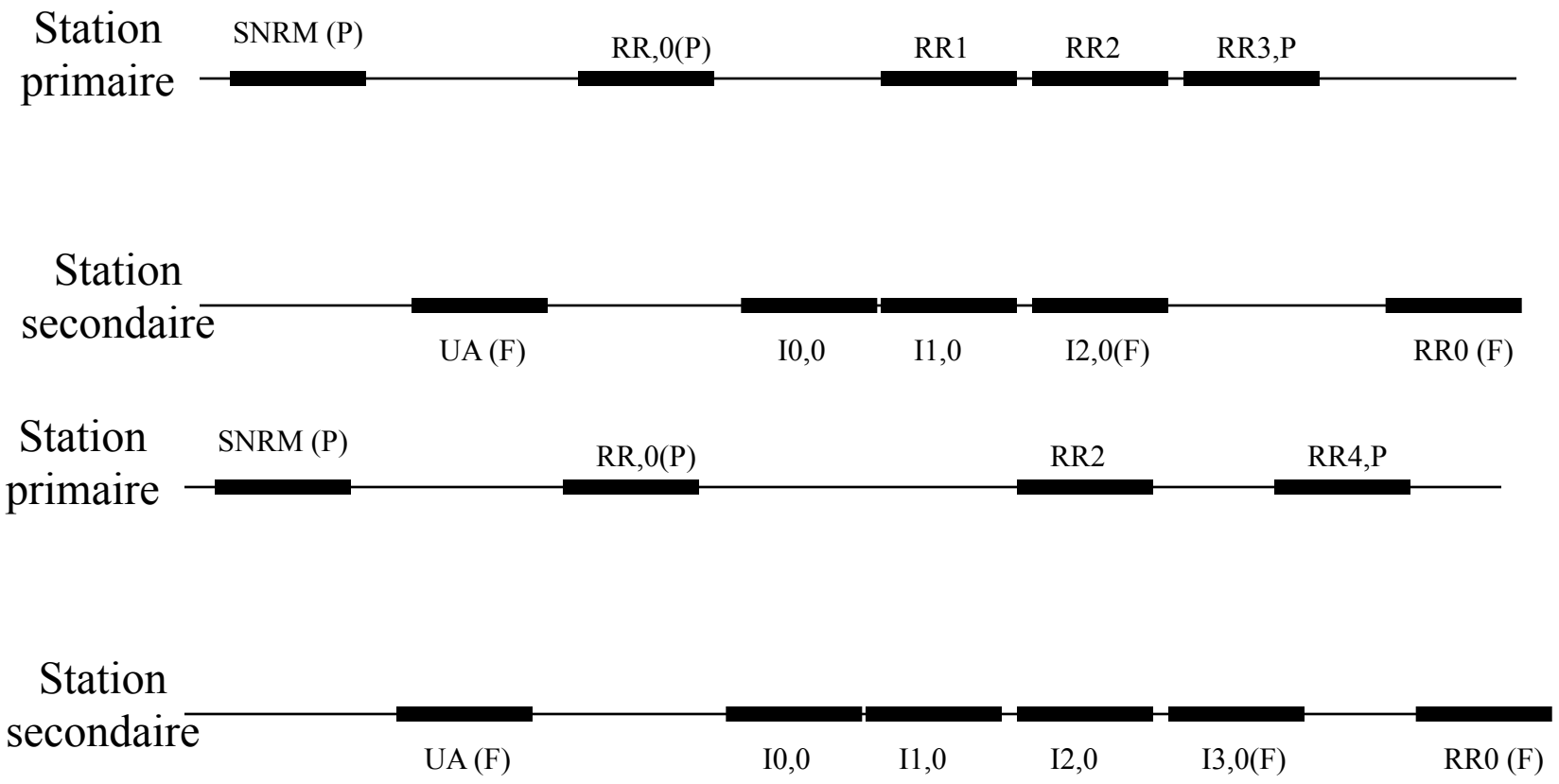
Exemple de communication en mode SNRM (2)

Mode NRM à l'alternat avec transfert d'information depuis le primaire et le secondaire.



Exemple de communication en mode SNRM (3)

Etablissement du mode NRM bidirectionnel avec transfert d 'information depuis le secondaire seulement



Analogies

- **De nombreux mécanismes présents avec HDLC se retrouvent dans les couches supérieures (ex: TCP)**

numérotation des trames	=> numéro de séquence TCP
acquittements	=> champ ACK de l'en-tête TCP
timer de retransmission	=> idem avec TCP
fenêtre d'émission	=> idem avec TCP
protection par FCS	=> protection par checksum

- **mais on ne travaille pas au même niveau (2 versus 4) !**

Protocoles dérivés

LAP B : Link Access Protocol Balanced

- ➢ Caractéristiques : asynchrone et fullduplex (ABM),
- ➢ Utilisé par X25 (Transpac en France)

LAP X : liaison half duplex,

LAP D : utilisé par le RNIS.

La couche réseau

Rôle : Fournir les procédures et moyens fonctionnels nécessaires à l'échange des informations données par la couche transport.

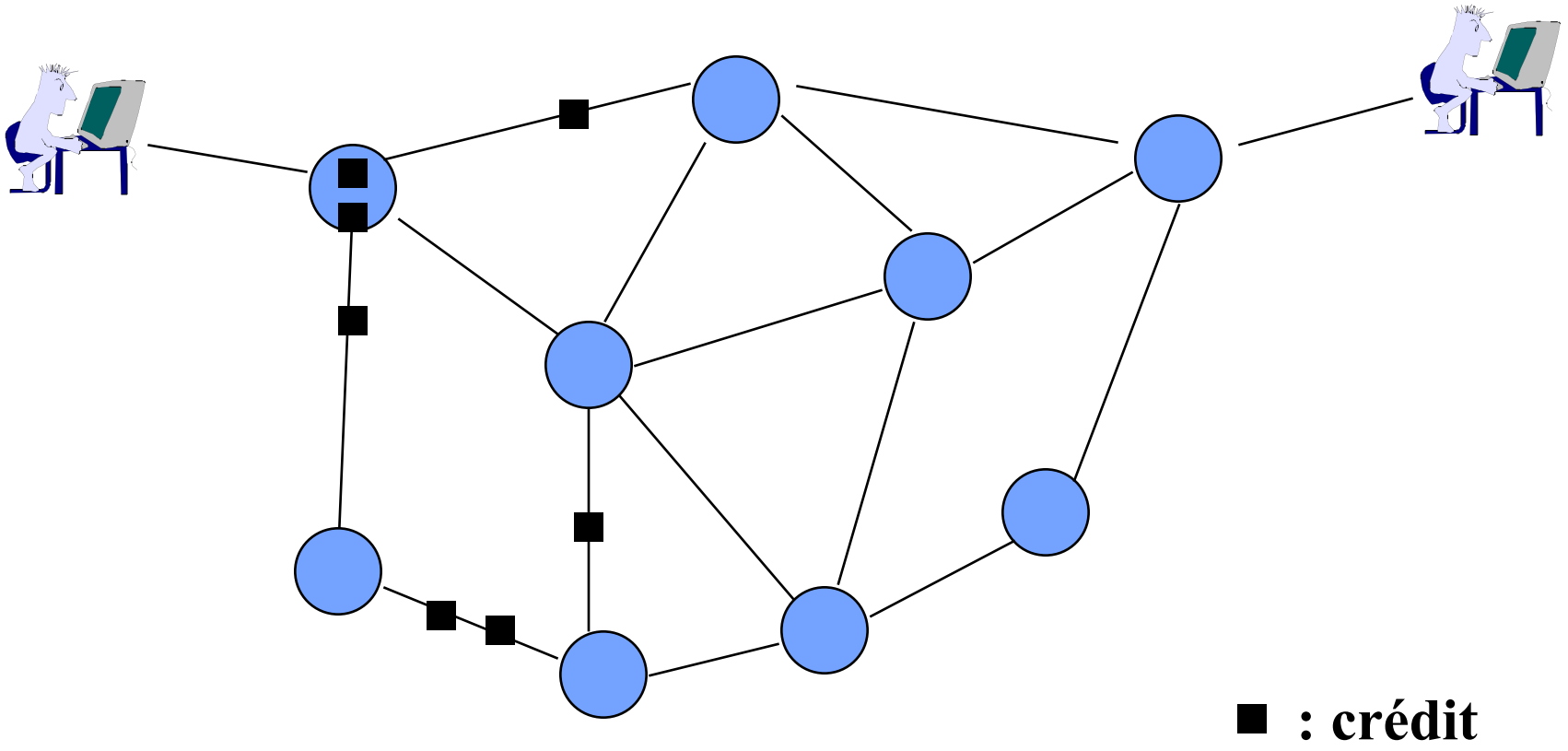
Services offerts par la couche réseau :

- service d'adressage,
- intégrité du transport des données :
  - contrôle de flux,
  - service de routage,
  - détection d'erreurs de transmission
- services facultatifs :
  - livraison en séquence de paquets de données,
  - transfert accéléré de paquets de supervision,
  - multiplexage de connexions réseau …

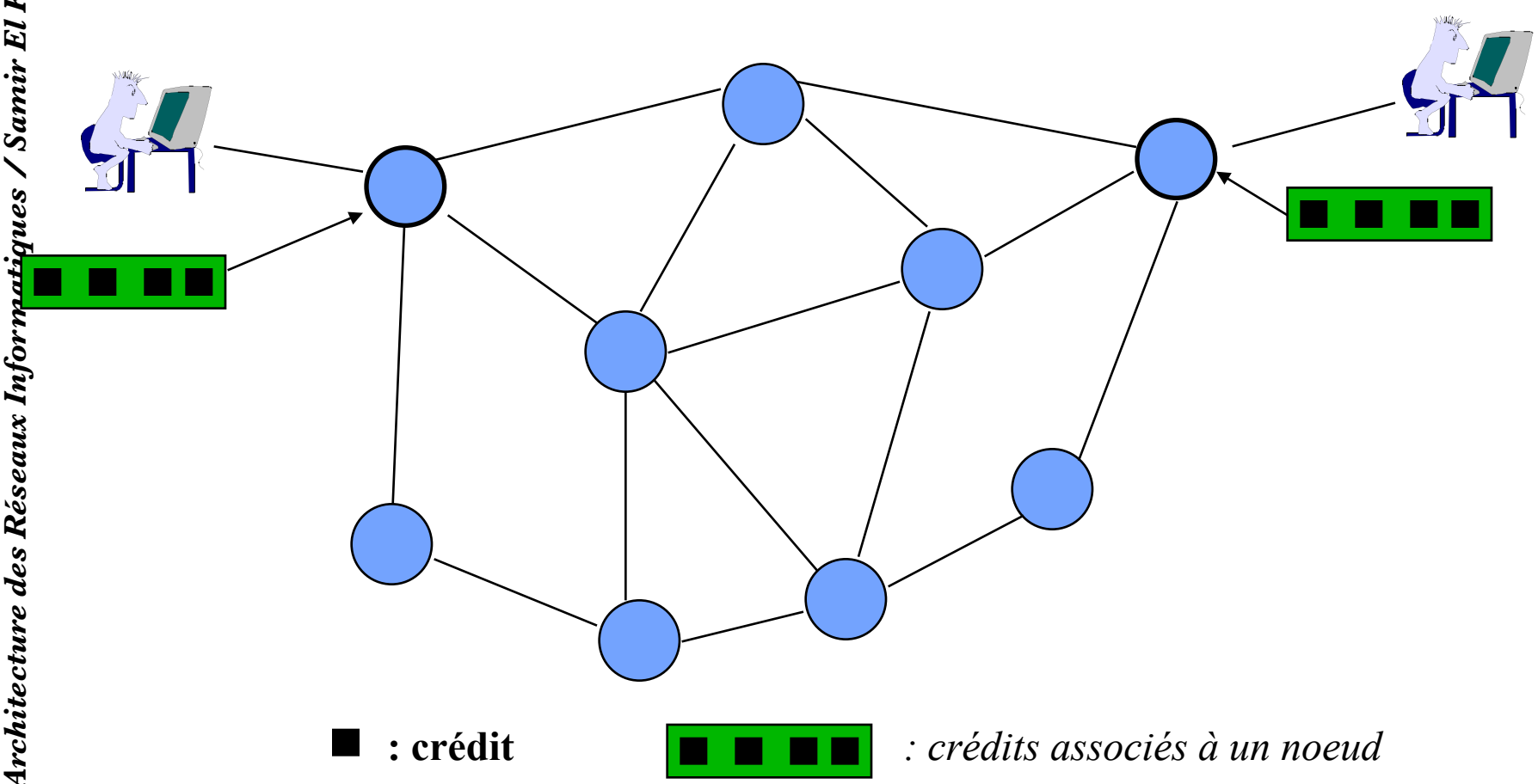
Protocoles de la couche réseau :

- le datagramme,
- le circuit virtuel (X25).

Politique à seuil utilisant des **crédits** circulant dans le réseau



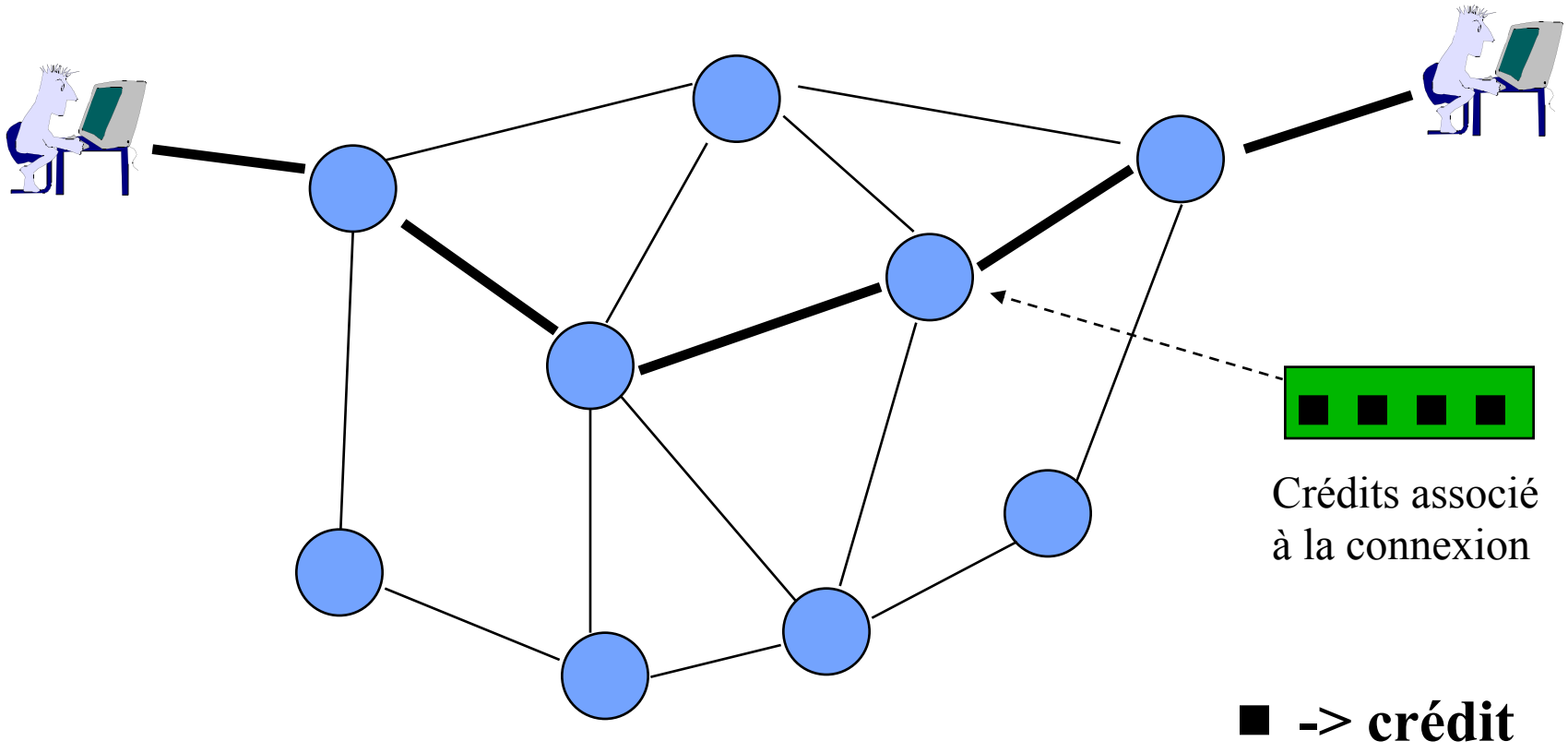
Politique à seuil utilisant des **crédits** dédiés à des nœuds origines ou à des connexions réseau



■ **: crédit**

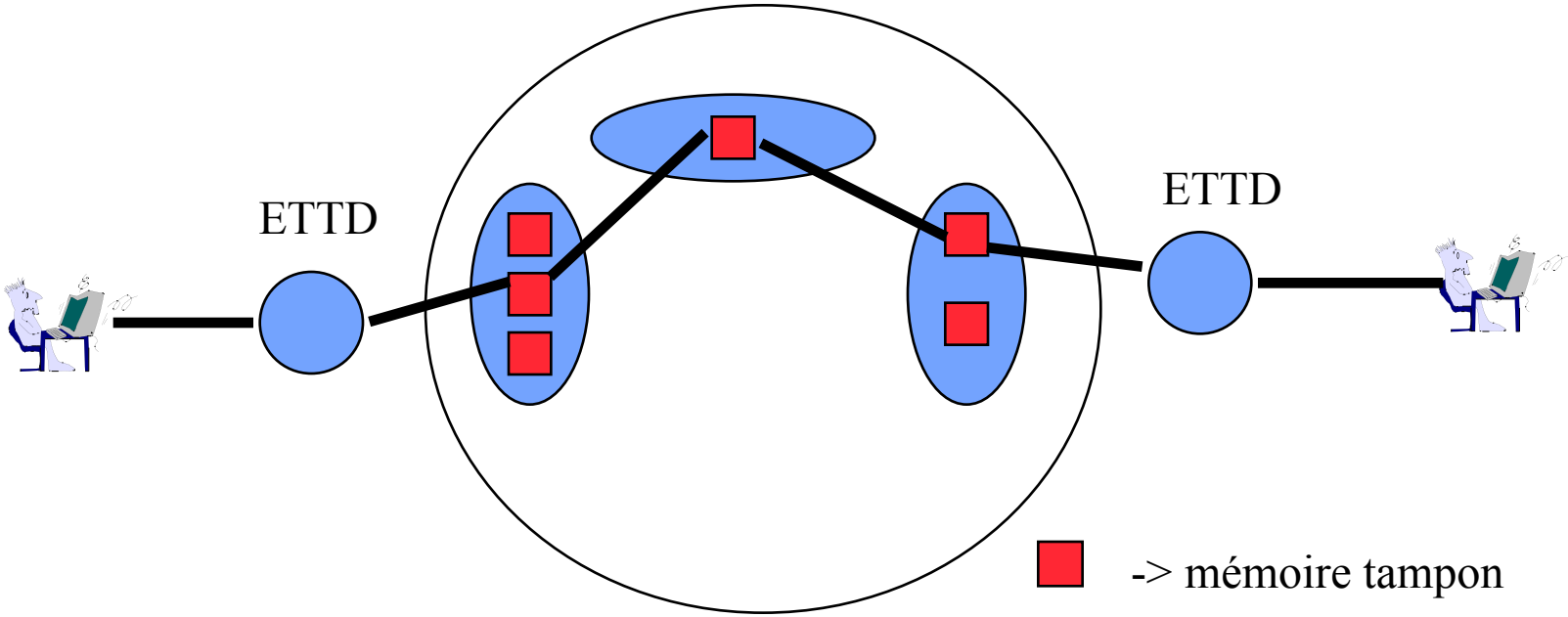
: crédits associés à un noeud

Politique à seuil utilisant des **crédits** associés à chaque connexion de transport (contrôle de niveau 4)



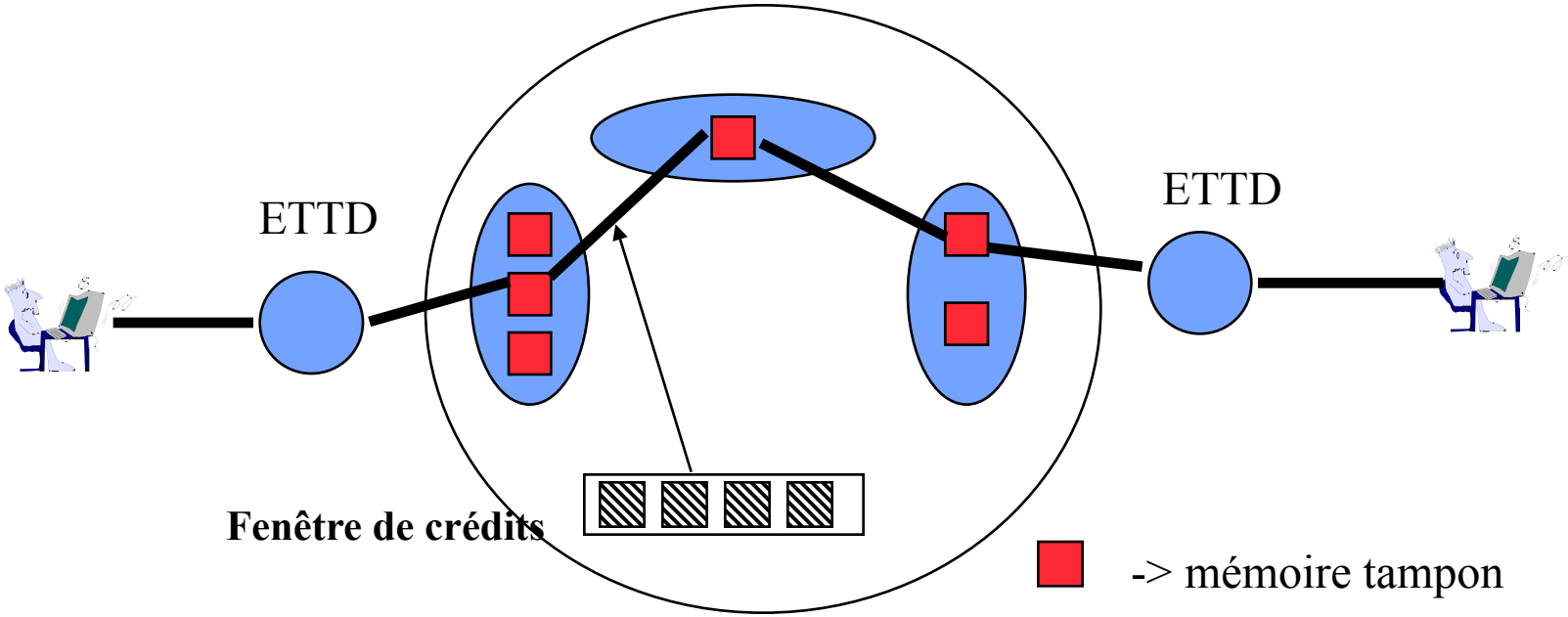
Politique de pré-allocation

<u>Principe</u> : Avant de faire transiter un message, on lui réserve des ressources pour éviter la congestion des nœuds. Les ressources sont principalement les liaisons et les mémoires tampons des nœuds de commutation.



Pré-allocation et politique à seuil

<u>Principe</u> : Ajout d'une fenêtre de crédits de contrôle sur la connexion de transport.



Contrôle de congestion

Utilisation de mémoires tampons de réserve,

Limiter la durée de vie d'un paquet,

Abandon des paquets voulant passer par une liaison surchargée,

Fermeture autoritaire d'un circuit virtuel ;

- ➢ une fois que la congestion a disparue, la reprise est gérée par la couche transport.

Le routage

<u>Objectif</u> : Permettre la reconfiguration rapide du réseau en cas de panne, anomalie ou congestion.

<u>Principal outil</u> : La table de routage.

Différents types de routage :

- ➢ centralisé,
- ➢ distribué,
- ➢ statique,
- ➢ dynamique
- ➢ adaptatif.

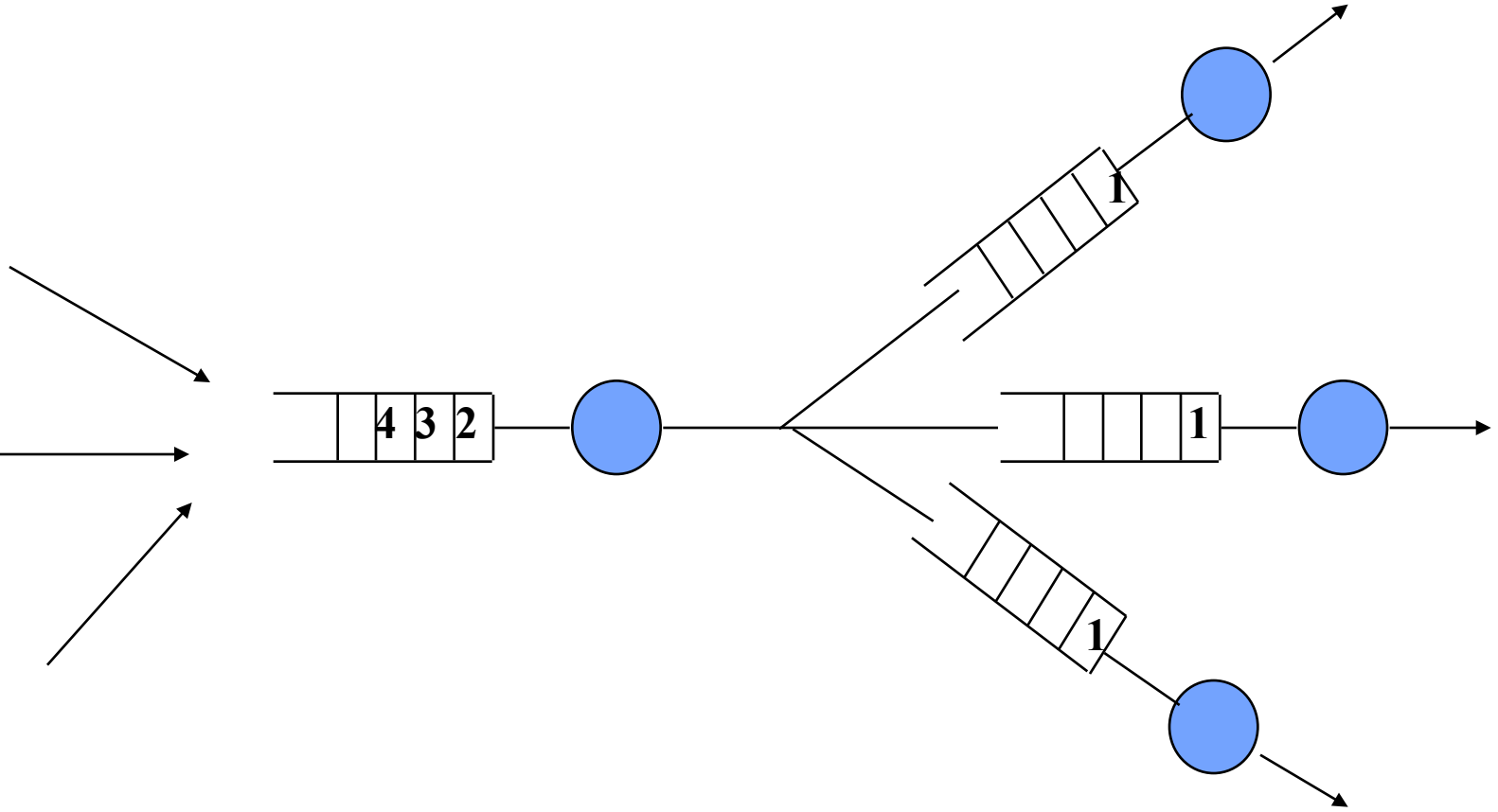
Le routage centralisé

<u>Principe</u> : Cette technique s'appuie sur l'utilisation d'un centre de contrôle.

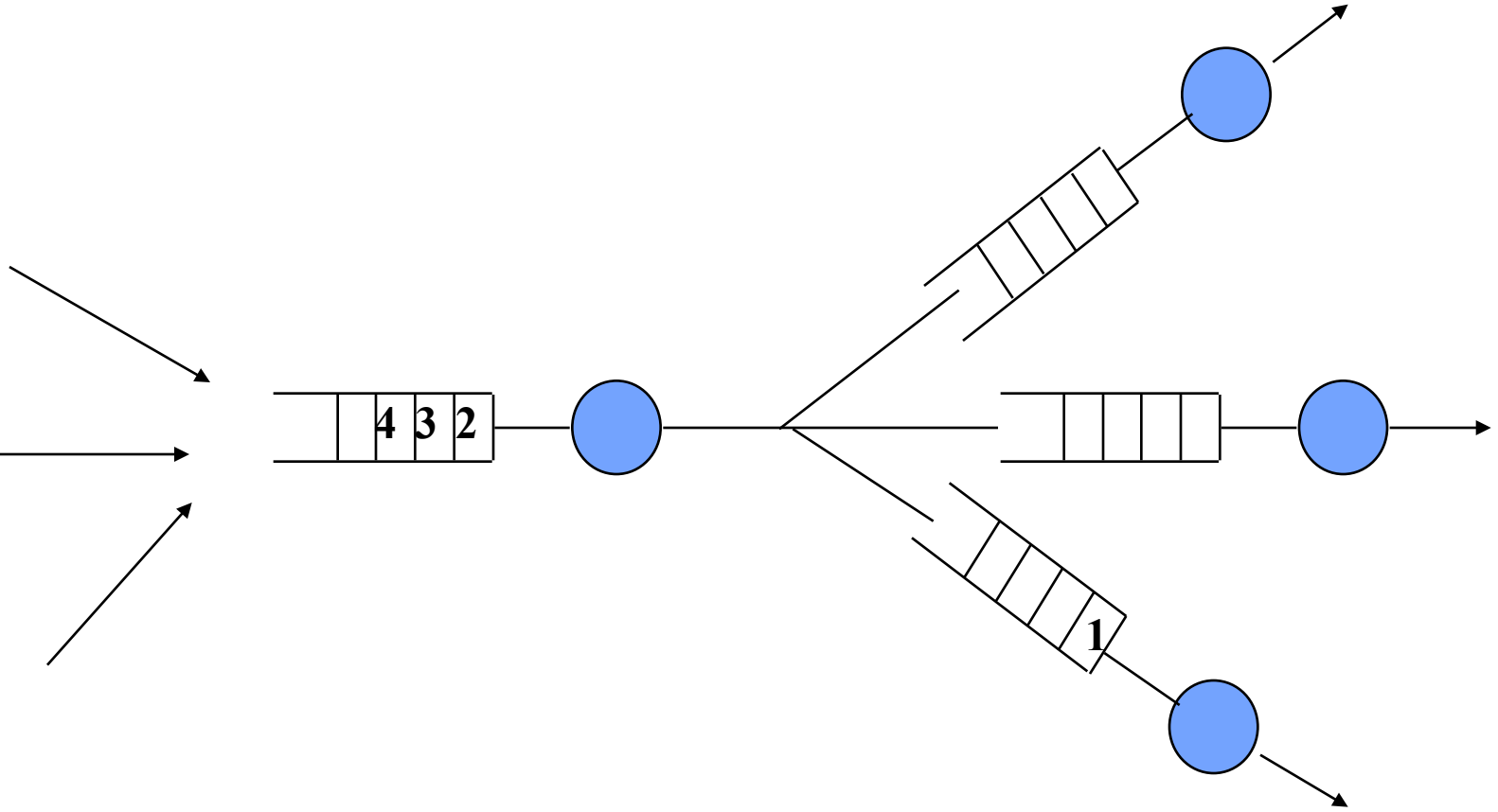
Mise en œuvre :

- ➢ <u>Statique</u> : En cas de défaillance d'un élément du réseau le centre de contrôle envoie une nouvelle table de routage à chaque nœud.
- ➢ <u>Dynamique synchrone</u> : Les nœuds envoient régulièrement un compte-rendu de leur état. Le centre de contrôle élabore alors une table de routage qu'il envoie à tous les nœuds à des dates fixées.
- ➢ <u>Dynamique asynchrone</u> : Les nœuds envoient un compte-rendu de leur état lorsque celui-ci à "suffisamment" changé. Le centre de contrôle élabore des tables de routage au fur et à mesure qu'il reçoit des informations. Il envoie à tous les autres la première table qui lui paraît "suffisamment" différente de la précédente.

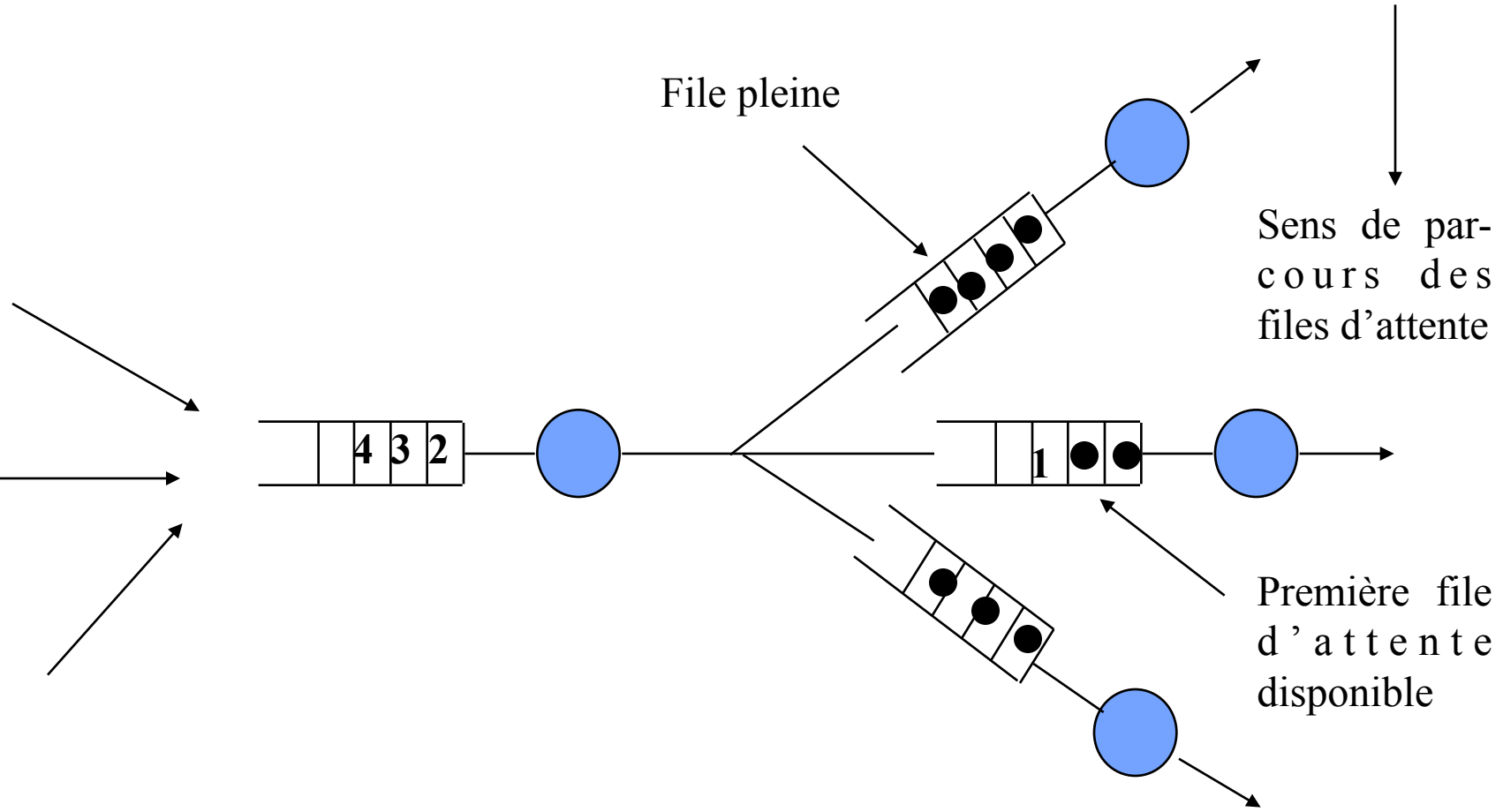
Le routage distribué par inondation



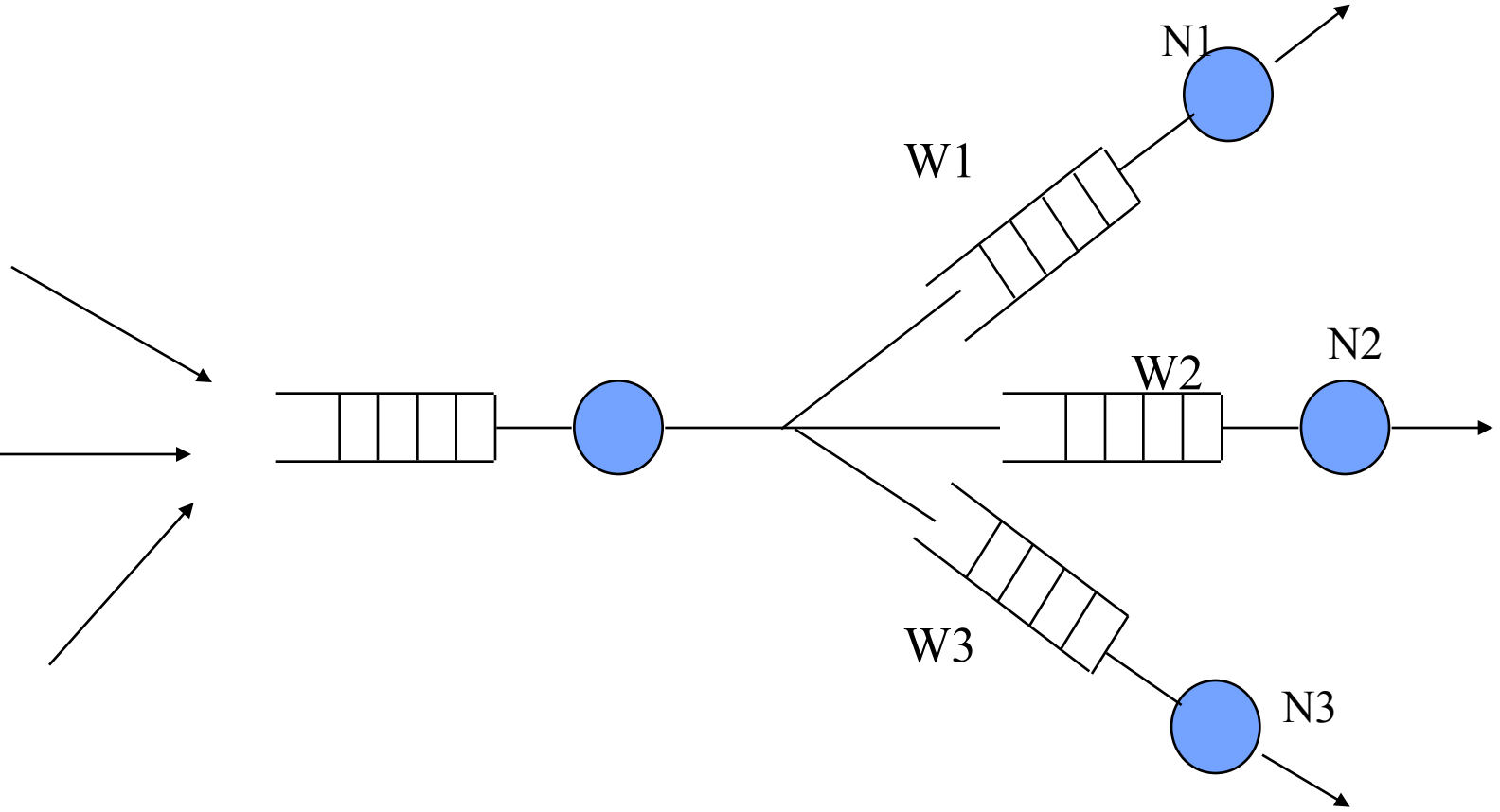
Le routage distribué aléatoire



Le routage distribué - Hot Potatoe



Le routage distribué adaptatif (1)



Le routage distribué adaptatif (2)

	M1	M2	M3	M4
N1	160	260	180	218
N2	140	255	175	200
N3	100	247	140	220

W1	W2	W3
26	40	64

	vers M1	vers M2	vers M3	vers M4
Par N1	160+26	260+26	180+26	218+26
Par N2	140+40	255+40	175+40	200+40
Par N3	100+64	247+64	140+64	220+64

TABLE DE ROUTAGE

Destination	M1	M2	M3	M4
File de sortie	N3	N1	N3	N2

L'adressage

Objectif :

- ➢ fournir l'adresse du destinataire,
- ➢ indiquer l'adresse du noeud source.

Chaque adresse dépend du contexte dans lequel elle est définie.

Contexte lié à un mécanisme d'adressage spécifique.

- ➢ Des entités différentes peuvent avoir la même adresse dans des contextes différents,
- ➢ A des instants différents, la même adresse peut représenter des entités différentes dans le même contexte (cf. mécanisme d'adressage d'AppleTalk),
- ➢ Une entité donnée peut avoir des adresses distincts dans des contextes différents (cf. un routeur multi-protocoles ou un ordinateur connecté à des réseaux différents).

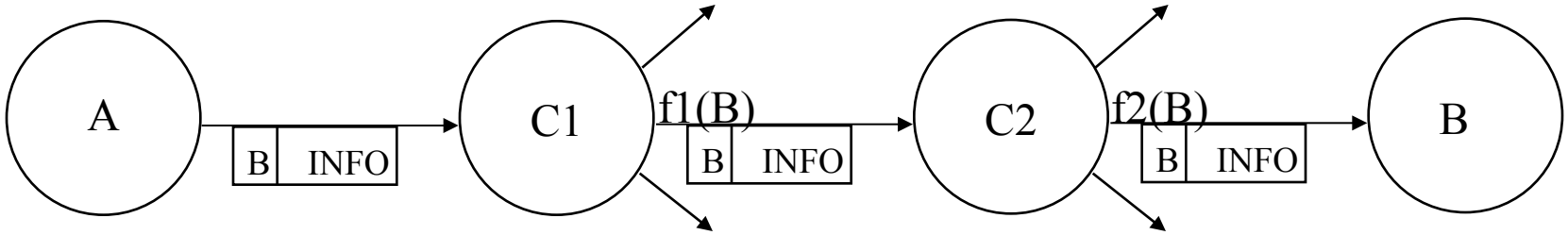
Adresse globale : Identifie une entité dans plusieurs contextes.

Adresse locale : correspond à un contexte limité

Adressage global distribué

L'adresse du destinataire accompagne le bloc de données.

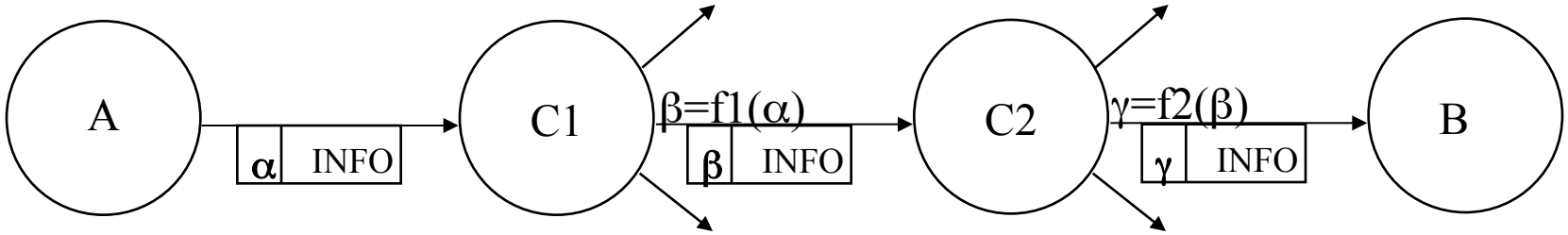
Utilisée dans TRANSPAC pour l'acheminement du paquet d'appel dans la phase d'établissement du circuit virtuel.



Adressage en cascade

L'entité destinataire est identifiée sur un chemin par une suite d'adresses locales.

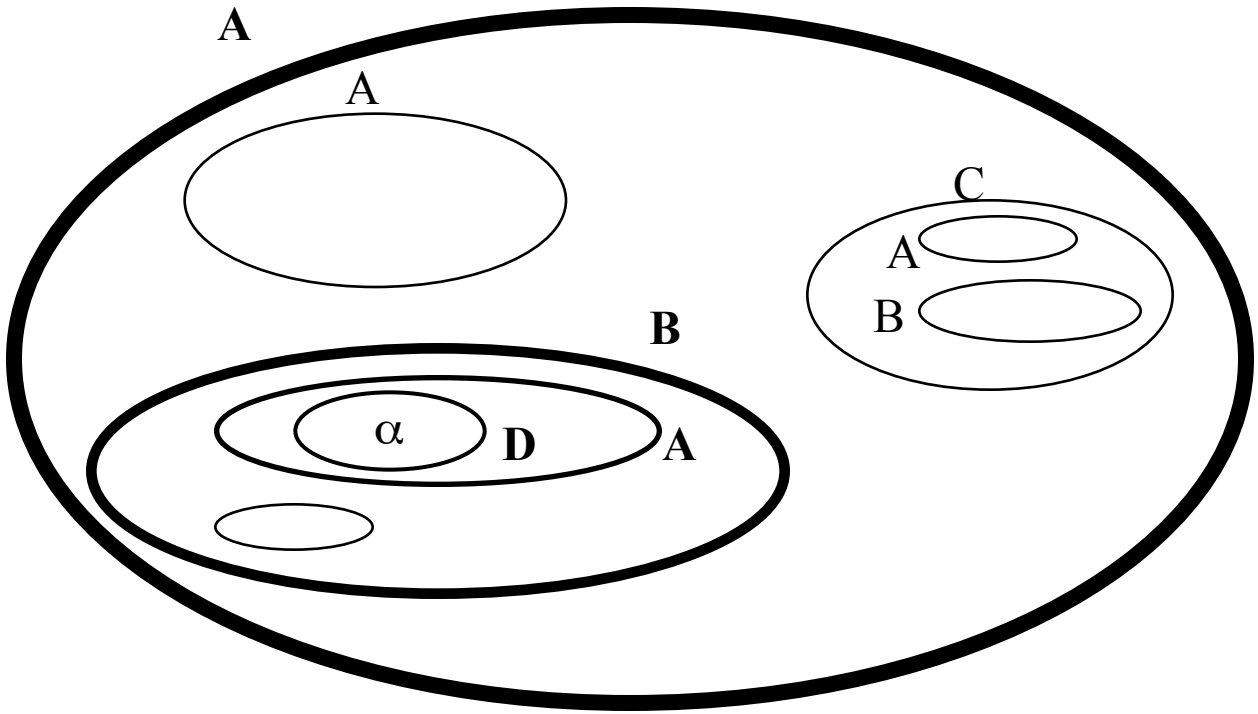
<u>Problème</u> : En cas de panne d'une ressource, il faut reconstituer un chemin d'adresses locales en partant d'un nœud ayant une adresse globale.



Adresse hiérarchique

Adresse = adresse globale (A, B, A, D)

Systématiquement employée dans les réseaux de grand taille.



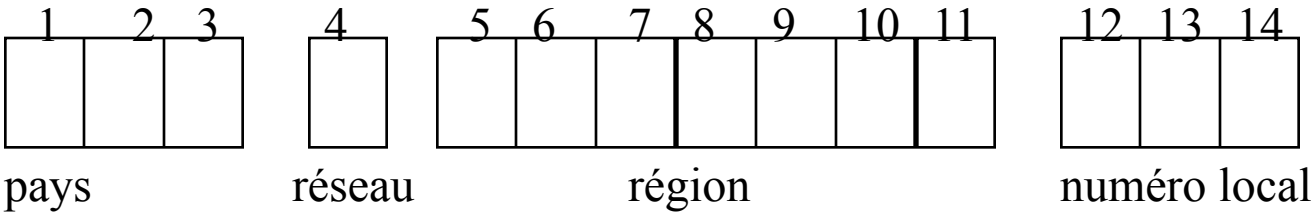
Principales normes d'adressage

Elles utilisent des séquences de chiffre décimaux. Chaque chiffre est codé sur un demi-octet

X25 niveau 3 :



X121 :



Norme ISO 8073 et recommandation X224 du CCITT

Transport de bout en bout,

➢ Assure le transfert entre deux entités quelque soit le type des réseaux traversés.

Sélection d'une qualité de service :

➢ optimisation des ressources réseaux disponibles,

➢ taux d'erreur résiduel faible,

➢ probabilité de panne faible,

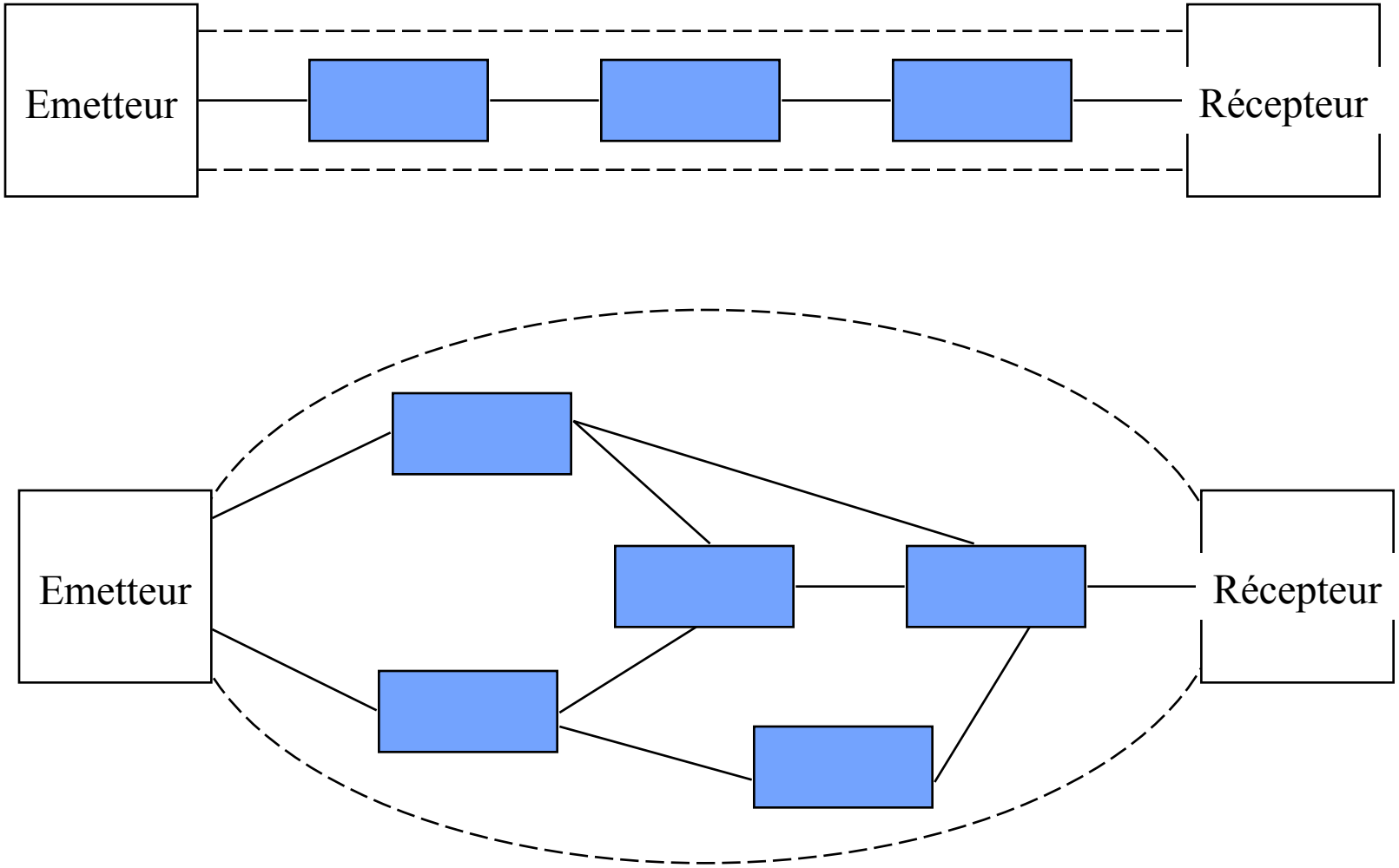
➢ gestion des connexions (délai d'établissement, sécurité des données, ...)

Transparence :

➢ Toute configuration binaire doit être acceptée pour être délivrée sans modification.

Adressage : indépendant des méthodes des couches inférieures.

Transport de bout en bout



Principales fonctions (1)

Concaténation/séparation :

➢ Cette fonction permet de concatener plusieurs TPDU dans une même NDSU. L'entité de transport réceptrice doit alors séparer les différentes TPDU qui sont dans une même NSDU.

Segmentation/Réassemblage :

➢ Cette fonction est utilisée pour éclater une TDSU unique (message) en plusieurs TPDU de données (paquets) qui seront ensuite réassembler pour retrouver leur format d'origine sur l'unité réceptrice.

Multiplexage/Démultiplexage :

➢ Fonction utilisée pour partager une connexion de réseau unique entre plusieurs connexions de transport.

Principales fonctions (2)

Contrôle de flux :

➢ Fonction utilisée pour réguler le flot de TPDUs de données entre deux entités de transport sur une connexion de transport.

Reprise sur erreur :

➢ Permet la reprise sur les erreurs signalées et, si le mécanisme existe, sur les erreurs détectées et non signalées.

➢ Permet de traiter les déconnexions ou les réinitialisations signalées par le réseau sans que l'utilisateur du service de transport n'en soit informé.

Détection d'erreur :

➢ Permet de détecter la perte, la duplication, le déséquencement, ou l'altération de TPDU.

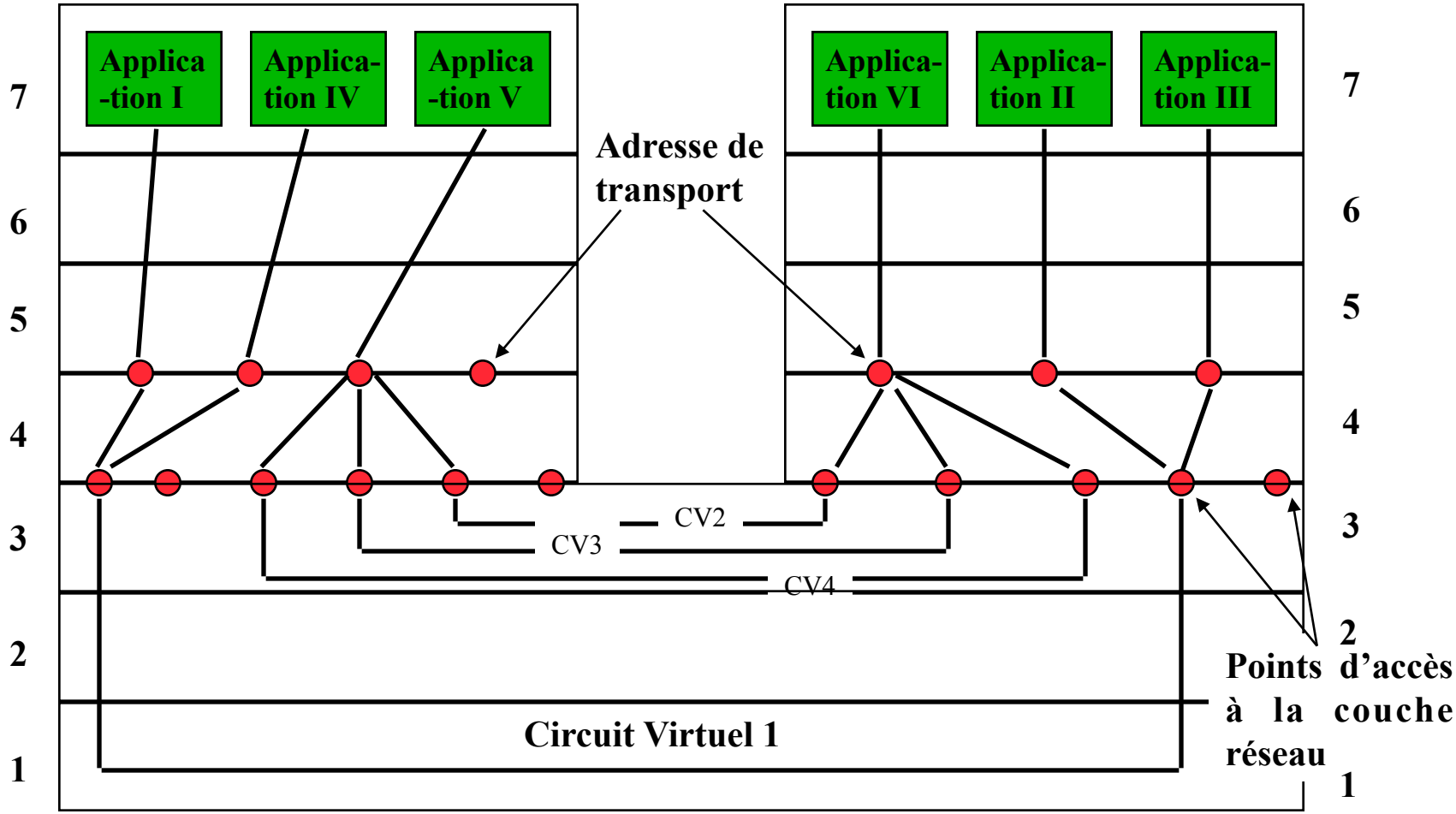
Données express :

➢ Permet de contourner le contrôle de flux des TPDU de données normales, le flot de données express étant sujet à un contrôle de flux séparé.

Classes de protocole de transport

Classe de protocole / Mécanismes	0	1	2	3	4
Segmentation	oui	oui	oui	oui	oui
Concaténation	non	oui	oui	oui	oui
Contrôle de flux	non	non	oui	oui	oui
Numérotation des TPDU	non	oui	oui	oui	oui
Multiplexage	non	non	oui	oui	oui
Données exprès	non	oui	oui	oui	oui
Reprise sur erreur ou resynchronisation	non	oui	non	oui	oui
Détection d'erreur	non	non	non	non	oui

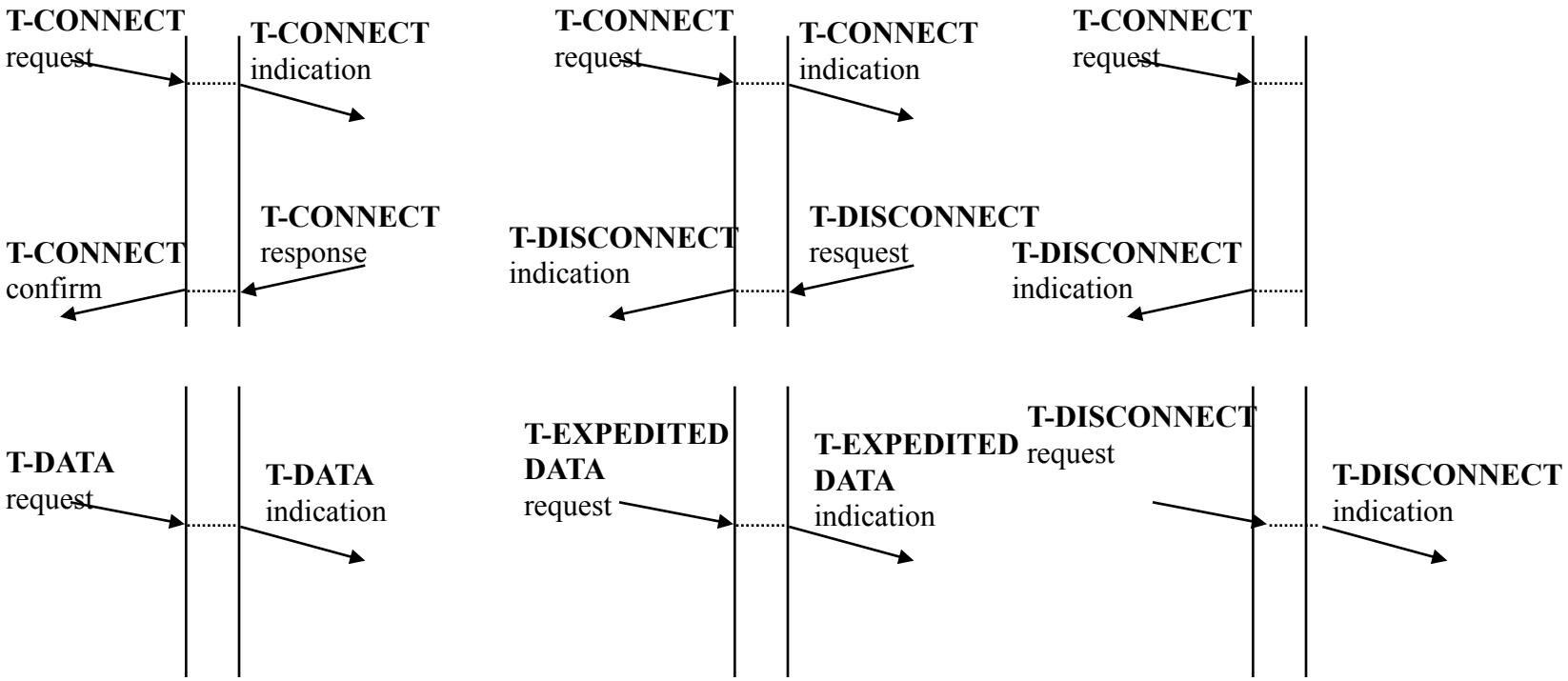
Adresses et chemins des données



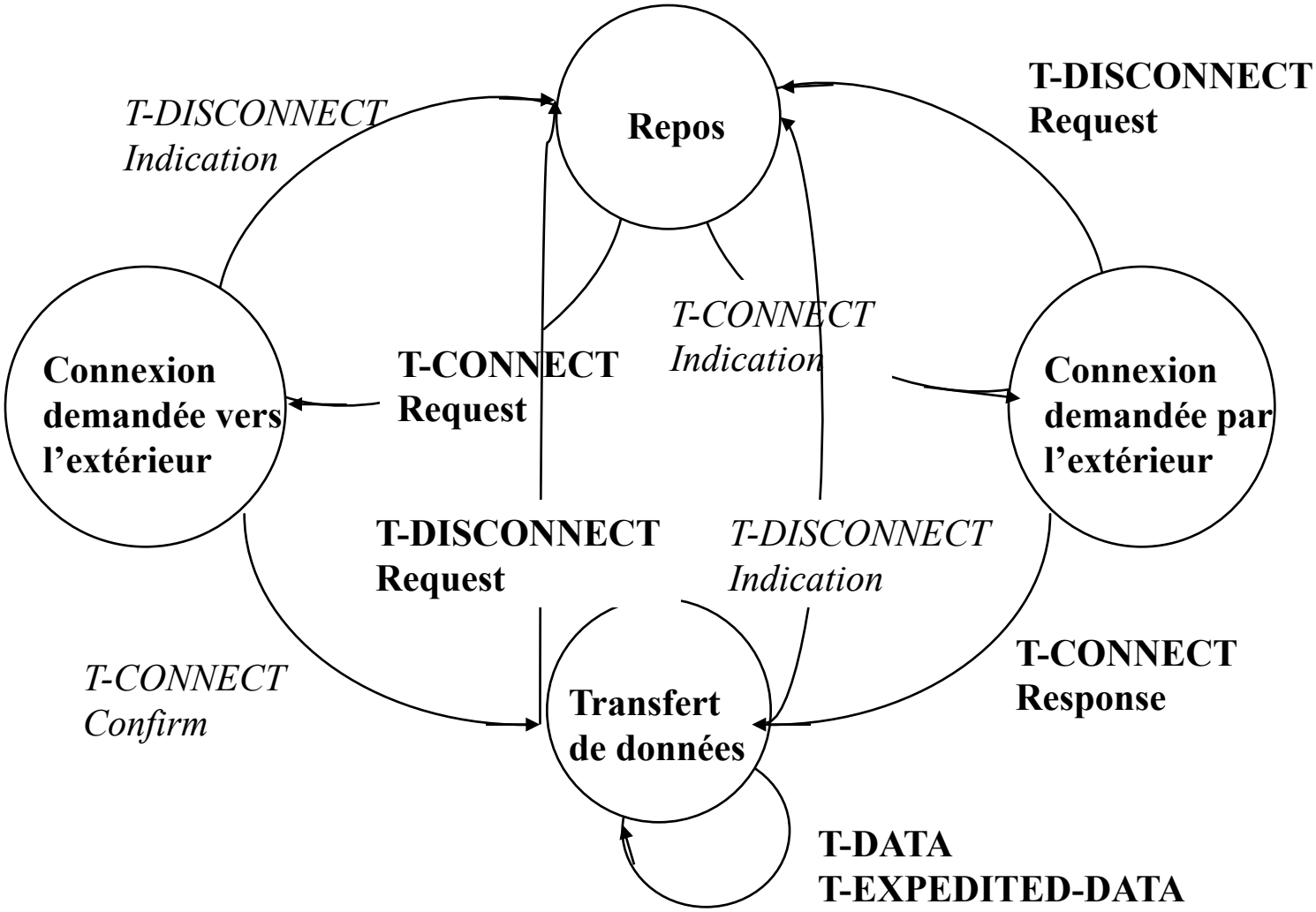
Primitives du service de transport

Phase	Service	Nom de la primitive du service	Paramètres
Etablissement	Etablissement de connexion	**T-CONNECT REQUEST** (demande de connexion) **T-CONNECT INDICATION** (indication de connexion) **T-CONNECT RESPONSE** (réponse de connexion) **T-CONNECT CONFIRM** (confirmation de connexion)	Adresse appelée Adresse appelante Option transfert de données rapides Qualité de service Données utilisateur
Transfert	Transfert de données normales	**T-DATA REQUEST** (demande d'envoi de données) **T-DATA INDICATION** (indication de données)	Données utilisateur
	Transfert de données express	**T-EXPEDITED - DATA REQUEST** **T-EXPEDITED - DATA INDICATION**	Données utilisateur
Libération	Libération de connexion	**T-DISCONNECT REQUEST**	Données utilisateur
		T-DISCONNECT INDICATION	Raison de la déconnexion Données utilisateur

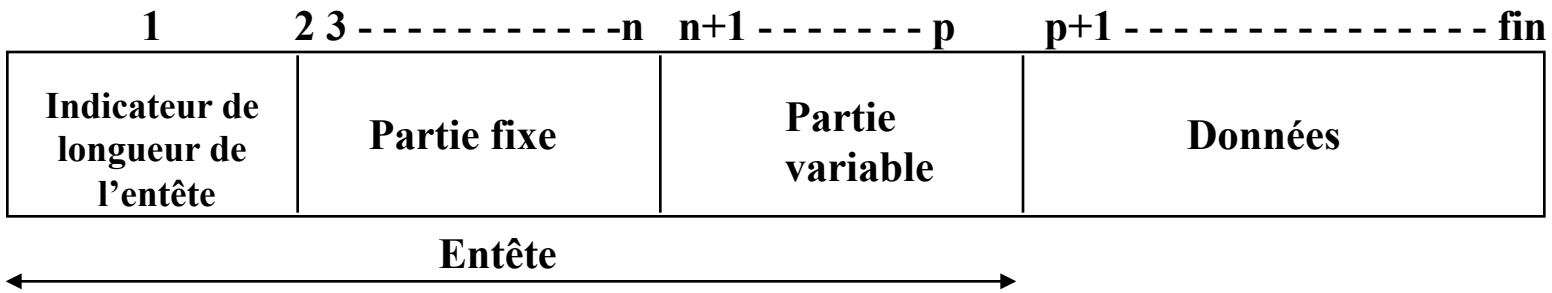
Enchaînement temporel des primitives de transport



Etat d'une extrémité de connexion de transport



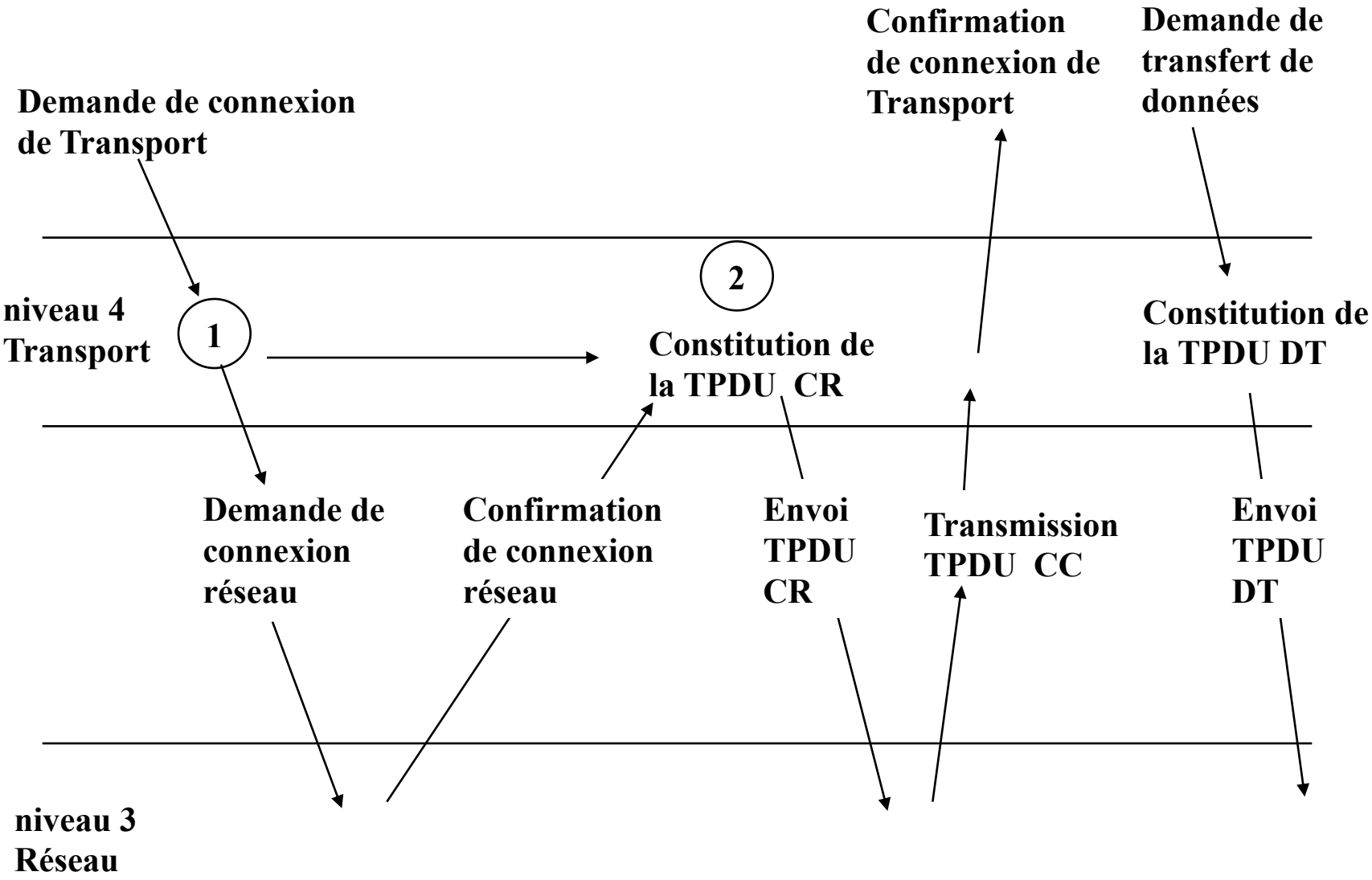
Structure d'une TPDU



Structure d'un paramètre de la partie variable



Etablissement d'une connexion de transport



La couche session : niveau 5

Synchronisation de dialogue

Etablissement et maintien des connexions de session

Echanger des données structurées

La couche session : Concepts

<u>Dialogue</u> : C'est un échange de données entre deux points de synchronisation majeur

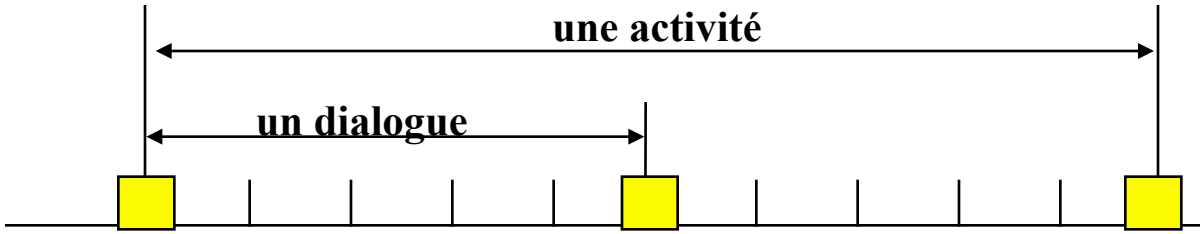
<u>Synchronisation</u> : Il s'agit ici de synchronisations structurelles qui permettent au niveau supérieur de gérer des points de reprise en cas d'anomalie au cours d'une session.

- ➢ Synchronisation majeur : Correspond à une confirmation explicite,
- ➢ Synchronisation mineur : Peuvent être posés à l'intérieur d'un dialogue et ne sont pas complètement confirmés un à un. La confirmation d'un des ces points confirme tous les points antérieurs.

<u>Activité</u> : C'est un ensemble d'un ou plusieurs dialogues. Elle correspond à un découpage logique des informations qui peut utiliser une ou plusieurs connexions de session.

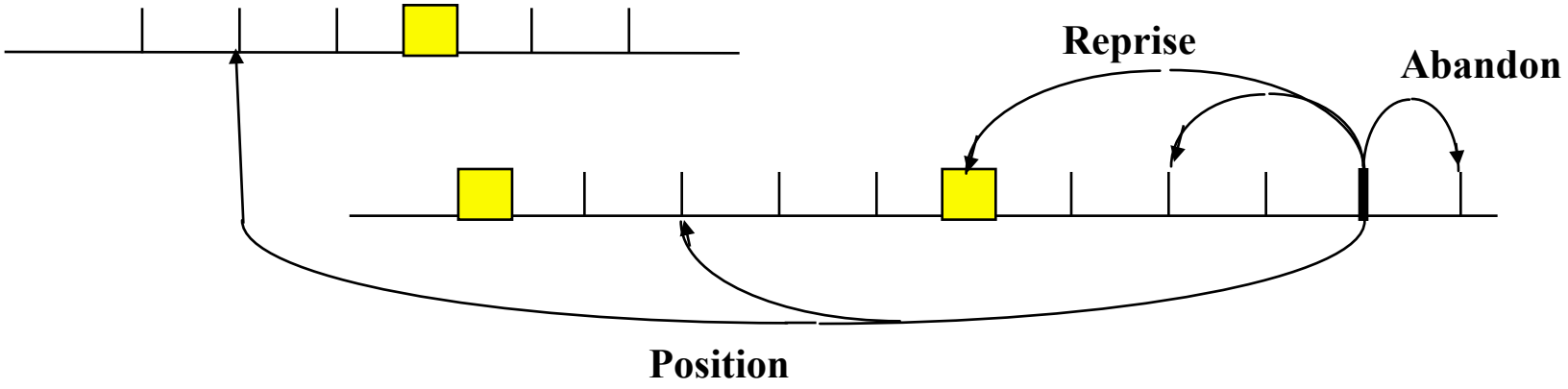
<u>Connexion de session</u> : C'est un canal logique entre deux entités voulant communiquer en utilisant les mécanismes de synchronisation de la couche session. Elle peut acheminer successivement plusieurs activités

Activité et synchronisation de la couche session



-> point de synchronisation majeur

-> point de synchronisation mineur



Services de la couche session

Phase 1 - Etablissement d'une connexion de session

- ➢ Négociation des différents services qui vont être utilisés dans la suite de la connexion,

Phase 2 - Transfert de données :

- ➢ Transfert effectif de données (normales, exprès, typées, avec confirmation),
- ➢ Gestion des jetons,
- ➢ Gestion de la synchronisation,
- ➢ Gestion des erreurs,
- ➢ Gestion des activités (démarrage, reprise, interruption, abandon).

Phase 3 - Terminaison de la connexion de session

- ➢ Terminaison ordonnée (Terminaison négociée qui arrête la connexion sans aucune perte d'information),
- ➢ ou Abandon utilisateur (Terminaison unilatérale),
- ➢ ou Abandon réseau (Terminaison unilatérale).

Services de gestion des jetons

Gèrent 4 types de jetons : jeton de données, jeton de terminaison, jeton de synchronisation mineur, jeton de synchronisation majeur et de gestion d'activité.

Les différents services sont :

- la demande de jeton,
- le don de jeton,
- la demande de contrôle de jeton -> allocation de tous les jetons à un utilisateur -> aucune activité n'est en cours.

La couche présentation : niveau 6

<u>Objectif</u> : Facilité l'échange de données quelque soit leur format et leur système de codage.

Les principaux services :

- ➢ le terminal virtuel,
- ➢ le fichier virtuel,
- ➢ la soumission de travaux à distance,
- ➢ le cryptage, l'authentification et la compression des données.

Deux grandes catégories de services : connectés et non connectés.

- ➢ Mode connecté : Mode privilégié des applications demandant un temps réel ou un accès à distance :
  - ❒ l'interactif (gestion des transactions, interrogation des bases de données, traitement distribué, ...)
  - ❒ la soumission de travaux à distance,
  - ❒ le transfert de fichier (protocole FTAM)
  - ❒ le télex,
  - ❒ ...
- ➢ Mode non connecté : Correspond aux traitements de type différés :
  - ❒ la messagerie (norme X400),
  - ❒ transfert de document par la messagerie,
  - ❒ ...

Applications

telnet

ftp

ftp anonymous et Archie

smtp avec mime

Wais (annuaire indexé)

Gopher

http (World Wide Web)

…